内部培训文档，半成品，不外传。



Appium 移动自动化测试

----- for Python

作者：虫师

ct.info



第一节　Appium 介绍

Appium 是一个自动化测试开源工具，支持 iOS 平台和 Android 平台上的原生应用，web 应用和混合应用。

所谓的“移动原生应用”是指那些用 iOS 或者 Android SDK 写的应用（Application 简称 app）。所谓的“移动 web 应用”是指使用移动浏览器访问的应用（Appium 支持 iOS 上的 Safari 和 Android 上的 Chrome）。所谓的“混合应用”是指原生代码封装网页视图——原生代码和 web 内容交互。比如，像 Phonegap，可以帮助开发者使用网页技术开发应用，然后用原生代码封装，这些就是混合应用。

重要的是，Appium 是一个跨平台的工具：它允许测试人员在不同的平台（iOS，Android）使用同一套 API 来写自动化测试脚本，这样大大增加了 iOS 和 Android 测试套件间代码的复用性。

第二节　Appium 与 Selenium

Appium 类库封装了标准 Selenium 客户端类库，为用户提供所有常见的 JSON 格式 selenium 命令以及额外的移动设备控制相关的命令，如多点触控手势和屏幕朝向。

Appium 客户端类库实现了 Mobile JSON Wire Protocol（一个标准协议的官方扩展草稿）和 W3C Webdriver spec（一个传输不可预知的自动化协议，该协议定义了 MultiAction 接口）的元素。

Appium 服务端定义了官方协议的扩展，为 Appium 用户提供了方便的接口来执行各种设备动作，例如在测试过程中安装/卸载 app。这就是为什么我们需要 Appium 特定的客户端，而不是通用的 Selenium 客户端。当然，Appium 客户端类库只是增加了一些功能，而实际上这些功能就是简单的扩展了 Selenium 客户端，所以他们仍然可以用来运行通用的 Selenium 会话。

第三节　支持多平台、多语言

appium 是跨平台的，可以用在 OSX，Windows 以及 Linux 桌面系统上运行。

appium 选择了 client-server 的设计模式。只要 client 能够发送 http 请求给 server，那么的话 client 用什么语言来实现都是可以的，这就是 appium 及 Selenium(WebDriver)如何做到支持多语言的；

appium 扩展了 webdriver 的协议，没有自己重新去实现一套。这样的好处是以前的 webdriver api 能够直接被继承过来，以前的 Selenium(WebDriver)各种语言的 binding 都可以拿来就用，省去了为每种语言开发一个 client 的工作量；

语言/框架	Github 版本库以及安装指南
Ruby	https://github.com/appium/ruby_lib
Python	https://github.com/appium/python-client
Java	https://github.com/appium/java-client
JavaScript (Node.js)	https://github.com/admc/wd
Objective C	https://github.com/appium/selenium-objective-c
PHP	https://github.com/appium/php-client
C# (.NET)	https://github.com/appium/appium-dotnet-driver
RobotFramework	https://github.com/jollychang/robotframework-appiumlibrary

第一章　Appium 环境搭建

如果是初次学习 Appium 框架的话，那么你需要做好心理准备，因为对于新手来说，这个安装过程并不太简单。

第一节　安装 Appium

Appium 官方网站：http://appium.io/

Easy setup process, run a test now.

```
> brew install node          # get node.js
> npm install -g appium     # get appium
> npm install wd            # get appium client
> appium &                  # start appium
> node your-appium-test.js
```

官方首页给出了 appium 的安装步骤。

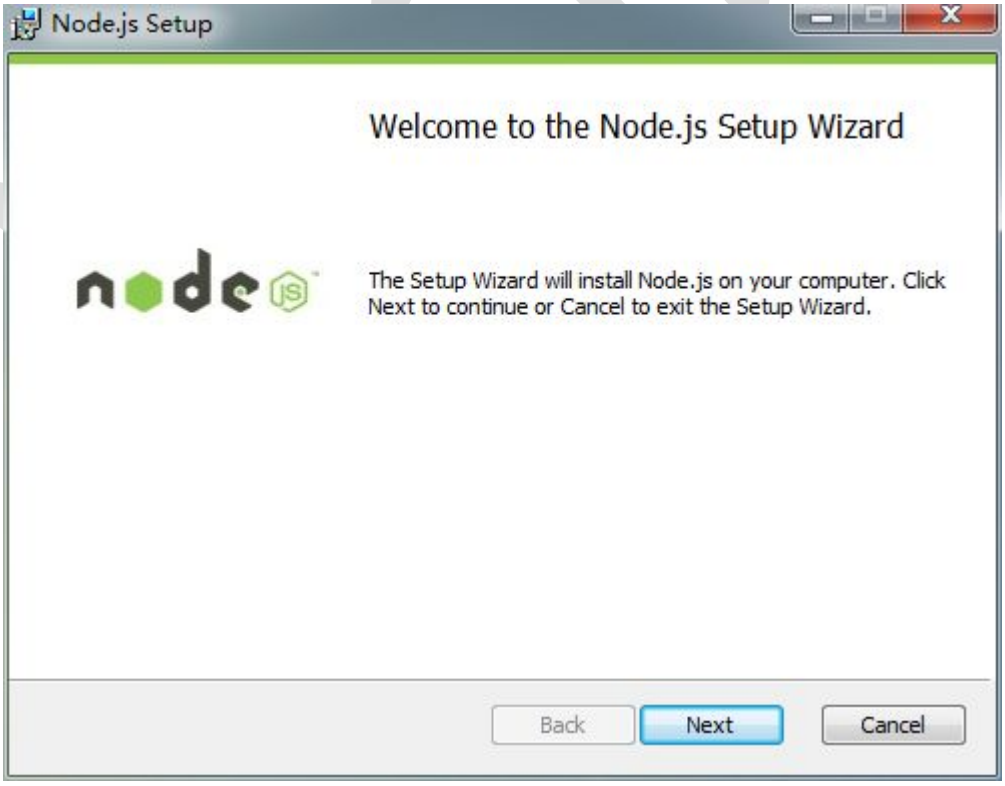
所以，我们需要先安装 node. js 。node. js 官方网站：https://nodejs.org/



Windows Installer (.msi)	32-bit	64-bit
Windows Binary (.exe)	32-bit	64-bit
Mac OS X Installer (.pkg)	Universal	
Mac OS X Binaries (.tar.gz)	32-bit	64-bit
Linux Binaries (.tar.gz)	32-bit	64-bit
SunOS Binaries (.tar.gz)	32-bit	64-bit
Source Code	node-v0.12.4.tar.gz	

根据你的操作系统选择相应的版本进行下载。这里我以 Windows 7 (64) 为例进行安装，选择 Windows installer(.msi) 64-bit 版本进行下载。

下载完成，双击进行安装，如下图。



安装完成，打开 Windows 命令提示符，敲入“npm”命令回车。

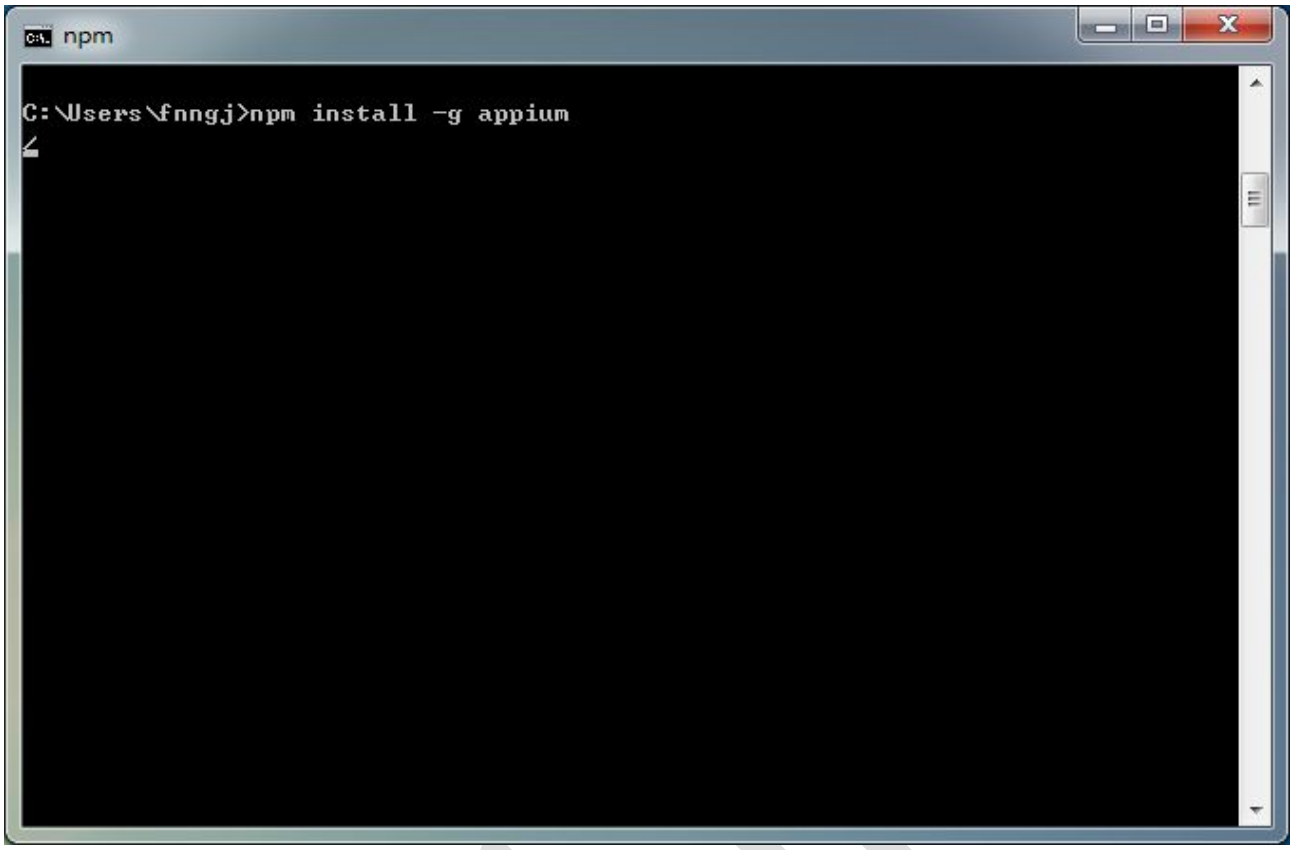


如果出现如上图信息，表示 node.js 安装成功。

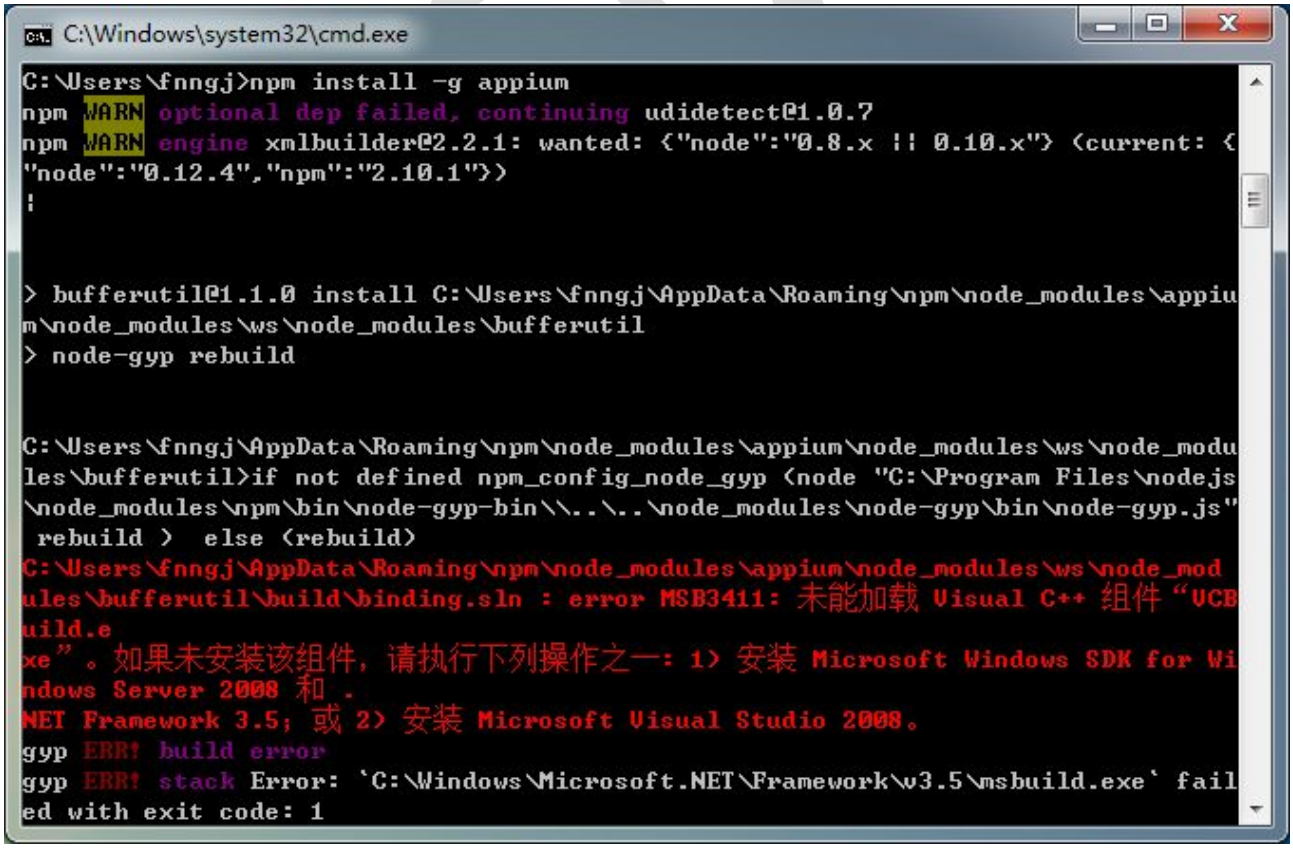
npm 是一个 node 包管理和分发工具，已经成为了非官方的发布 node 模块（包）的标准。有了 npm，可以很快的找到特定服务要使用的包，进行下载、安装以及管理已经安装的包。

下面通过 npm 安装 Appium 。

```
C:\Users\fnngj>npm install -g appium
```



当然，这种方式的 Appium 会很慢，为尊重官网上的介绍，而且大多 Appium 相关资料也会介绍这种安装安装方式。



提示笔者缺少“VCBuid.exe”。

如果未安装该组件，请执行下列操作之一:

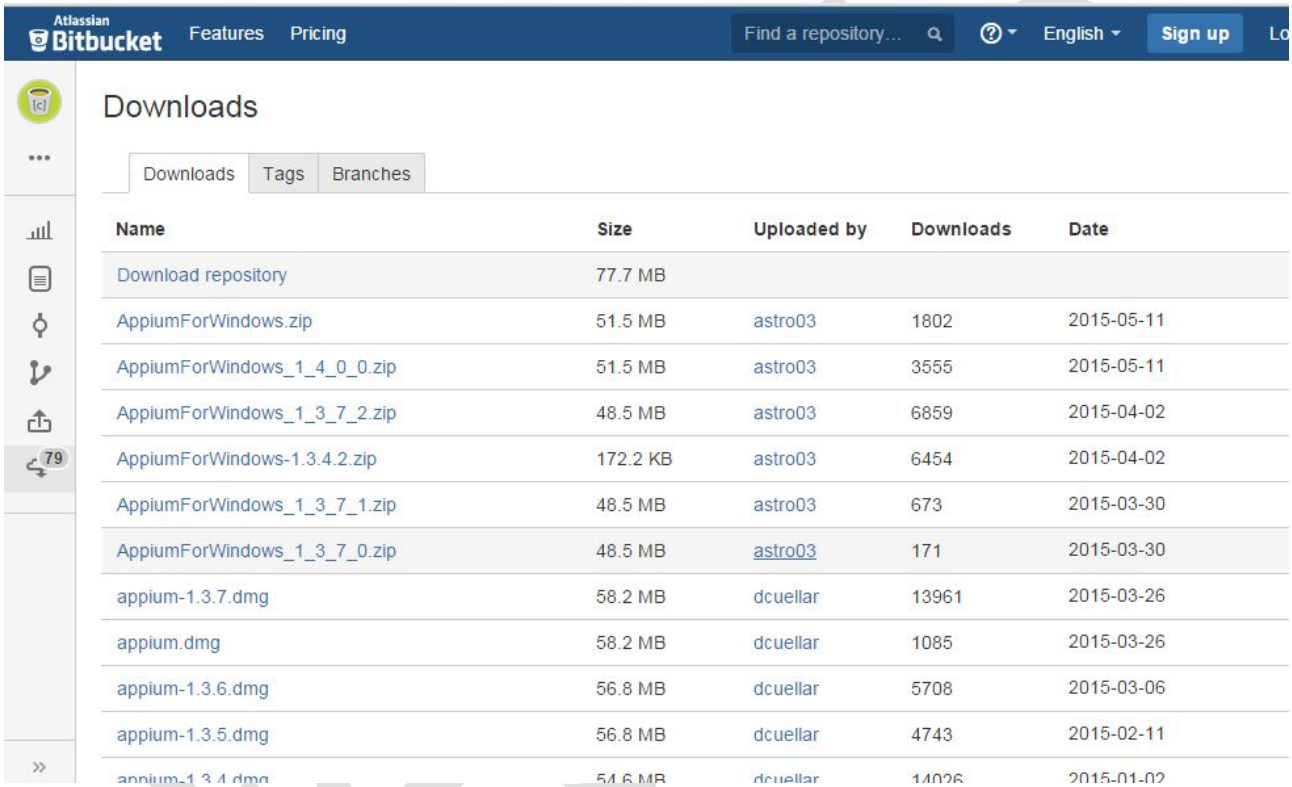
1）安装 Microsoft Windows SDK for Windows Server 2008 和 .NET Framework 3.5；

2) 安装 Microsoft Visual Studio 2008。

这是因为 Appium 是由.NET 开发的，所以，它会依赖 .NET framework 相关组件。你当然可以按照提示下载安装 1）或 2）从而再次尝试安装 Appium 。

但其实，我们可以在 Appium 官方网站上下载操作系统相应的 Appium 版本。

https://bitbucket.org/appium/appium.app/downloads/

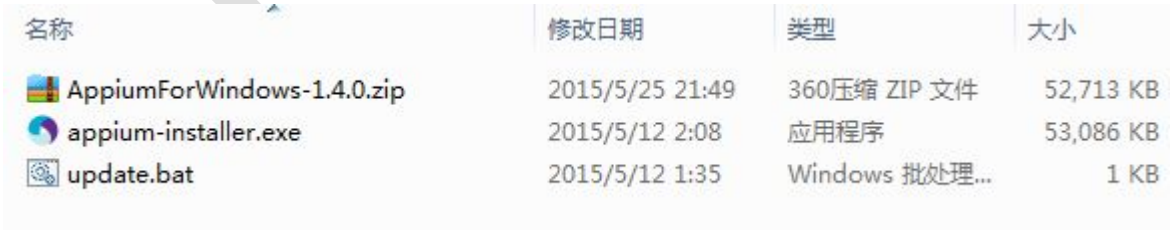


Name	Size	Uploaded by	Downloads	Date
Download repository	77.7 MB			
AppiumForWindows.zip	51.5 MB	astro03	1802	2015-05-11
AppiumForWindows_1_4_0_0.zip	51.5 MB	astro03	3555	2015-05-11
AppiumForWindows_1_3_7_2.zip	48.5 MB	astro03	6859	2015-04-02
AppiumForWindows-1.3.4.2.zip	172.2 KB	astro03	6454	2015-04-02
AppiumForWindows_1_3_7_1.zip	48.5 MB	astro03	673	2015-03-30
AppiumForWindows_1_3_7_0.zip	48.5 MB	astro03	171	2015-03-30
appium-1.3.7.dmg	58.2 MB	dcuellar	13961	2015-03-26
appium.dmg	58.2 MB	dcuellar	1085	2015-03-26
appium-1.3.6.dmg	56.8 MB	dcuellar	5708	2015-03-06
appium-1.3.5.dmg	56.8 MB	dcuellar	4743	2015-02-11

当前最新版本为 AppiumForWindows_1_4_0_0.zip ，注意这是一个 Windows 版本，如果你的电脑为 MAC 请下载 appium-1.3.7.dmg。虽然你已经看到了这些下载包，但我不保证你能下载的下来。

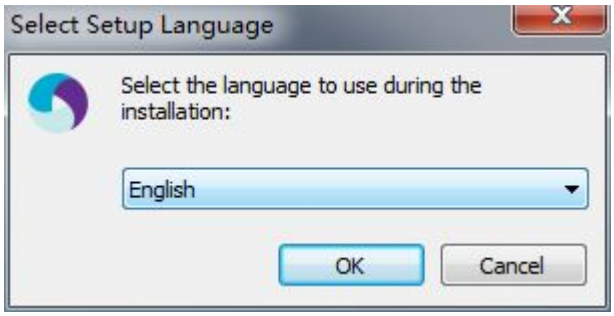
所以，再来提供一个百度网盘的下载链接：http://pan.baidu.com/s/1jGvAISu

我们以 Windows 为例，将下载的 AppiumForWindows_1_4_0_0.zip 进行解压，如下：

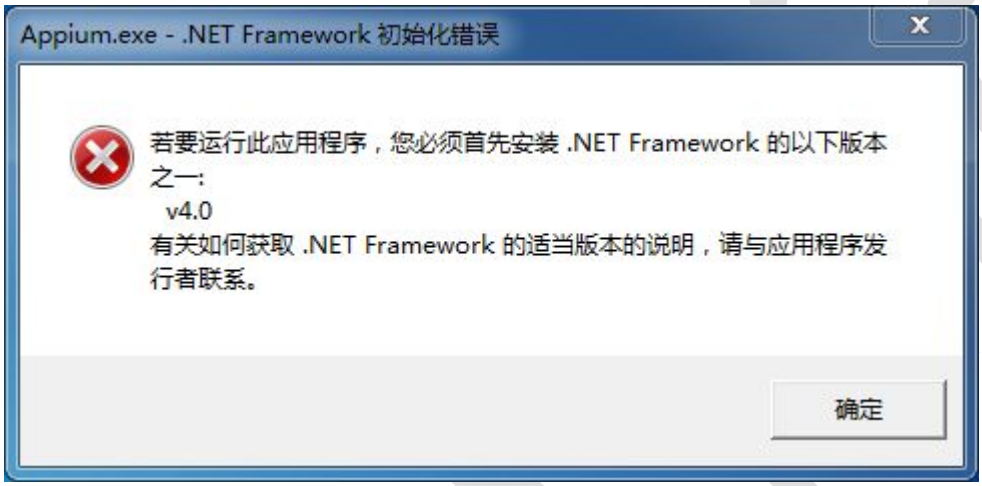


名称	修改日期	类型	大小
AppiumForWindows-1.4.0.zip	2015/5/25 21:49	360压缩 ZIP 文件	52,713 KB
appium-installer.exe	2015/5/12 2:08	应用程序	53,086 KB
update.bat	2015/5/12 1:35	Windows 批处理...	1 KB

点击“appium-installer.exe”进行安装。



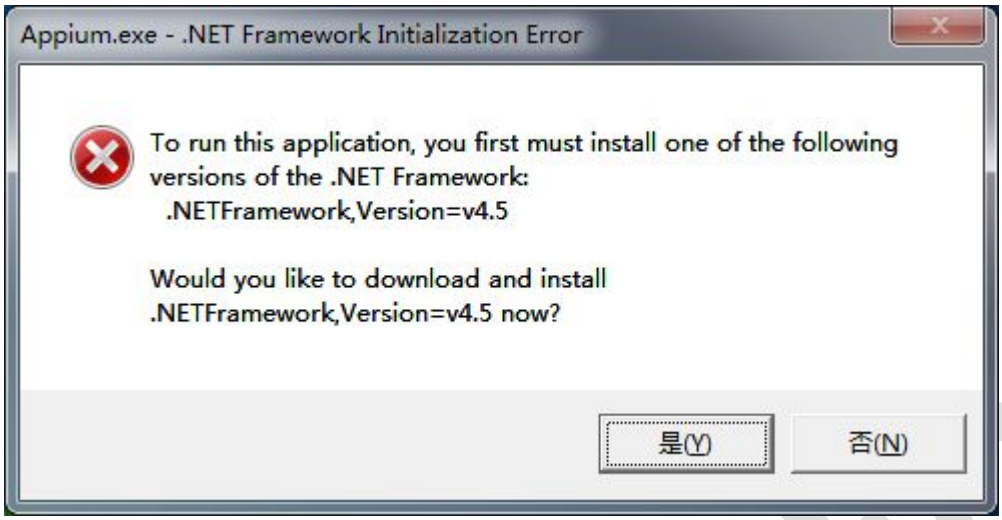
根据提示，一步一步进行安装，这里不再啰嗦。最终在会桌面上生成 Appium 图标，当我双击图标时，那么新的问题来了。



这个简单，百度“.net framework 4.0” ，百度软件中心提供该框架的下载，将其下载并安装即可。



安装完成，再次启动 Appium，再次弹出提示：



好吧，再次百度“.net framework 4.5”，进行下载安装。



好吧！Appium 终于可以启动起来了。至于 Appium 的原理和使用我们放到后面的章节进行介绍。

第二节　安装 Android 开发环境

如果你的环境是 MAC 那么可以直接跳过这一节。就像我们在用 Selenium 进行 web 自动化测试的时候一样，我们需要一个浏览器来执行测试脚本。那么移动端自动化测试，我们同样需要一部 Android 手机（手机需要链接电脑）或者 Android 模拟器。

安装 java 环境

众所周知，Android 是由 Java 语言开发的，所以想开发 Android 应用首先需要 Java 环境，所以，我们首先需要安装 Java 环境。

java 下载地址：http://www.java.com/zh_CN/download/manual.jsp

小知识：

java 环境分 JDK 和 JRE ，JDK 就是 Java Development Kit.简单的说 JDK 是面向开发人员使用的 SDK，它提供了 Java 的开发环境和运行环境。JRE 是 Java Runtime Enviroment 是指 Java 的运行环境，是面向 Java 程序的使用者，而不是开发者。

打开下载链接选择相应的版本进行下载。我们以 Windows 安装 JDK 为例，



双击下载的 JDK ，设置安装路径。这里我们选择默认安装在

D:\Program Files (x86)\Java\jdk1.8.0_25 目录下。

下面设置环境变量：

“我的电脑”右键菜单--->属性--->高级--->环境变量--->系统变量-→新建..

变量名：JAVA_HOME

变量值：D:\Program Files (x86)\Java\jdk1.8.0_25

变量名：CALSS_PATH

变量值：.;%JAVA_HOME%\lib\dt.jar;%JAVA_HOME%\lib\tools.jar;

找到 path 变量名—>“编辑”添加：

变量名：PATH

变量值：%JAVA_HOME%\bin;%JAVA_HOME%\jre\bin;

在 Windows 命令提示符下验证 java 是否成功：

cmd.exe

```
……
C:\Users\fnngj>java
用法: java [-options] class [args...]
           (执行类)
   或  java [-options] -jar jarfile [args...]
           (执行 jar 文件)
其中选项包括:
    -d32          使用 32 位数据模型 (如果可用)
    -d64          使用 64 位数据模型 (如果可用)
    -server       选择 "server" VM
    -hotspot      是 "server" VM 的同义词 [已过时]
                默认 VM 是 server.
……

C:\Users\fnngj>javac
用法: javac <options> <source files>
其中, 可能的选项包括:
  -g                         生成所有调试信息
  -g:none                    不生成任何调试信息
  -g:{lines,vars,source}     只生成某些调试信息
  -nowarn                    不生成任何警告
  -verbose                   输出有关编译器正在执行的操作的消息
  -deprecation               输出使用已过时的 API 的源位置
  -classpath <路径>            指定查找用户类文件和注释处理程序的位置
  -cp <路径>                   指定查找用户类文件和注释处理程序的位置
  ……
```

itest.inc

java 命令可以运行 class 文件字节码。

javac 命令可以将 java 源文件编译为 class 字节码文件

安装 android adt & SDK

Android SDK：

Android SDK 提供了你的 API 库和开发工具构建，测试和调试应用程序，Android。简单来讲，Android SDK 可以看做用于开发和运行 Android 应用的一个软件。

扩展理解：

ADT(Android Development Tools)： 目前 Android 开发所用的开发工具是 Eclipse，在 Eclipse 编译 IDE 环境中，安装 ADT，为 Android 开发提供开发工具的升级或者变更，简单理解为在 Eclipse 下开发工具的升级下载工具。adt 只是一个 eclipse 的插件，里面可以设置 sdk 路径

SDK(Software Development Kit)： 一般是一些被软件工程师用于为特定的软件包、软件框架、硬件平台、操作系统等建立应用软件的开发工具的集合。在 Android 中，他为开发者提供了库文件以及其他开发所用到的工具。简单理解为开发工具包集合，是整体开发中所用到的工具包，如果你不用 Eclipse 作为你的开发工具，你就不需要下载 ADT，只下载 SDK 即可开发。SDK 可以自己编译，在 linux 环境下通过 make 命令进行，耗时比较长，需要有耐心哦亲。。。然后我们可以把自己编译的 SDK 通过 ADT 导入 eclipse。在此基础上可以对源码包进行修改，比如修改 android system/app/phone.apk 中的源码，

通过官方地址下载：http://developer.android.com/sdk/index.html

当然，我不能保证你可以访问到这个地址，原因你懂的！

这里提供 adt-bundle 下载链接：

http://dl.google.com/android/adt/adt-bundle-windows-x86-20140702.zip

https://dl.google.com/android/adt/adt-bundle-windows-x86_64-20140702.zip

http://dl.google.com/android/adt/adt-bundle-mac-x86_64-20140702.zip

http://dl.google.com/android/adt/adt-bundle-linux-x86-20140702.zip

http://dl.google.com/android/adt/adt-bundle-linux-x86_64-20140702.zip

Android-sdk 下载地址：

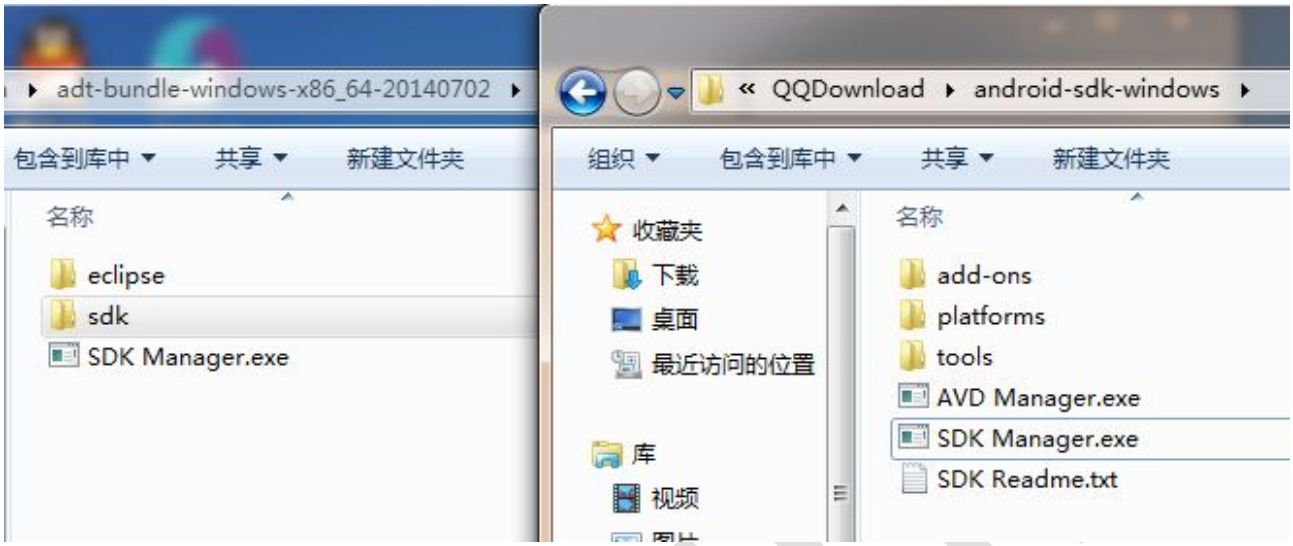
http://dl.google.com/android/android-sdk_r23.0.2-windows.zip

http://dl.google.com/android/installer_r23.0.2-windows.exe

http://dl.google.com/android/android-sdk_r23.0.2-macosx.zip

http://dl.google.com/android/android-sdk_r23.0.2-linux.tgz

根据我的验证 adt 和 sdk 都可以用来运行 android 模拟器，所以下载哪个都可以。下载链接中已经标识了系统信息，请根据自己的操作系统下载相应的版本。



Adt 和 sdk 解压包对比，相对 adt 的内容要多一些，如果你不做 android 开发的话，两者都可以。

下面设置 Android 环境变量，方法与 java 环境变量类似。我本机的目录结果为：

D:\android\android-sdk-windows

下面设置环境变量：

“我的电脑”右键菜单--->属性--->高级--->环境变量--->系统变量->新建..

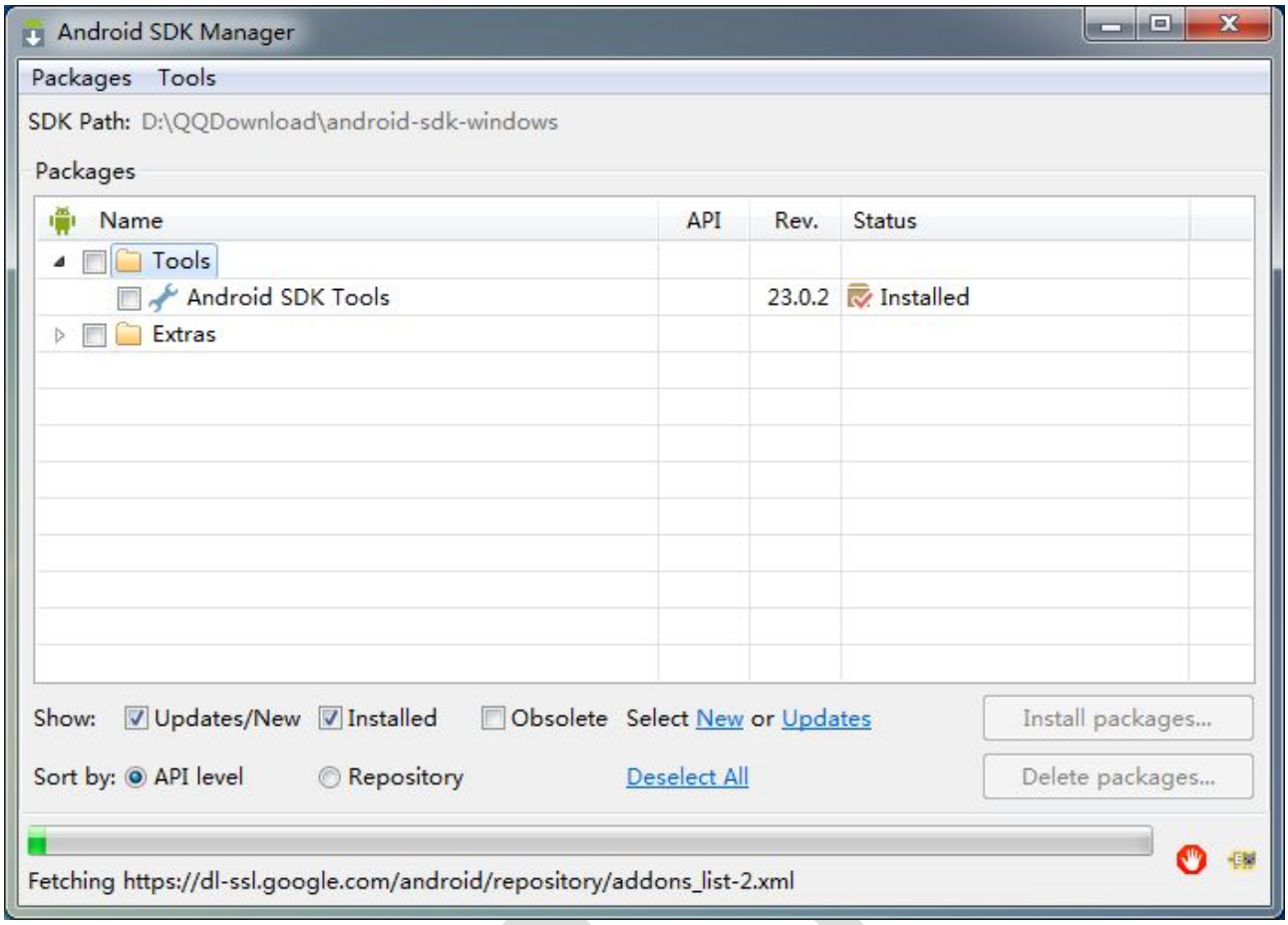
变量名：ANDROID_HOME 变量值：D:\android\android-sdk-windows

找到 path 变量名—>“编辑”添加：

变量名：PATH 变量值：;%ANDROID_HOME%\platform-tools;%ANDROID_HOME%\tools;

SDK Manager 安装模拟器

双击启动 SDK Manager.exe 程序。

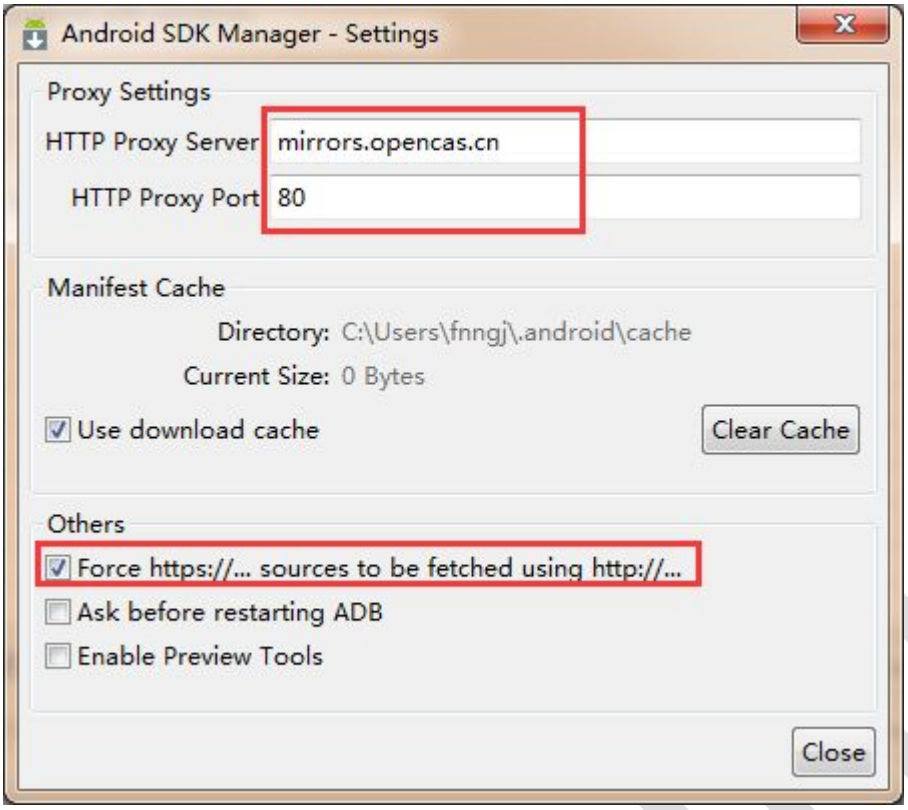


当然，我不能保证你 SDK 可以更新。所以，这里推荐一个网站：http://www.androiddevtools.cn/

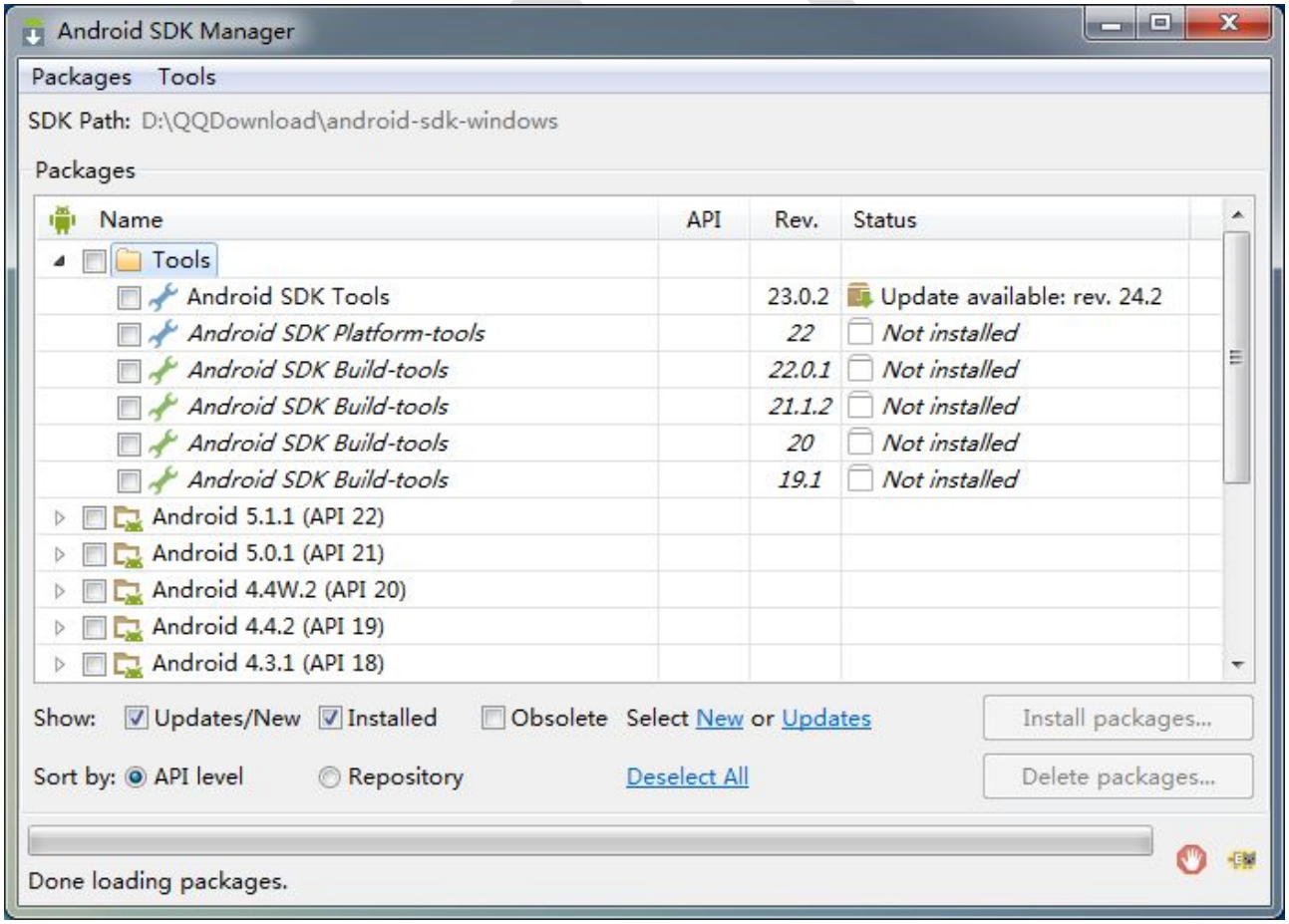
它提供了国内的代理，以及各种开发工具的安装。

在 Android SDK Manager 的菜单栏上点击“Tools”---->“Options...”设置相关代码，如下图：

内部培训文档，半成品，不外传。



然后，就可以更新了：

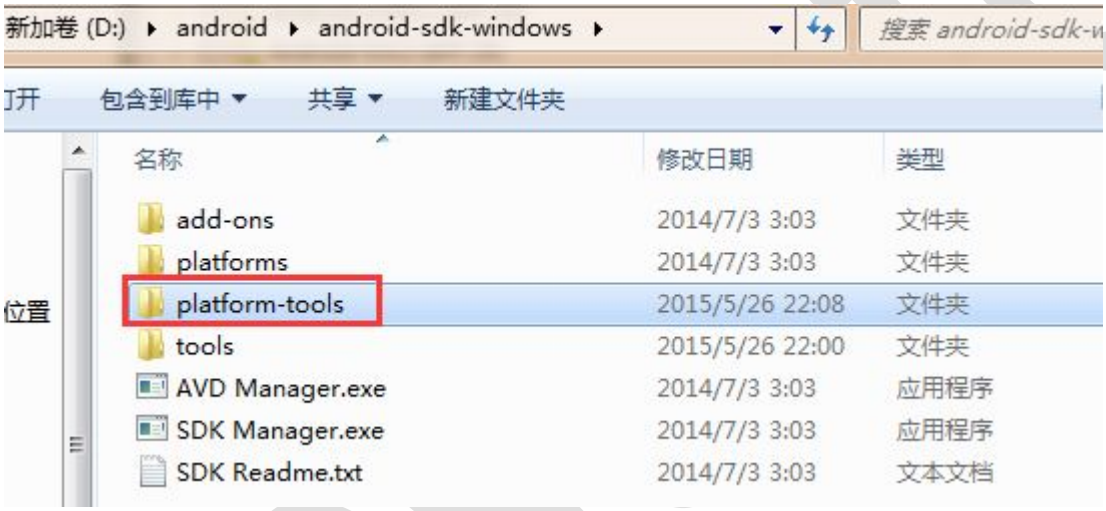


安装 SDK Platform-Tools

Android SDK Platform-tools 是版本有区别的工具文件夹，里面有 adb 、aapt、fastboot 等工具包。在 AndroidDevTools.cn 网站上找到 Android SDK Platform-tools 下载链接。

版本号	Windows	Mac OSX	Linux
platform-tools-r22	platform-tools_r22-windows.zip	platform-tools_r22-mac.zip	platform-tools_r22-linux.zip
platform-tools-r21	platform-tools_r21-windows.zip	platform-tools_r21-mac.zip	platform-tools_r21-linux.zip
platform-tools-r20	platform-tools_r20-windows.zip	platform-tools_r20-mac.zip	platform-tools_r20-linux.zip

把解压出来的 platform-tools 文件夹放在 android sdk 根目录下，并把 adb 所在的目录添加到系统 PATH 路径里(这个我们在设置 android 环境变量时已经添加)。



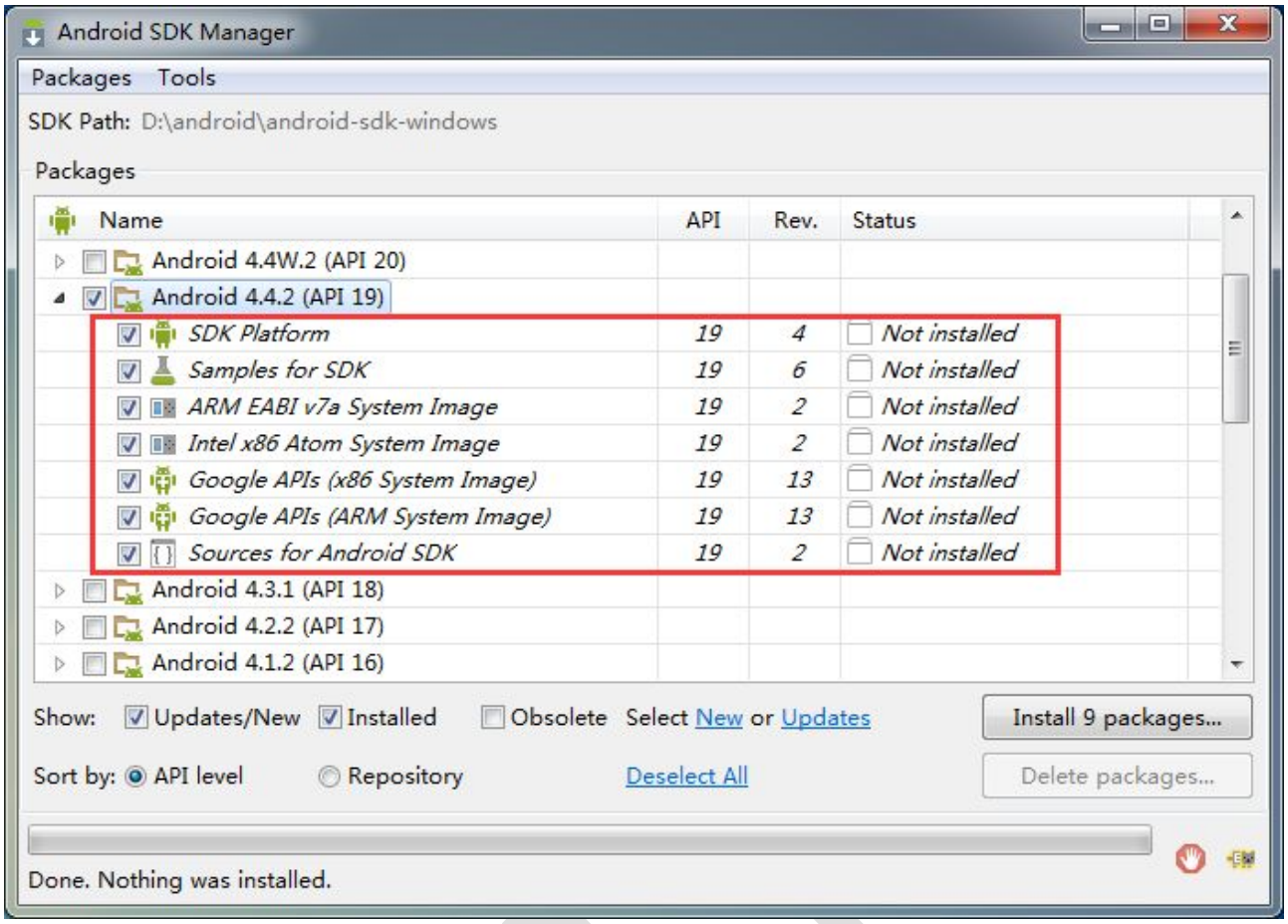
打开 Windows 命令提示符，通过“appium-doctor”命令检查 appium 环境。



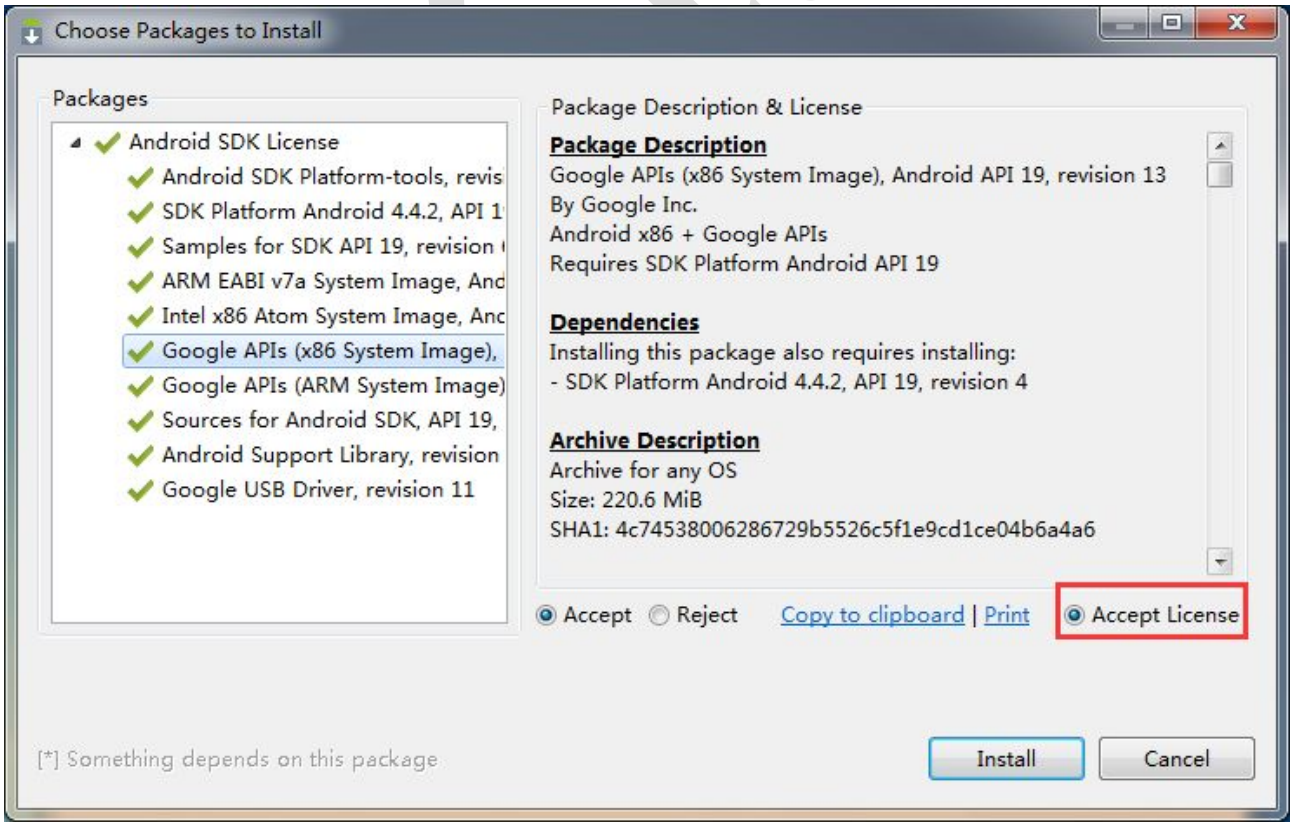
如果出现“All Checks were successful”的提示，说明基本环境已经搞好。你以为已经大工告成了么？还早着呢！！

第三节　安装 Android 模拟器

现就可以下载一个 Android 版本的模拟器了。我这里以 Android 4.4.2 版本为例。



如上图，勾选所需要安装的工具，点击右下角“Install x packages...”



选择“Accept License” 选项，点击“Install”按钮时行安装。

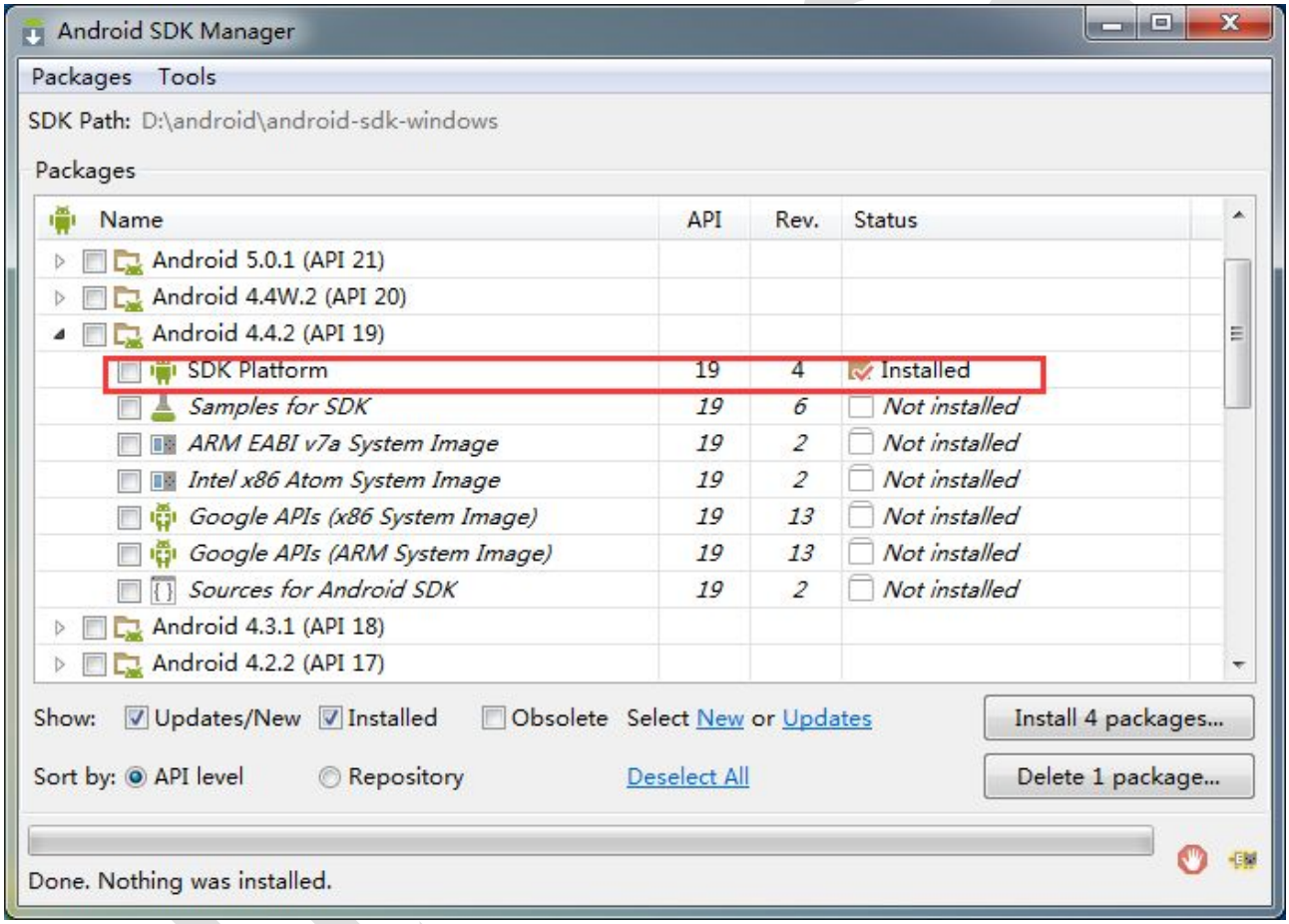
但是，你可能会发现这种方法会提示“Download interrupted: URL not found.”这样的错误，那么我们只能将这些工具单个的下载安装了。

好吧!再次感谢 AndroidDevTools.cn 网站的共享。以下下载链接均有其共享。

一、安装 SDK platform

android 4.4.2 ： http://pan.baidu.com/s/1eQf8ZgI

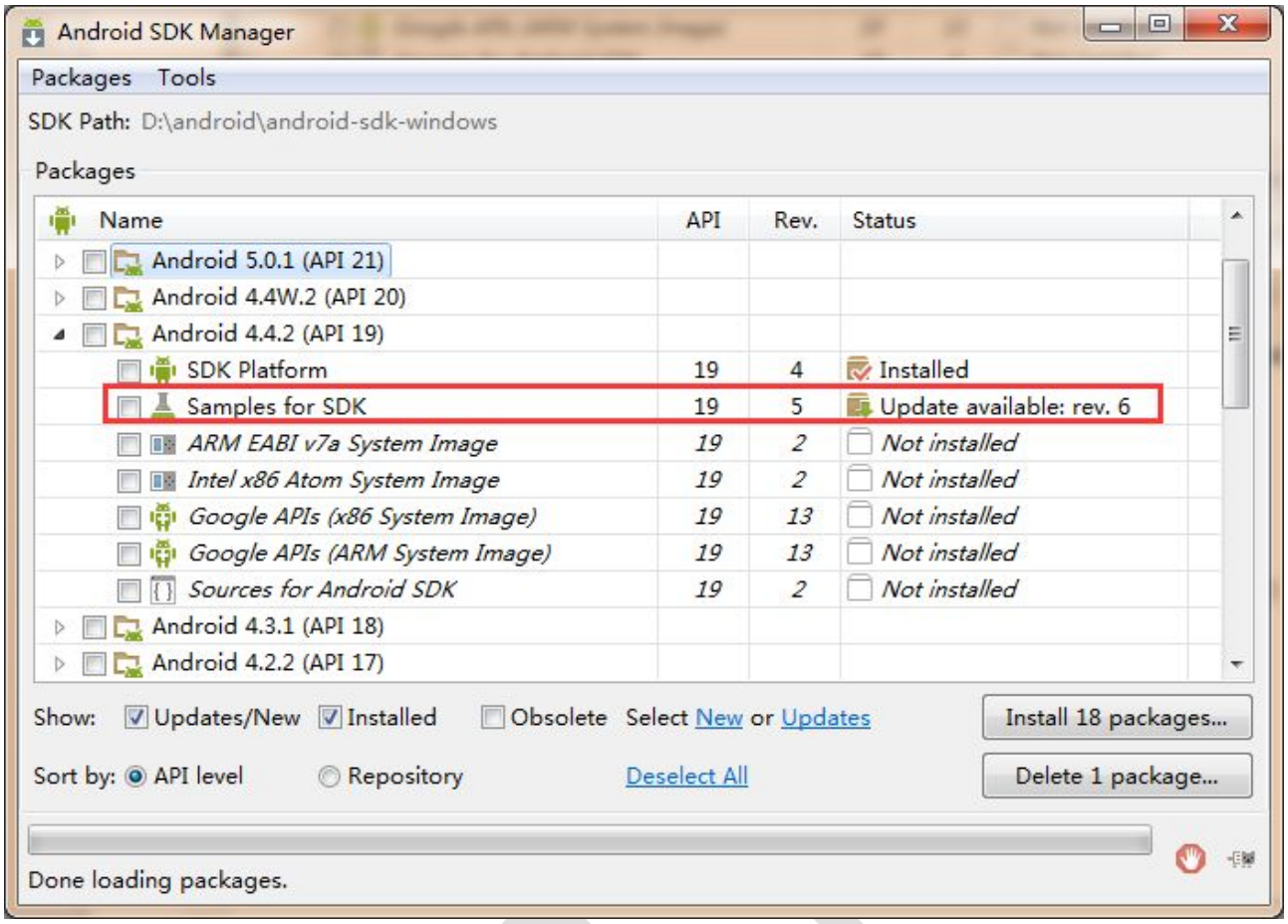
这是 Android 开发所需的 sdk，下载并解压后，将解压出的整个文件夹复制或者移动到 .../android-sdk-windows/platforms/文件夹，然后重新打开 SDK Manager.exe



二，安装 Samples for SDK

android 4.4.2 ： http://pan.baidu.com/s/1dDeSKt7

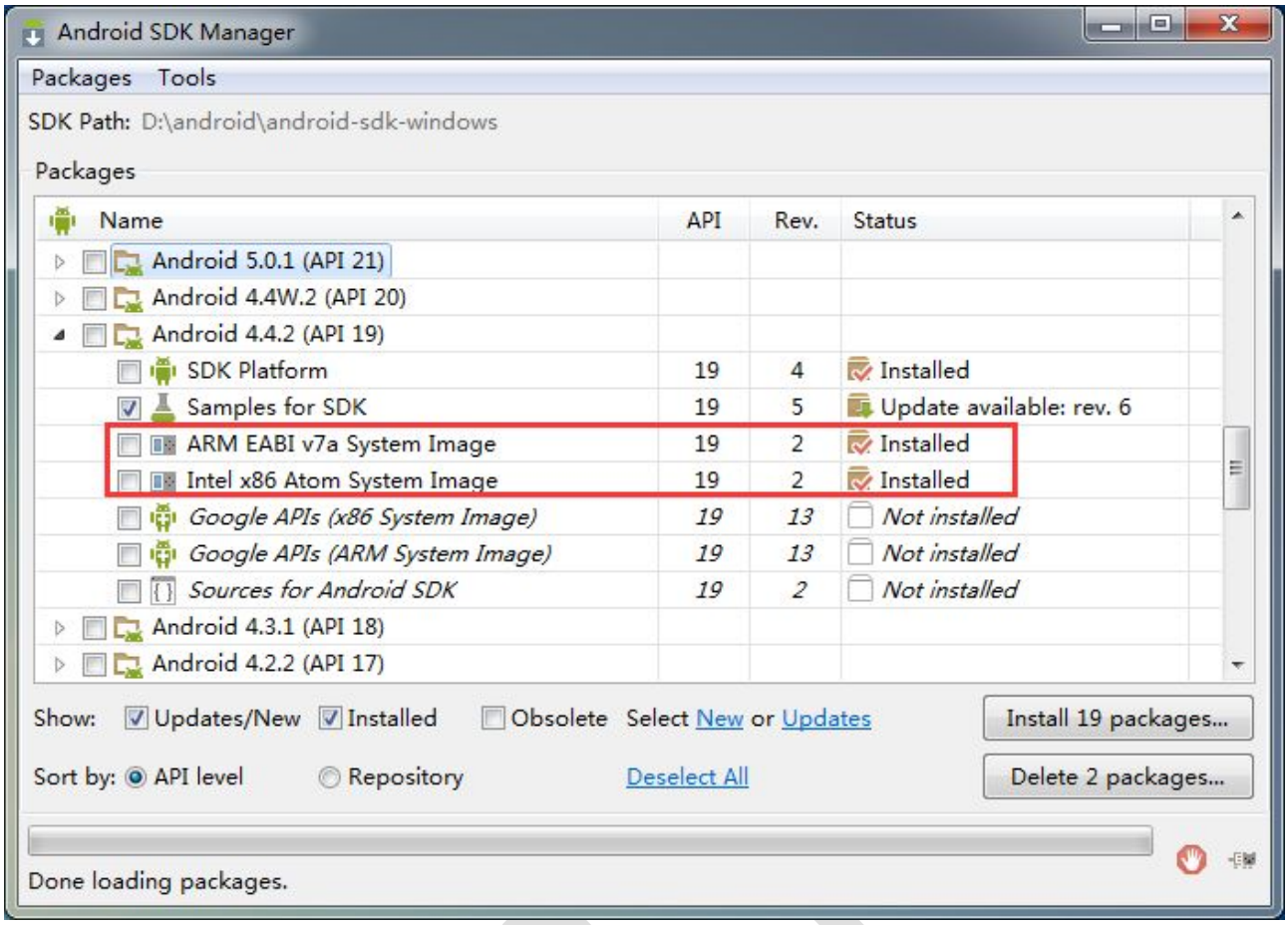
这是 Android SDK 自带的示例代码，下载并解压后，将解压出的整个文件夹复制或者移动到 .../android-sdk-windows/samples 文件夹下，然后重启 SDK Manager.exe。



三，安装 SDK System images

android 4.4.2 : http://pan.baidu.com/s/1i3Jwhed

这是在创建模拟器时需要的 system image，也就是在创建模拟器时 CPU/ABI 项需要选择的，下载并解压后，将解压出的整个文件夹复制或者移动到.../android-sdk-windows/system-images 文件夹下即可， 如果没有 system-images 目录就先创建此文件夹，然后重新打开 SDK Manager.exe。

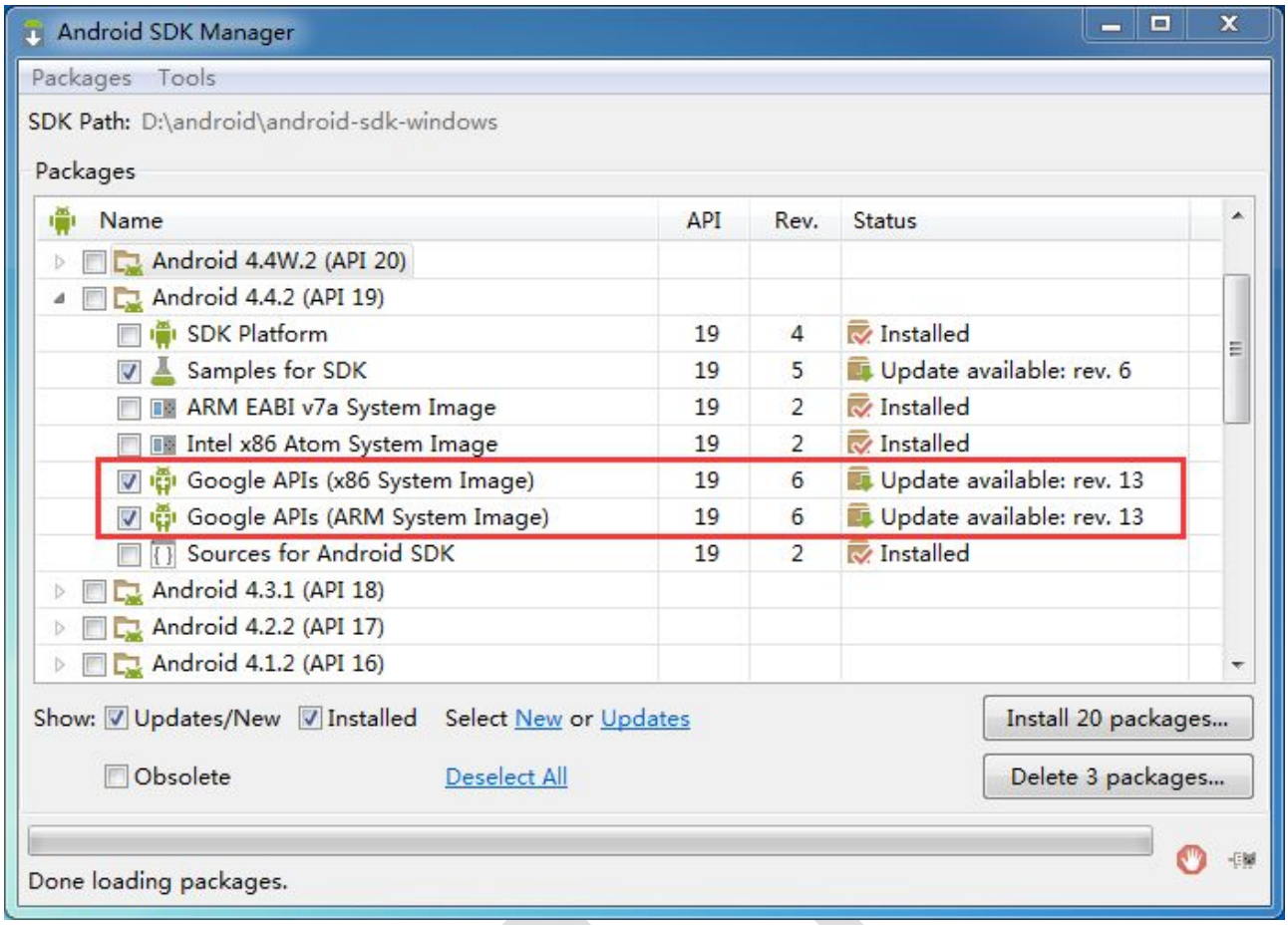


四， GoogleMap APIs SDK

android 4.4.2 （ARM）： http://pan.baidu.com/s/1bno0mFt

android 4.4.2 （x86）： http://pan.baidu.com/s/1jGgKyZc

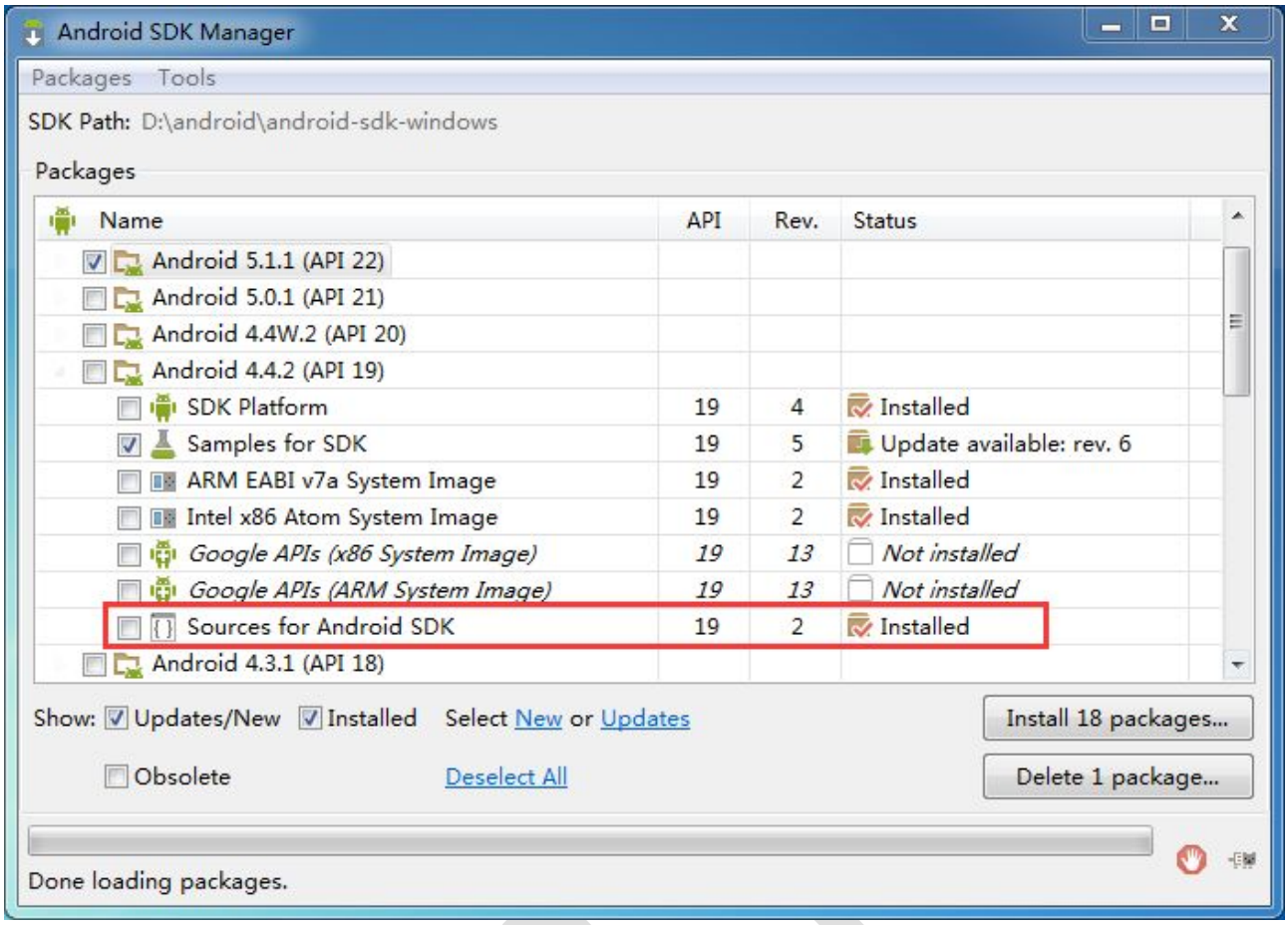
这是 GoogleMap APIs SDK，下载并解压后，将解压出的整个文件夹复制或者移动到 .../android-sdk-windows/add-ons 文件夹下，然后打开 SDK Manager



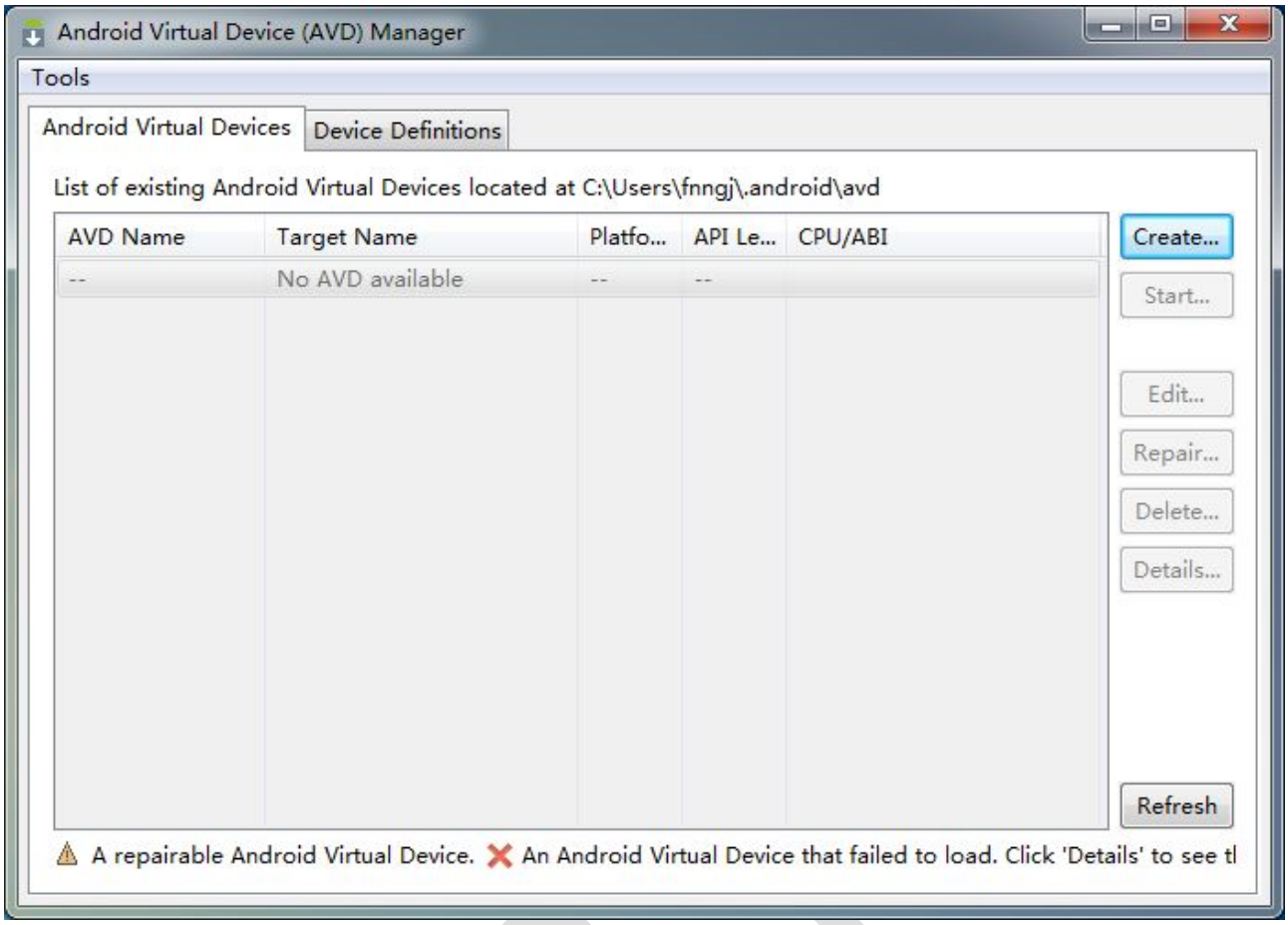
五，Android Framework Source Code

android 4.4.2 : http://pan.baidu.com/s/1hqGGrVA

这是 Android Framework Source Code，下载并解压后，将解压出的整个文件夹复制或者移动到.../android-sdk-windows/sources 文件夹下，然后重新打开 SDK Manager.exe。

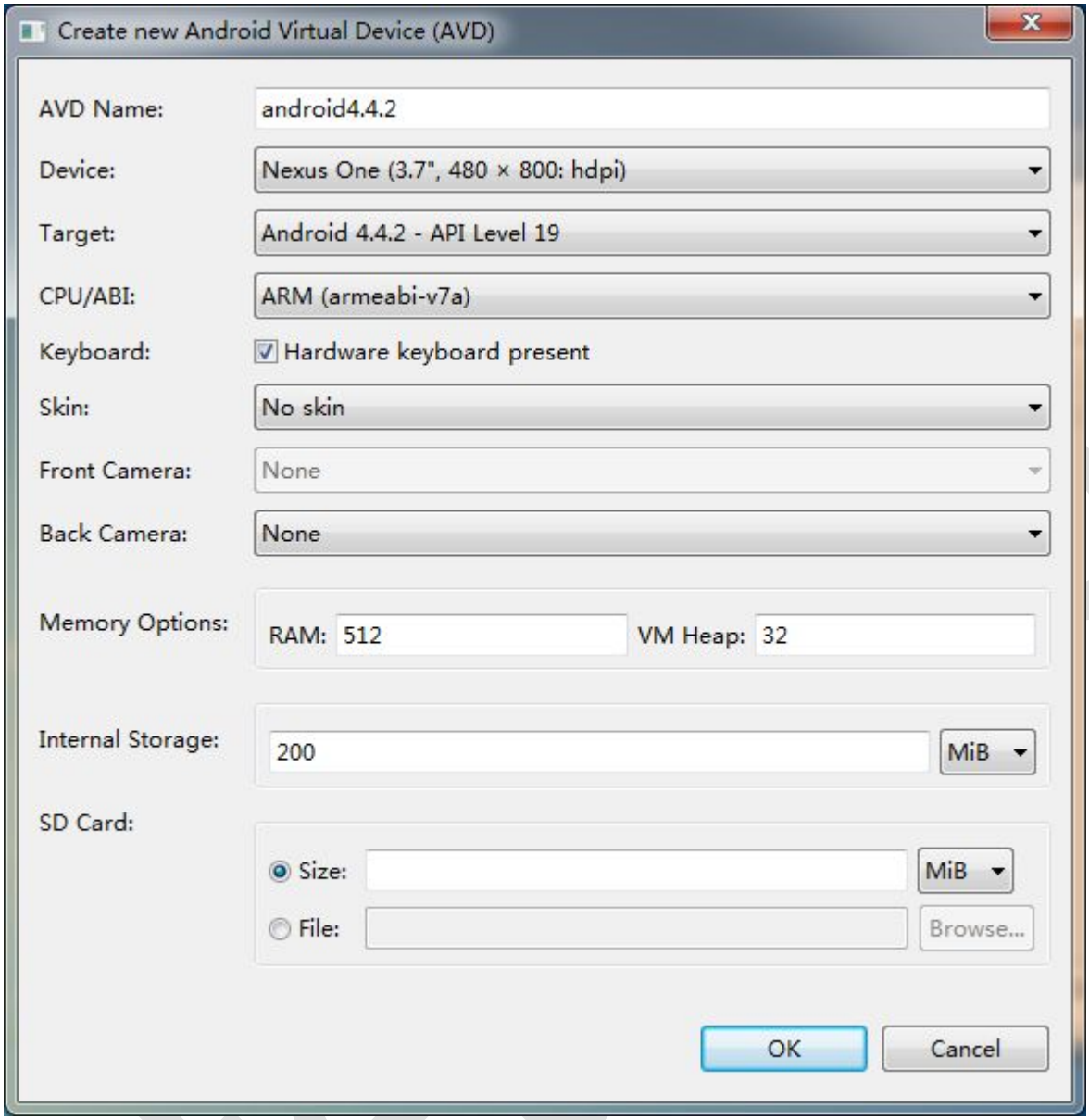


上面的每一个工具都不是必须安装，为了保险起见，我们每一个包都做了安装。下面双击“AVD Manager.exe”创建 android 模拟器。



点击“Create....”

itest.



如果显示屏分辨率比较底的话，尽量选择低分辨率的“Device”。

点击“OK”，在 AVD Manager 窗口，点击“Start...”按钮启动 android 模拟器。



因为新虚拟机没了实体键，所以我们可以利用键盘按键来操作 android 虚拟机。

模拟器按键	键盘按键
后退	ESC
菜单	F1 或 Page Up
开始	F2 或 Page Down
呼叫	F3

挂断	F4
电源按钮	F7
禁止/启用所有网络	F8
开始跟踪	F9
停止跟踪	F10
旋转屏幕（横/竖屏切换）	Ctrl+F11
主页	HOME
方向键 左/上/右/下	小键盘 4/8/6/2
方向键 中心键	小键盘 5
调低音量	小键盘 负号(-)
调高音量	小键盘 加号(+)

test.info

第四节 安装 Appium Client

Appium Client 是对 webdriver 原生 api 的一些扩展和封装。它可以帮助我们更容易的写出用例，写出更好懂的用例。

Appium Client 是配合原生的 webdriver 来使用的，因此二者必须配合使用缺一不可。

在本文档的开始已经介绍 Appium 支持多种编程语言编写自动化测试脚本，由于我个人熟悉 Python 语言，所以选择 Python 语言来编写自动化测试脚本。

访问 Python 官方网站：https://www.Python.org/

下载最新版本的 Python2，截止作者发稿，最新版本为 Python2.7.10 版本。读者根据自己的平台选择相应的版本进行下载；对于 Windows 用户来说，如果你的系统是 32 位的请选择 x86 版本，如果是 64 位系统请选择 64 版本进行下载。下载完成会得到一个以.msi 为后缀名的文件，双击进行安装。如图 2.1。



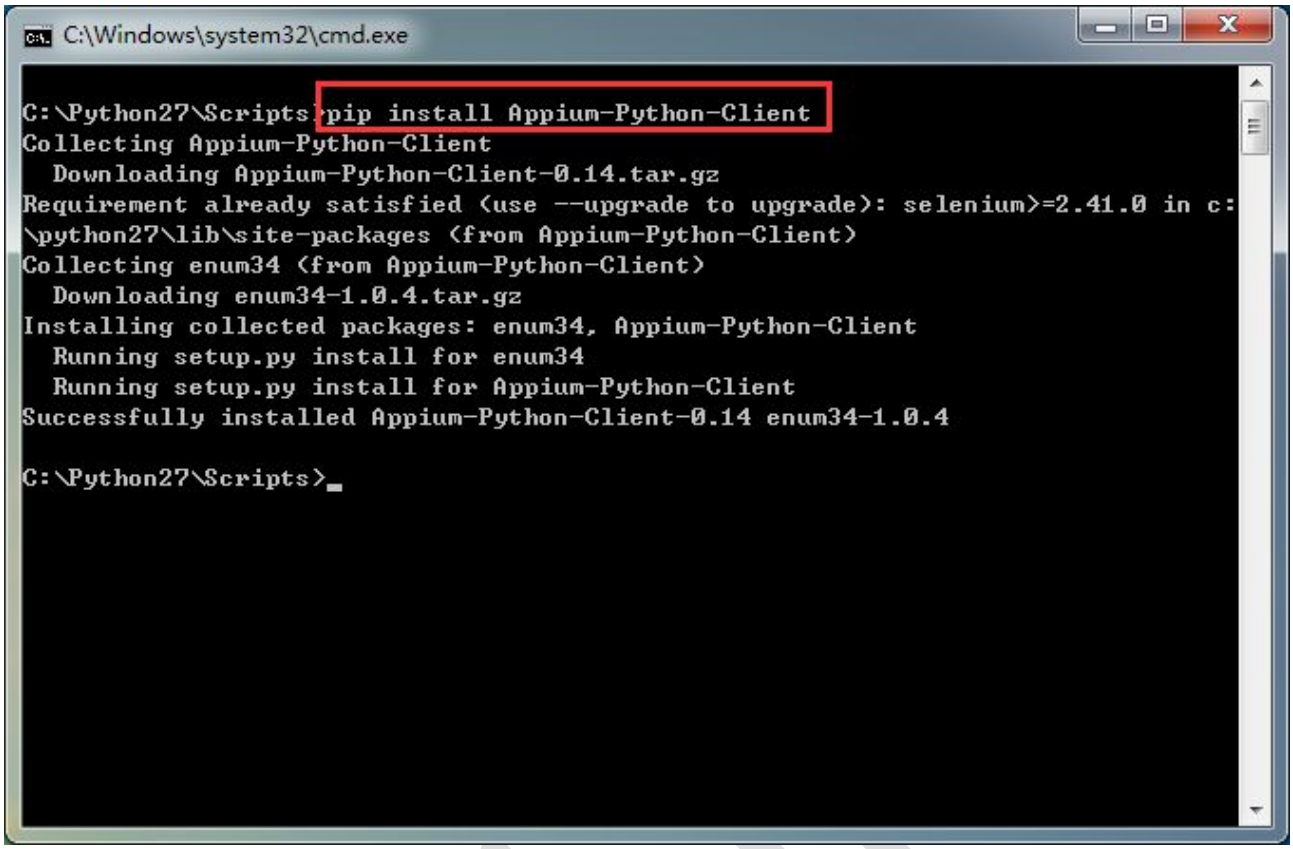
pip 是一个安装和管理 Python 包的工具，通过 pip 去安装 Python 包将变得十分简单，我们将省去了搜索--查找版本--下载--安装等繁琐的过程

推荐通过 pip 进行安装“Appium-Python-Client”。

cmd.exe

```
C:\package\setuptools-7.0>Python install Appium-Python-Client
```

itest.info



第五节　第一个自动化脚本

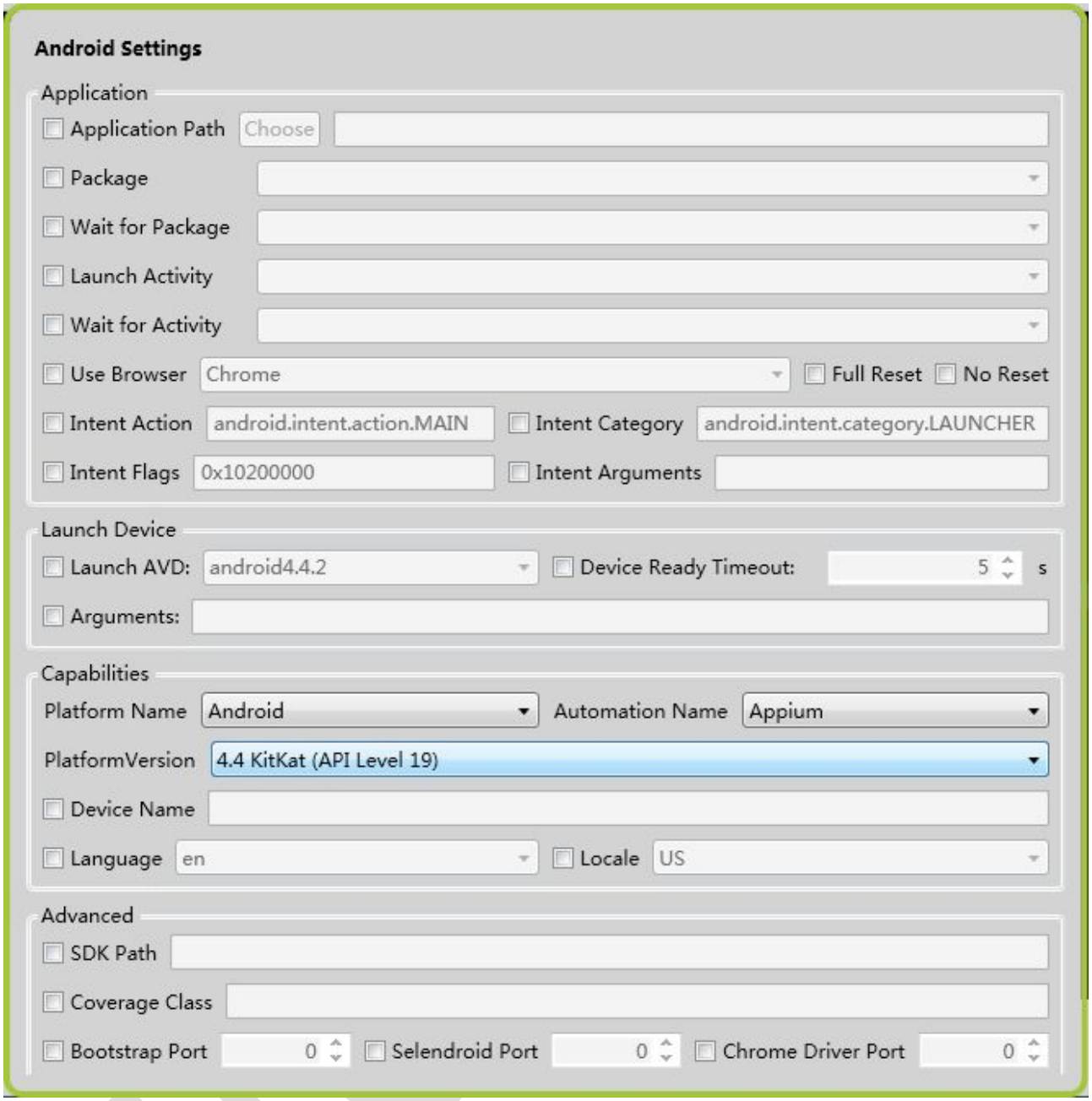
首先启动 android 模拟器。



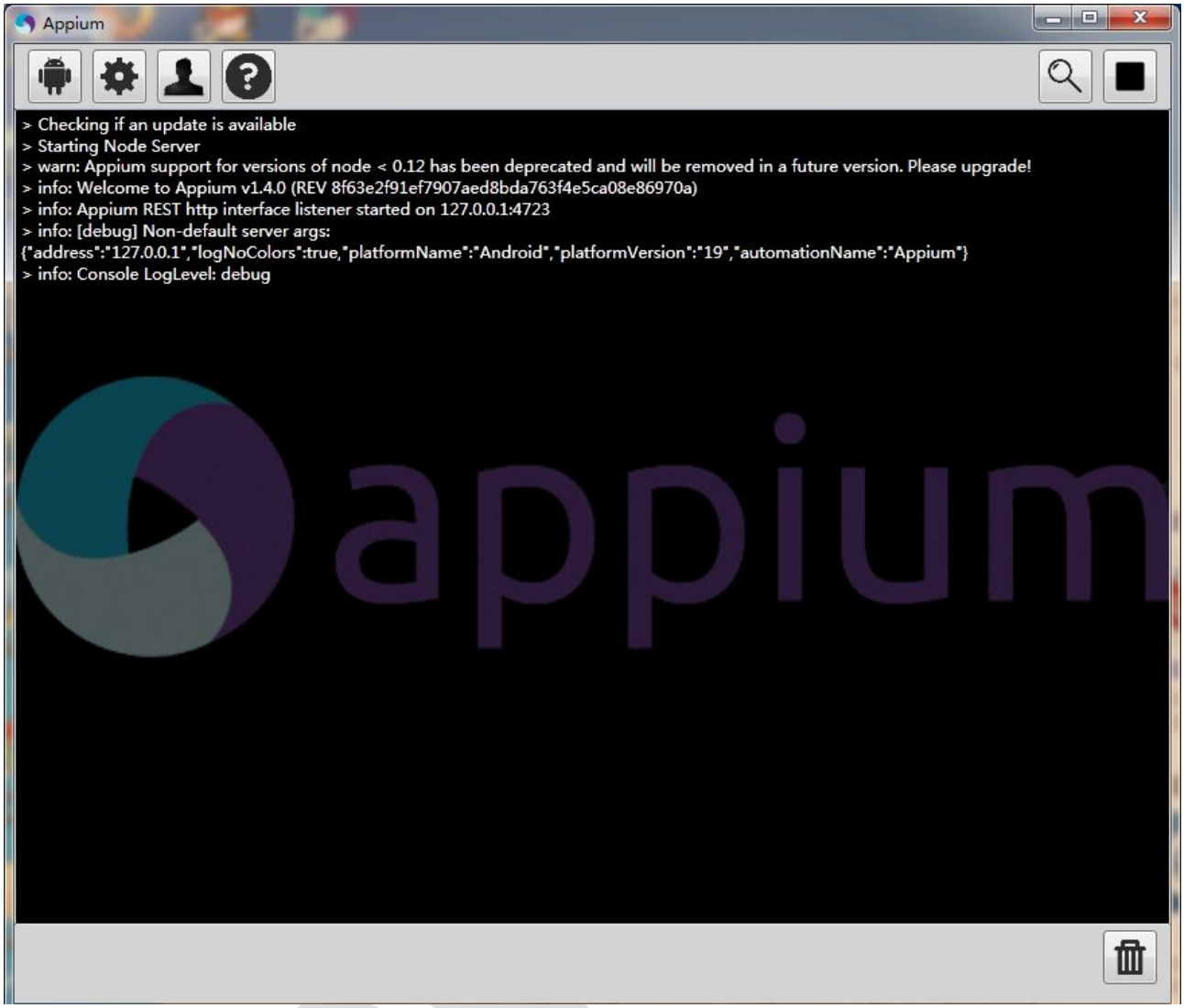
第二步，启动 Appium。



点击左上角有“机器人”的按钮：

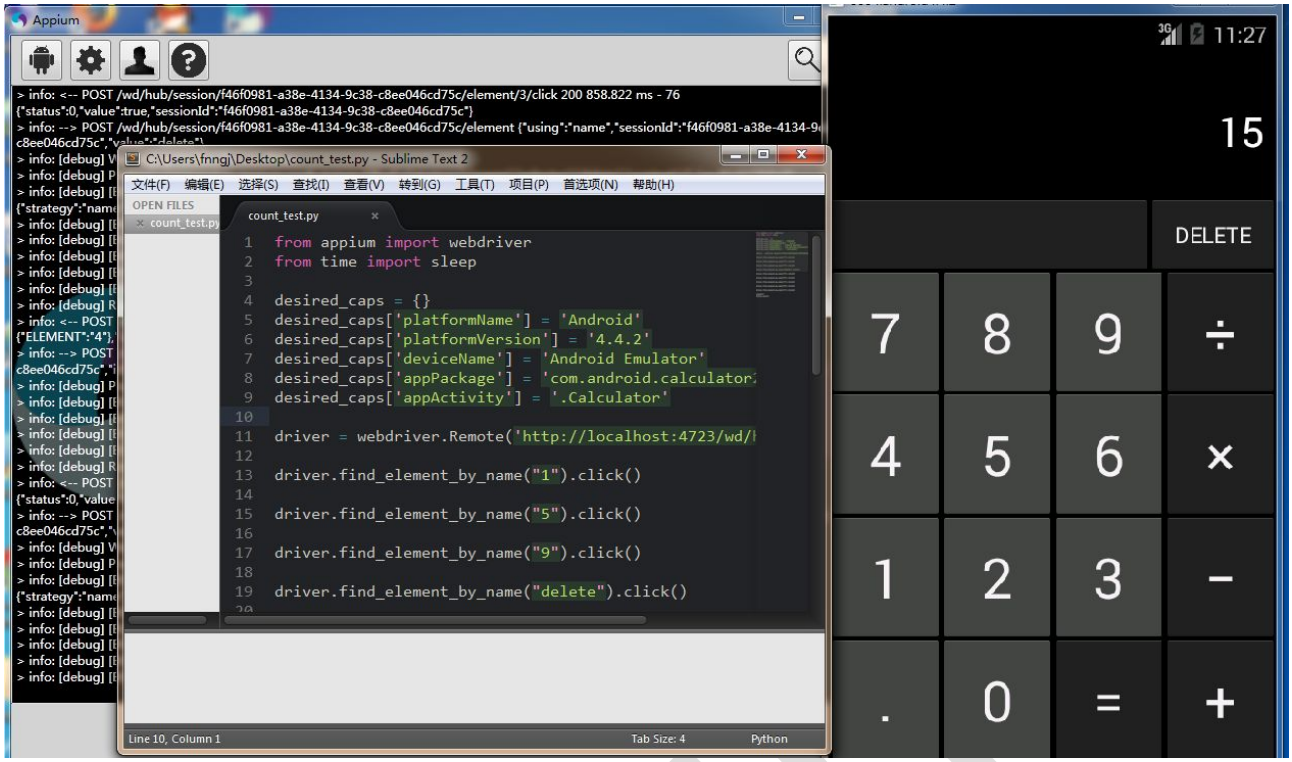


然后点击右上角的“三角”按钮启动 Appium。



第三步，通过 Python 编写自动化测试脚本。

内部培训文档，半成品，不外传。



cal_test.py

```
#coding=utf-8
from appium import webdriver

desired_caps = {}
desired_caps['platformName'] = 'Android'
desired_caps['platformVersion'] = '4.4.2'
desired_caps['deviceName'] = 'Android Emulator'
desired_caps['appPackage'] = 'com.android.calculator2'
desired_caps['appActivity'] = '.Calculator'

driver = webdriver.Remote('http://localhost:4723/wd/hub', desired_caps)

driver.find_element_by_name("1").click()

driver.find_element_by_name("5").click()

driver.find_element_by_name("9").click()

driver.find_element_by_name("delete").click()

driver.find_element_by_name("9").click()

driver.find_element_by_name("5").click()
```

```
driver.find_element_by_name("+").click()

driver.find_element_by_name("6").click()

driver.find_element_by_name("=").click()

driver.quit()
```

作者水平有限，安装过程繁琐了一些，可能有些工具不是必装的，有些我还无法解释它的作用。但是，按照我的步骤可以一步一步的把 Appium 的自动化测试环境给搭建起来。至于我们安装这些工具的作用，可以放后面慢慢来理解。

在安装的过程中我们横跨三大编程语言，Appium 是基于.NET 框架开发的，android 是基于 JAVA 开发的，而 client 端我们选用 Python 语言来编写自动化脚本。这就要求测试人员对编程语言要有一定的基础和认识，不然后面的学习之路一定举步维艰。

第二章 Appium 基础

上一节的最后我们编写了一个自动化测试脚本来测试 Android 自带的计算器功能，但是可能并不十分清楚脚本中的每一行的含义；假如让你去测试一个 APP 的话，你依然不从下手，怎么定位对象？Appium 都提供了哪些方法？比如：解锁、滑动屏幕、双击、点击、输入怎么实现。这些疑问将会在这一章中得到答案。

第一节 Desired Capabilities 详解

Desired Capabilities 在启动 session 的时候是必须提供的。

Desired Capabilities 本质上是以 key value 字典的方式存放，它告诉 appium server 这样一些事情：

- 本次测试是启动浏览器还是启动移动设备？
- 是启动 andorid 还是启动 ios？
- 启动 android 时，app 的 package 是什么？
- 启动 android 时，app 的 activity 是什么？

如果有学习过 WebDriver 的同学一定知道 WebDriver 的 Desired Capabilities 是这样的（以 Python 为例）：

remote_ts.py

```
from selenium.webdriver import Remote
from selenium.webdriver.common.desired_capabilities import DesiredCapabilities

driver = Remote(command_executor='http://127.0.0.1:4444/wd/hub',
              desired_capabilities={'platform': 'ANY',
                      'browserName':'chrome',
                      'version': '',
                      'javascriptEnabled': True
                      })
```

- 'platform': 'ANY' 平台默认可以是任何（Windows，MAC，android）。
- 'browserName': 'chrome' 浏览器名字是 chrome。
- 'version': '' 浏览器的版本默认为空。
- 'javascriptEnabled': True javascript 启动状态为 True。

Appium 的 Desired Capabilities 是扩展了 webdriver 的 Desired Capabilities 的。

- automationName：使用哪种自动化引擎。appium（默认）还是 Selendroid?
- platformName：使用哪种移动平台。iOS, Android, orFirefoxOS?
- deviceName：启动哪种设备，是真机还是模拟器？iPhone Simulator, iPad Simulator, iPhone Retina 4-inch, Android Emulator, Galaxy S4, etc...
- app：应用的绝对路径，注意一定是绝对路径。如果指定了 appPackage 和 appActivity 的话，这个属性是可以不设置的。另外这个属性和 browserName 属性是冲突的。
- browserName：移动浏览器的名称。比如 Safari' for iOS and 'Chrome', 'Chromium', or 'Browser' for Android；与 app 属性互斥。
- udid：物理机的 id。比如 1ae203187fc012g。(UDID 是由子母和数字组成的字符串的序号，用来区别每一个唯一的 Android/iOS 设备。)

and_test.py

```
# Android environment
from appium import webdriver

desired_caps = {}
desired_caps['platformName'] = 'Android'
desired_caps['platformVersion'] = '4.2'
desired_caps['deviceName'] = 'Android Emulator'
```

```
desired_caps['app'] = PATH('../../../apps/selendroid-test-app.apk')

driver = webdriver.Remote('http://localhost:4723/wd/hub', desired_caps)
……
```

ios_test.py

```
# iOS environment
from appium import webdriver

desired_caps = {}
desired_caps['platformName'] = 'iOS'
desired_caps['platformVersion'] = '7.1'
desired_caps['deviceName'] = 'iPhone Simulator'
desired_caps['app'] = PATH('../../apps/UICatalog.app.zip')

driver = webdriver.Remote('http://localhost:4723/wd/hub', desired_caps)
……
```

下面这些属性是 android 平台特定的：

- appActivity：待测试的 app 的 Activity 名字。比如 MainActivity, .Settings。注意，原生 app 的话要在 activity 前加个"."。
- appPackage：待测试的 app 的 java package。比如 com.example.android.myApp, com.android.settings。

例如我们第一个自动化测试脚本中的配置：

cal_test.py

```
#coding=utf-8
from appium import webdriver

desired_caps = {}
desired_caps['platformName'] = 'Android'
desired_caps['platformVersion'] = '4.4.2'
desired_caps['deviceName'] = 'Android Emulator'
desired_caps['appActivity'] = '.Calculator'
desired_caps['appPackage'] = 'com.android.calculator2'
```

```
driver = webdriver.Remote('http://localhost:4723/wd/hub', desired_caps)
```

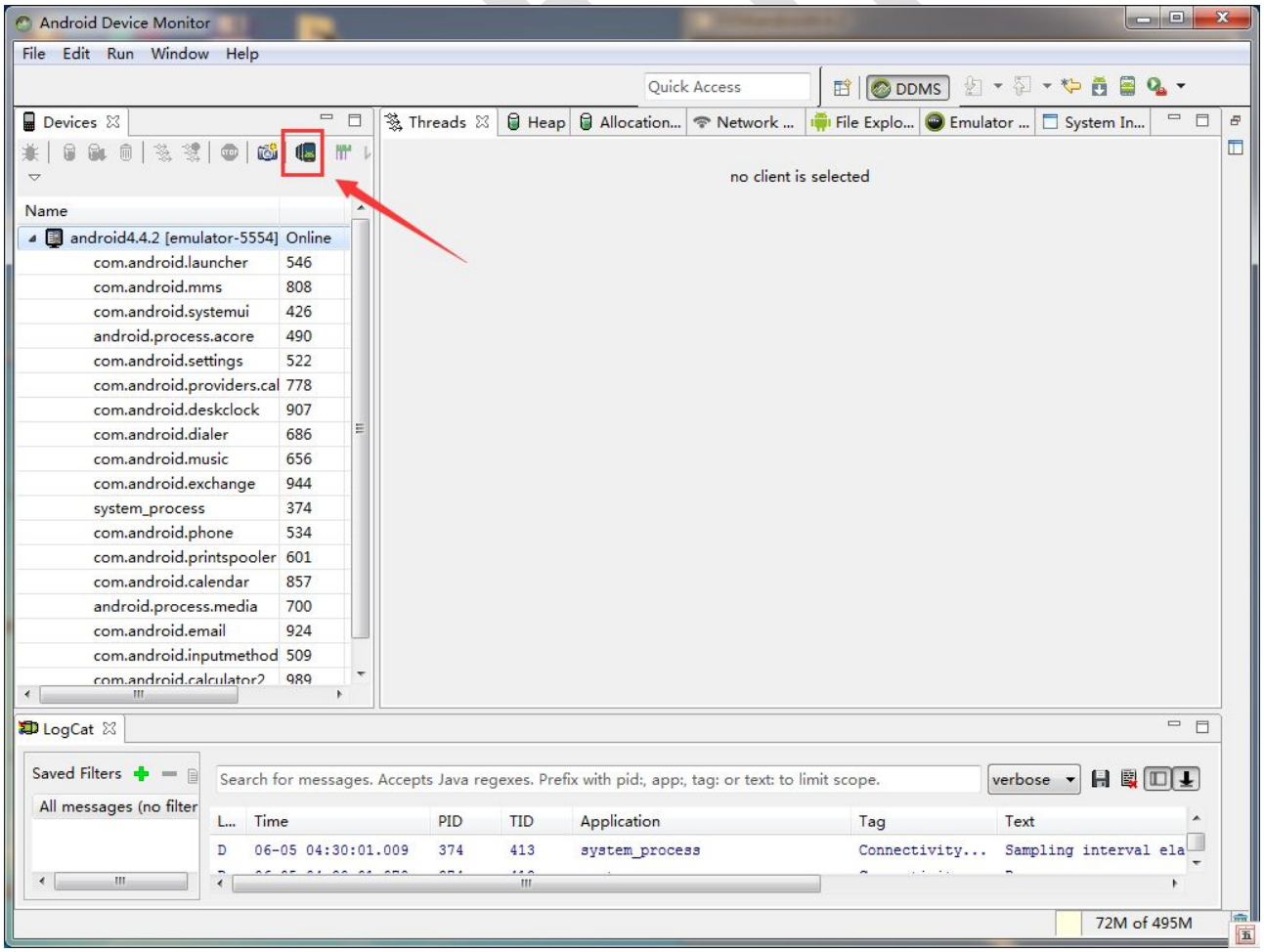
那么如何获取被测 app 的 Activity 和 Package 呢？这是下来需要解决的问题。

第二节　分支一&必备工具

先来看 app 的的 Package 如何获取。与此同时我们会接触到两个具：monitor.bat 和 uiautomatorviewer.bat。这两个工具是在做 android 自动化过程中必须会用的两个辅助工具，这里做个简单的介绍。

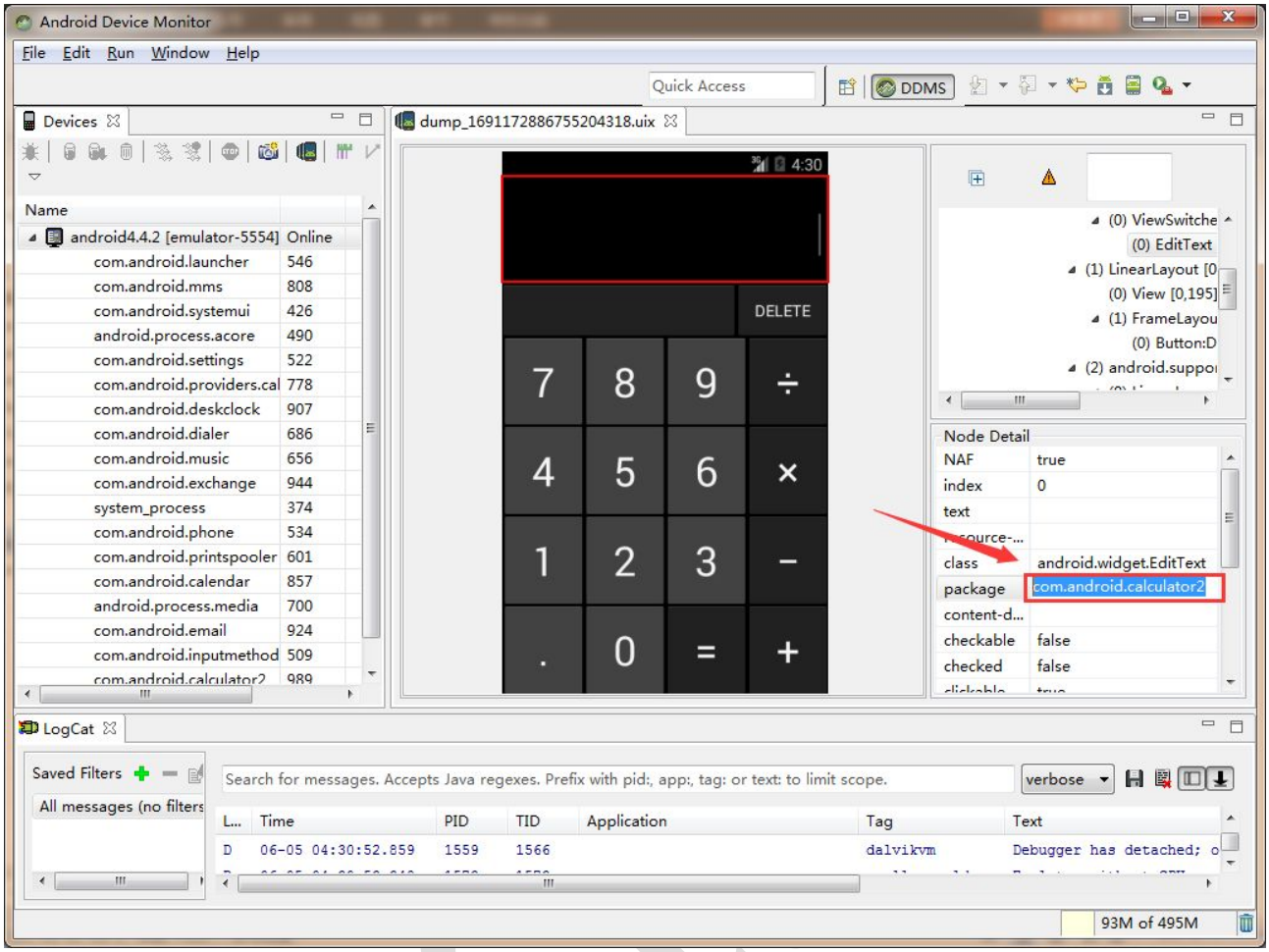
monitor.bat

此工具位于...\android-sdk-windows\tools\目录下，双击 monitor.bat 启动。界面如下：



如上图如示，点击 Devices 标签页工具栏上的小手机的图标，（注意：前提是我们已经启动了 android

虚拟机并启动了相应的 app）然后，就会在左下角看到 app 的 Package 信息了。



扩展理解：

monitor.bat 工具可帮助我们找到 android 的 conten-desciption。要想知道是 conten-desciption 是什么？我们还需要先知道另外一个东西---“Accessibility ID”，我们可以使用元素的 Accessibility ID 来找定位元素。也就是说我们可以将一个字符串 id 或标签附加到一个给定的元素。例如 IOS 的 accessibility identifier 或 android 的 conten-desciption。

通过过“Accessibility ID”定位元素的方法：

cal_test.py

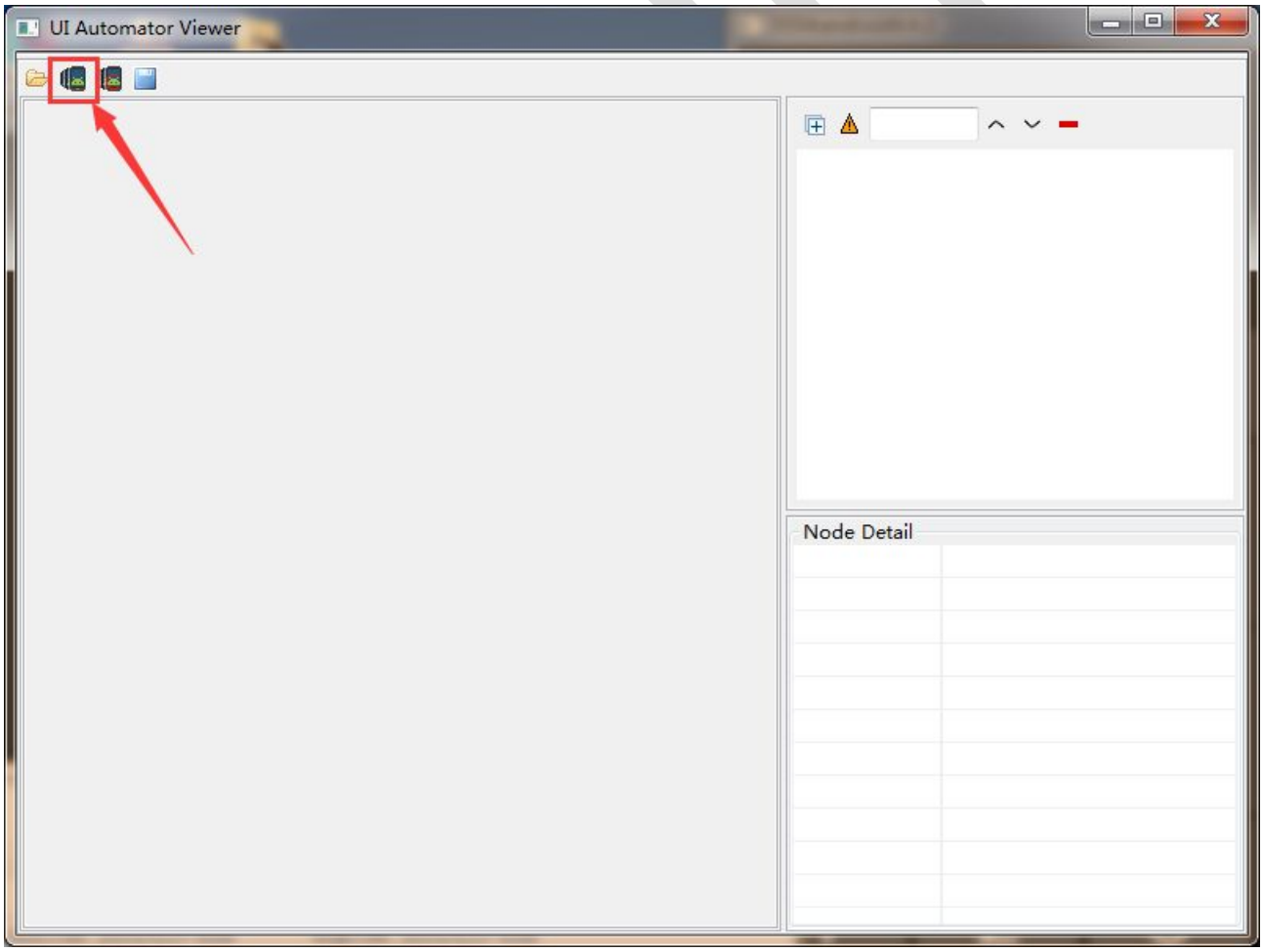
```
......
el = self.driver.find_element_by_accessibility_id('Animation')
self.assertIsNotNone(el)
......
els = self.driver.find_elements_by_accessibility_id('Animation')
self.assertIsInstance(els, list)
```

“Animation”就是元素的 Accessibility ID。也是许我翻译的不准确。那么你可以看下面的英文说明：

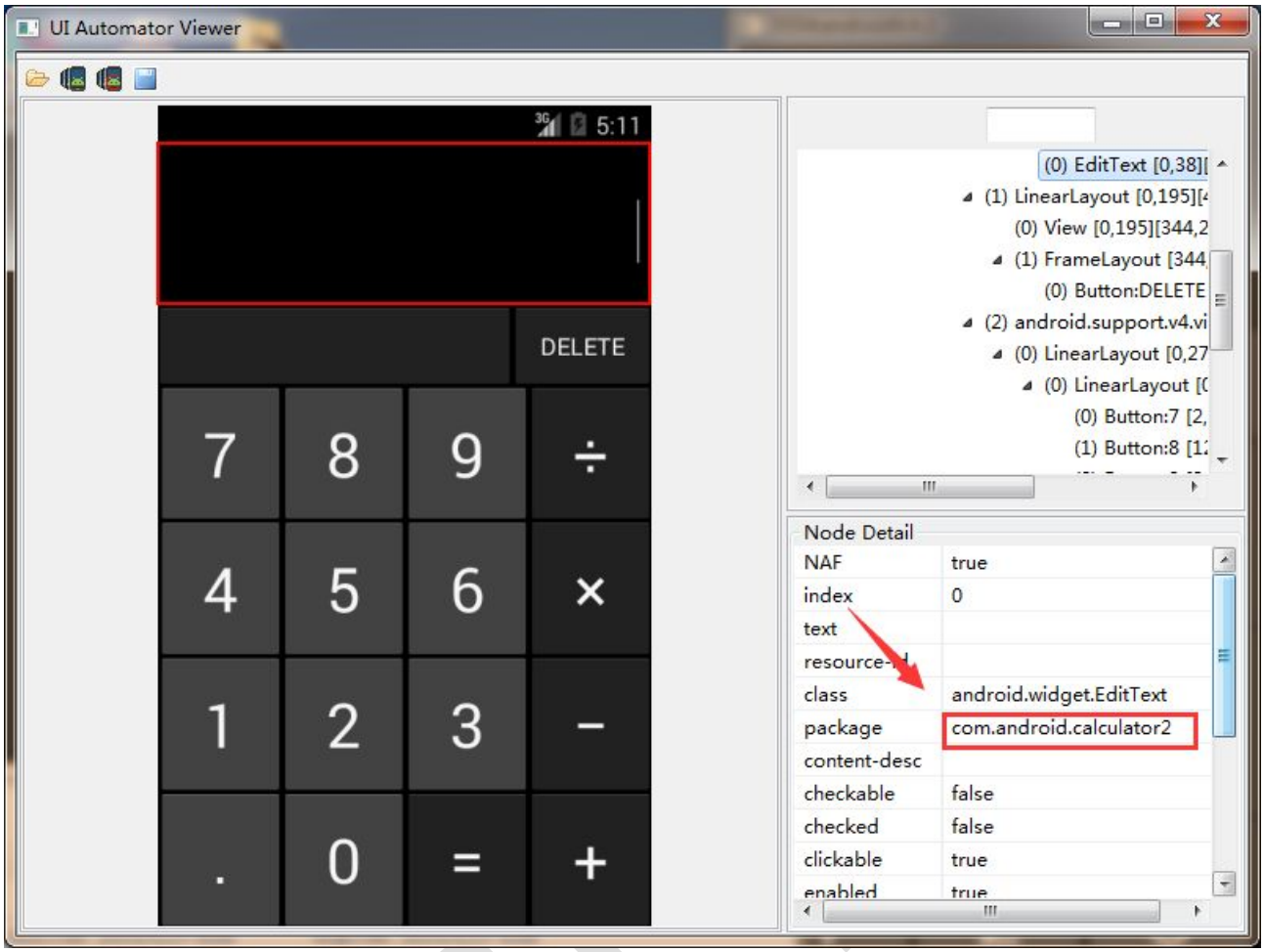
> Allows for elements to be found using the "Accessibility ID". The methods take a string representing the accessibility id or label attached to a given element, e.g., for iOS the accessibility identifier and for Android the content-description. Adds the methods driver.find_element_by_accessibility_id andfind_elements_by_accessibility_id.

uiautomatorviewer.bat

此工具位于...\android-sdk-windows\tools\目录下，双击 uiautomatorviewer.bat 启动。界面如下：



如上图点击工具栏上的第一个小手机的图标。



同样，前提是我们已经启动了 android 虚拟机并启动了相应的 app。如上图右侧可以看到 app 的 package 信息。

扩展理解：

该工具主要用来查看控件的属性，比如 resource id，class name 等。我们后面的对象定位中会经常用到它，犹如 firefox 浏览器中的 firebug 工具一样重要。当然，它也可查看被测 app 的 appPackage（Desired Capabilities 中使用），这也是我们在此处展示它的目的。

第三节　分支二&反编译 APP

作者跑题目了吧！？接下来不是应该介绍如何找 app 的 Activity 信息么？怎么扯到反编译代码了呢？好吧，我们就是要找 app 的 Activity，大概有以下几种方法。

方法一

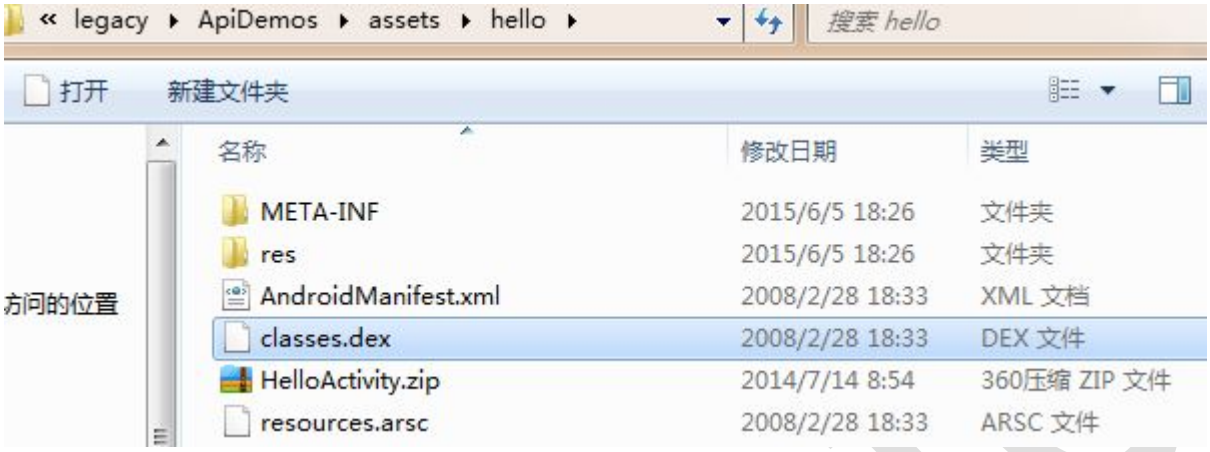
如有你有待测项目的源码，那么直接查看源码就好。如果没有，那么请联系有源码的同学，这是最简单的方法。

方法二

当然，身为一个有逼格的测试，一定要会一些逼格技能。那就是可以通过反编译 app 的方式来查看 app

的 Activity。

我们以...\android-sdk-windows\samples\android-19\legacy\ApiDemos\assets 目录下的 HelloActivity.apk 为例。复制并重命名为 HelloActivity.zip，并将其解压到一个指定的目录：

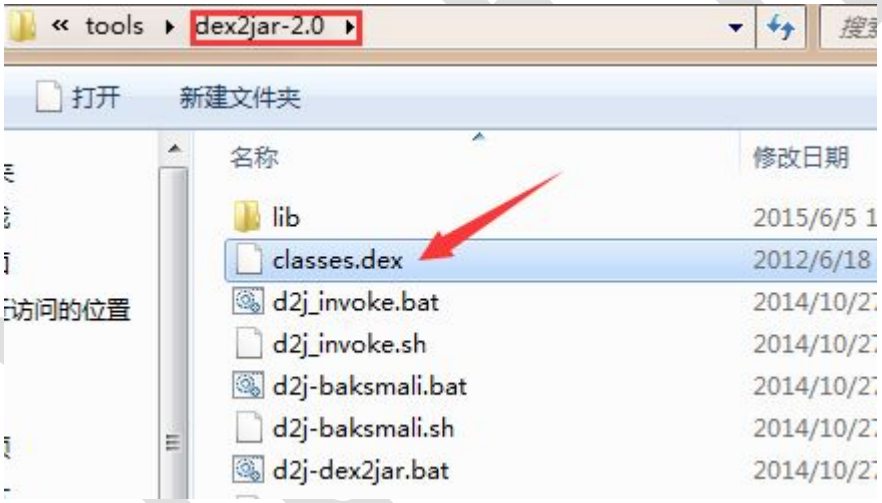


接下来需要借助两个小工具：

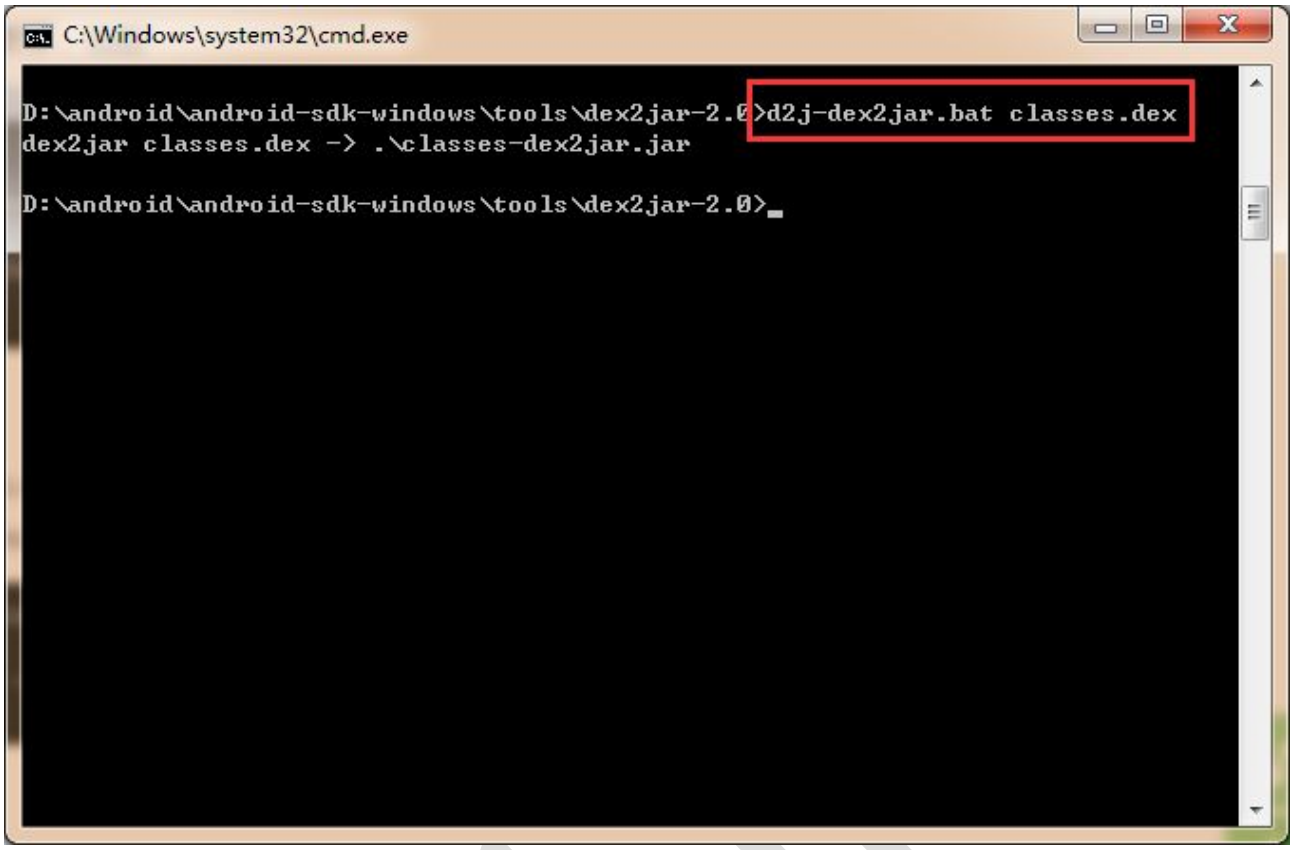
dex2jar ： http://sourceforge.net/projects/dex2jar/

jd-gui ： http://jd.benow.ca/

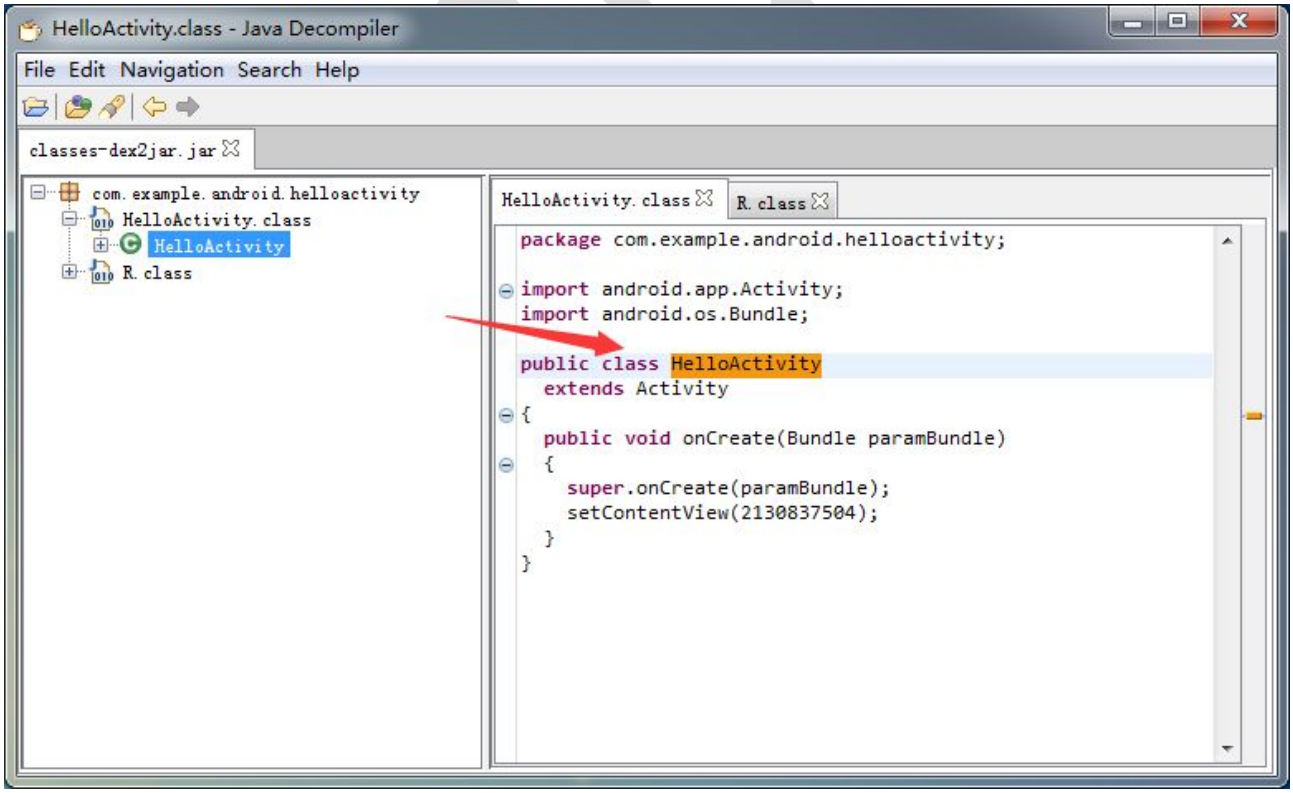
把解压之后的 classes.dex 文件拷贝到 dex2jar 目录：



在 dex2jar-2.0 目录下输入：d2j-dex2jar.bat classes.dex



然后在当前目录下就生成了一个 classes-dex2jar.jar 的文件。下面就可以通过 jd-gui 工具来打开这个 jar 文件。



上图所示的 HelloActivity 就是待测 app 的 main activity。当然，因为这个 apk 属于一个 demo 所以非常简单，对于一些项目的 apk，要想找到 app 的 activity 还需要你在 Java 代码的熟悉。

第四节　分支三&多渠道打包工具

接下来介绍第三种找 app 的 activity 的方法。

方法三

利用 Apktool 工具，该工具为 Android 多渠道打包工具。

Apktool 下载地址：https://github.com/wubo/apptools

需要通过 git 将项目克隆下来。

git

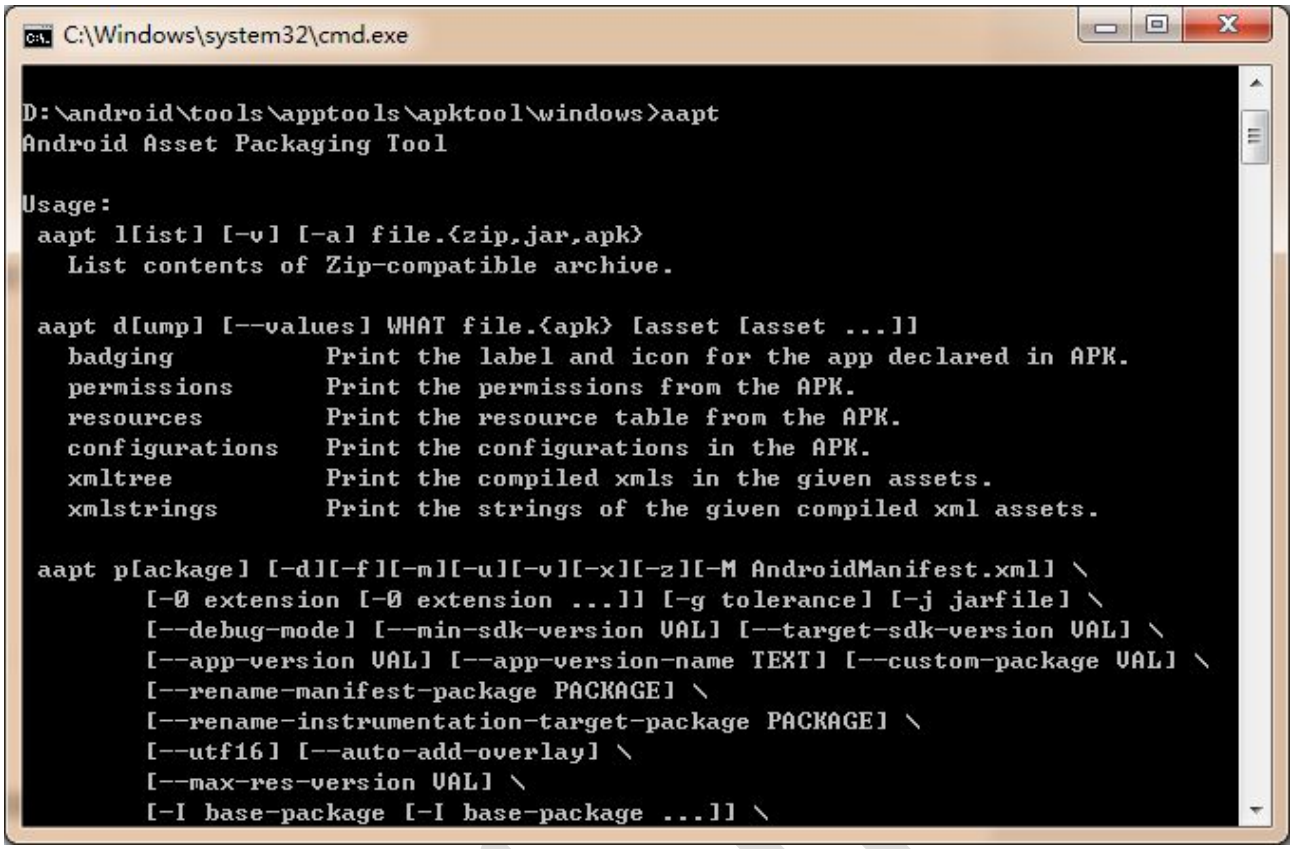
```
……
$ git clone git@github.com:wubo/apptools.git

Cloning into 'apptools'...
Warning: Permanently added the RSA host key for IP address '192.30.252.131' to t
he list of known hosts.
remote: Counting objects: 166, done.
Receiving objects:  59% (99/166), 7.46 MiB | 72.00 KiB/s
……
```

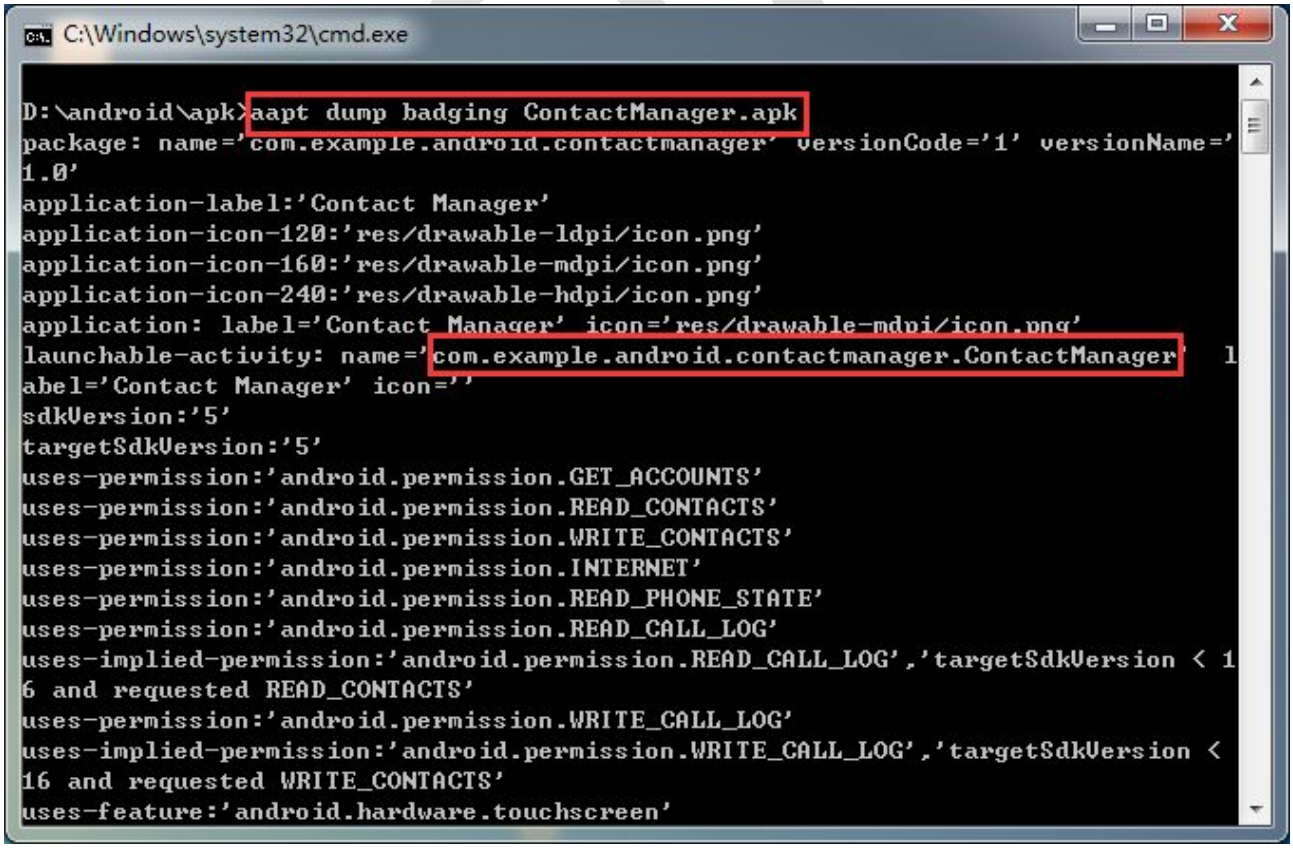
在克隆下来的...\apptools\apktool\windows\目录中会找到 aapt.exe 工具。我们实际要用的就是这个工具。它可以获取 apk 包的信息。

在当前目录下输入“aapt”命令可以查看该工具的使用。

itest.info



将该路径加到系统环境变量 path 中，或将 aapt.exe 与要查看的 apk 包放在同一个目录下。通过“aapt dump badging xxx.apk”命令查看。（xxx 替换为要查看的 apk 包的名字）



当然，除此之外还有很多可方法可以找到 app 的 Activity 信息。

第五节　启动其它应用

下面是启动 android 设置的配置。

setting_test.py

```
#coding=utf-8
from appium import webdriver

desired_caps = {}
desired_caps['platformName'] = 'Android'
desired_caps['platformVersion'] = '4.4.2'
desired_caps['deviceName'] = 'Android Emulator'
desired_caps['appPackage'] = 'com.android.settings'
desired_caps['appActivity'] = '.Settings'

driver = webdriver.Remote('http://127.0.0.1:4723/wd/hub', desired_caps)
……
```

下面是启动 android 通讯录的配置。

setting_test.py

```
#coding=utf-8
from appium import webdriver

desired_caps = {}
desired_caps['platformName'] = 'Android'
desired_caps['platformVersion'] = '4.4.2'
desired_caps['deviceName'] = 'Android Emulator'
desired_caps['appPackage'] = 'com.example.android.contactmanager'
desired_caps['appActivity'] = '.ContactManager'

driver = webdriver.Remote('http://127.0.0.1:4723/wd/hub', desired_caps)
……
```

下面是启动 android 自带浏览器的配置。

browser_test.py

```
#coding=utf-8
from appium import webdriver
```

itest.info

```
desired_caps = {
        'platformName': 'Android',
    'platformVersion': '4.4.2',
    'deviceName': 'Android Emulator',
    'browserName': 'Browser',
}

driver = webdriver.Remote('http://127.0.0.1:4723/wd/hub', desired_caps)

print "start  baidu page!"
driver.get('http://www.baidu.com')
```

第六节　控件定位

终于要讲定位了，哈哈！对于自动化测试来说，核心技能就是对象的定位。不管是 web 页面上的一个按钮或输入框，还是移动 app 上的一个按钮或输框，我们要想对其进行点击或输入操作，前提是要先找到这个对象。对于手工测试来说，是由测试人员来完人了，人通过眼睛与大脑来识别一个按钮或输入框，但自动测试工具没有这种能力，但是一个对象一定会有一些属性（如 id、class 等），自动化测试工具就是靠着这些属性来识别和查找对象。

webdriver 提供了八种元素定位方法：

- id
- name
- class name
- tag name
- link text
- partial link text
- xpath
- css selector

在 Python 语言中对应的定位方法如下：

find_element_by_id()
find_element_by_name()
find_element_by_class_name()
find_element_by_tag_name()
find_element_by_link_text()
find_element_by_partial_link_text()
find_element_by_xpath()

testinfo

find_element_by_css_selector()

一组元素定位的方法如下：

find_elements_by_id()
find_elements_by_name()
find_elements_by_class_name()
find_elements_by_tag_name()
find_elements_by_link_text()
find_elements_by_partial_link_text()
find_elements_by_xpath()
find_elements_by_css_selector()

Appium 完全继承了 WebDriver 中所定义的这些方法，除此之外对其进行了扩展，以便适合移动端对象的定位与操作。

由 Mobile JSON Wire Protocol 协议中定义的方法，更适合移动设备上的控件定位

- ios uiautomation：一个递归地、对应使用 UIAutomation library 搜索元素的字符串(iOS-only)
- android uiautomator：一个递归地、对应使用 UiAutomator Api 搜索元素的字符串(Android-only)
- accessibility id：一个递归地、使用本地 Accessibility 选项实现的 Id/Name 进行元素搜索的字符串。

对于 Python 来讲，在 WebDriver 定位方法的基础上增加了下列方法：

find_element_by_accessibility_id()
find_elements_by_accessibility_id()
find_element_by_android_uiautomator()
find_element_by_android_uiautomator()

一、name 定位

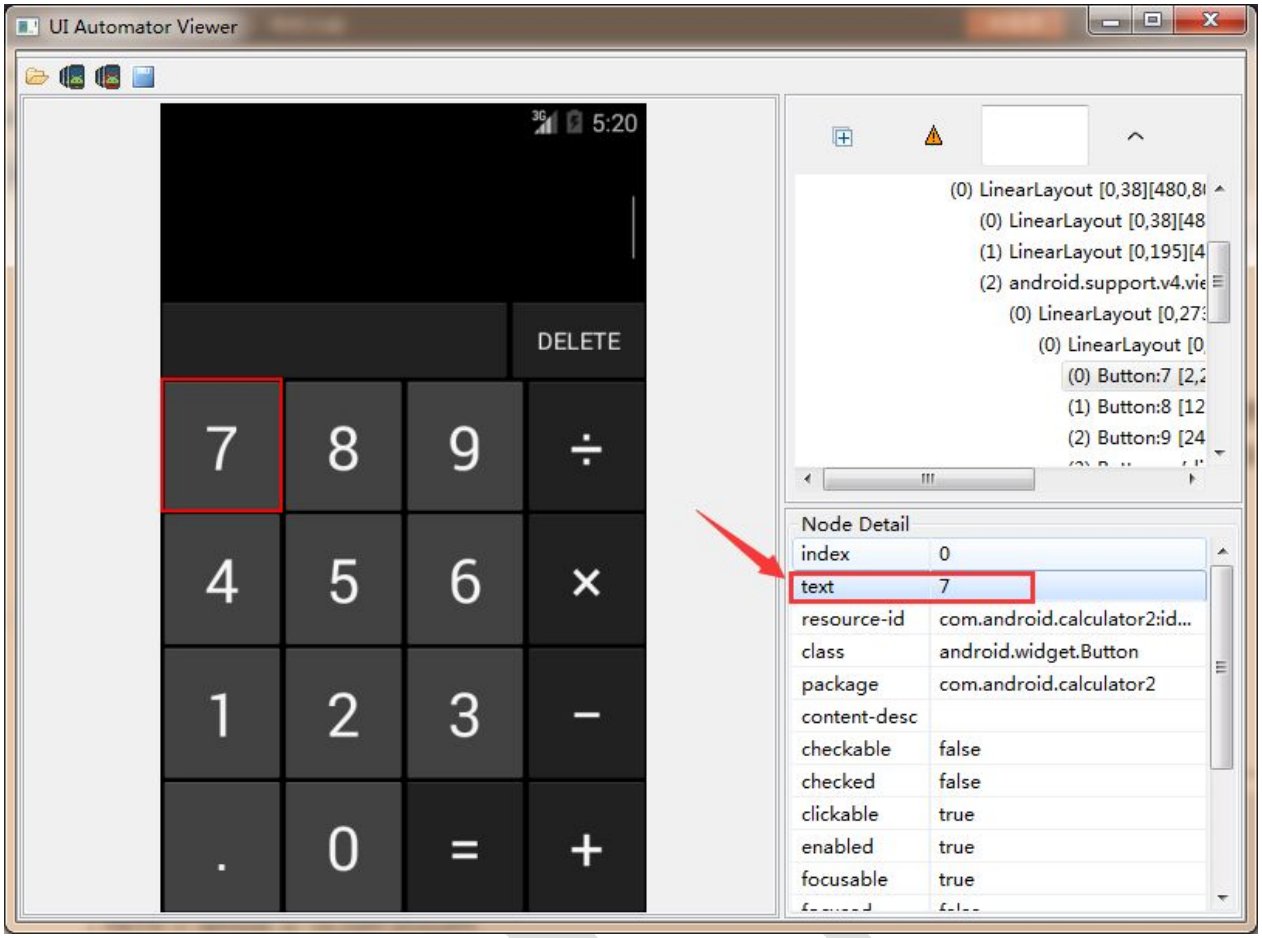
如何获得 Name

安卓设备没有找到适合的方法，尝试用 Appium Inspector，但是使用了当前最新的“AppiumForWindows-1.2.3.1”没有看到这个属性，且 Inspector 在 Windows 下面非常的不稳定，很容易 crash。真心期望 Appium 团队尽快解决这个问题。iOS 设备倒可以用 Appium Inspector 获得。

个人建议可以尝试先用 view 显示的文本作为 name 看是否能拿到该控件，按照我个人的经验一般都是会成功的，所以我很怀疑安卓上面控件的 name 是否就等于 text。如果确实还是不行的话就只好放弃用 name 了。或者等待 Appium 后来的稳定的 inspector 发布后看是否可以获得控件的 name。

打开 uiautomatorviewer.bat 工具：

itest.info



写法如我们第一个自动化测试脚本：

cal_test.py

```
……
driver.find_element_by_name("1").click()

driver.find_element_by_name("5").click()

driver.find_element_by_name("9").click()

driver.find_element_by_name("delete").click()
……
```

所以，我们暂时可以把元素的 text 属性看作 name 来使用。如果你你的 app 中不能正常的定位元素，那就换其它定位方式吧。

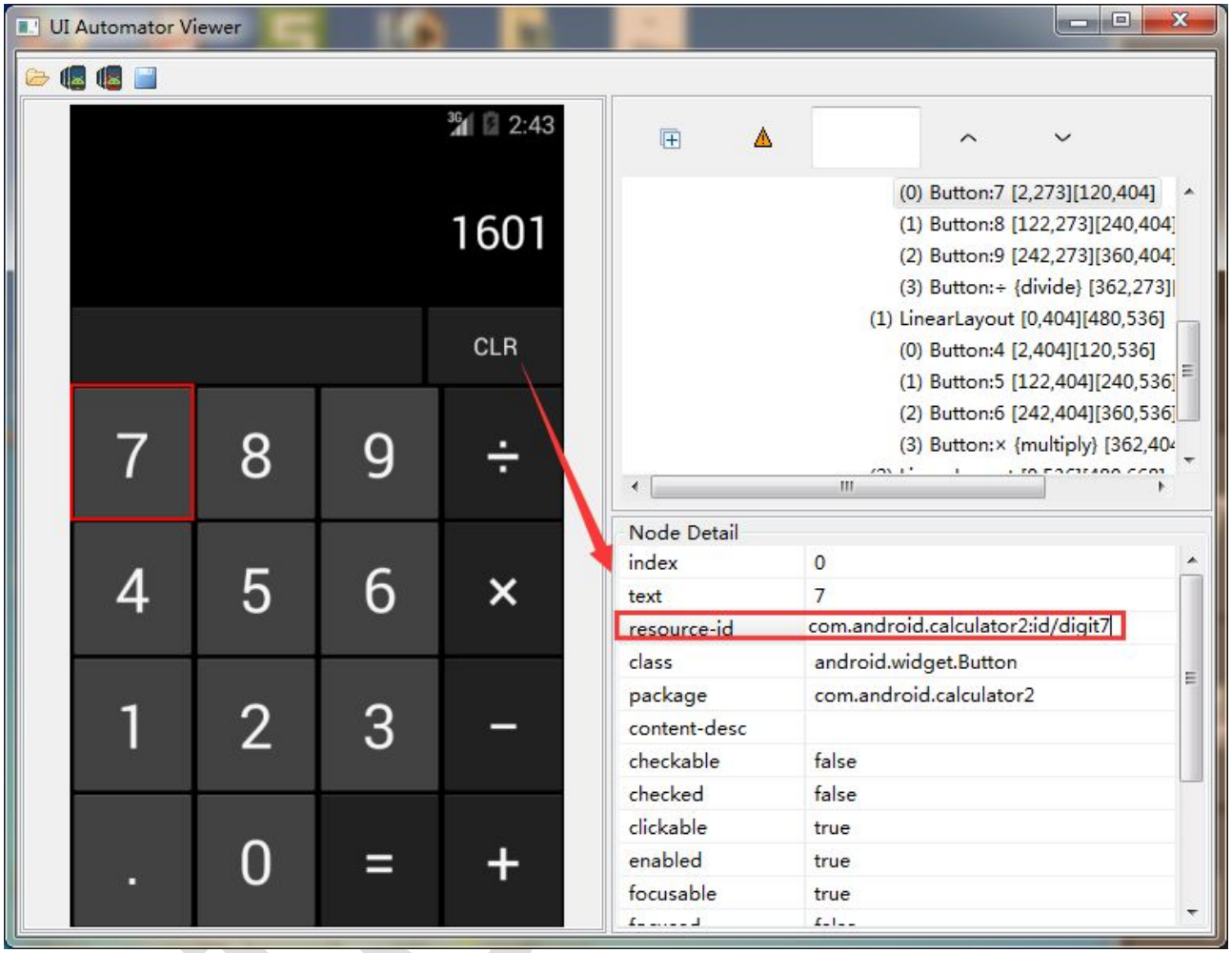
二、id 定位

如何获得 id

通过 uiautomatorviewer.bat 工具可以查看对象的 id 属性。

如果目标设备的 API Level 低于 18 则 UIAutomatorViewer 不能获得对应的 Resource ID，只有等于大于 18 的时候才能使用。

打开 uiautomatorviewer.bat 工具：



resource-id 就是我们理解的 id 属性了。

cal_test.py

```
.....
driver.find_element_by_id("com.android.calculator2:id/digit1").click()

driver.find_element_by_id("com.android.calculator2:id/digit5").click()

driver.find_element_by_id("com.android.calculator2:id/digit9").click()

driver.find_element_by_id("com.android.calculator2:id/plus").click()

driver.find_element_by_id("com.android.calculator2:id/digit1").click()
```

```
driver.find_element_by_id("com.android.calculator2:id/equal").click()
```

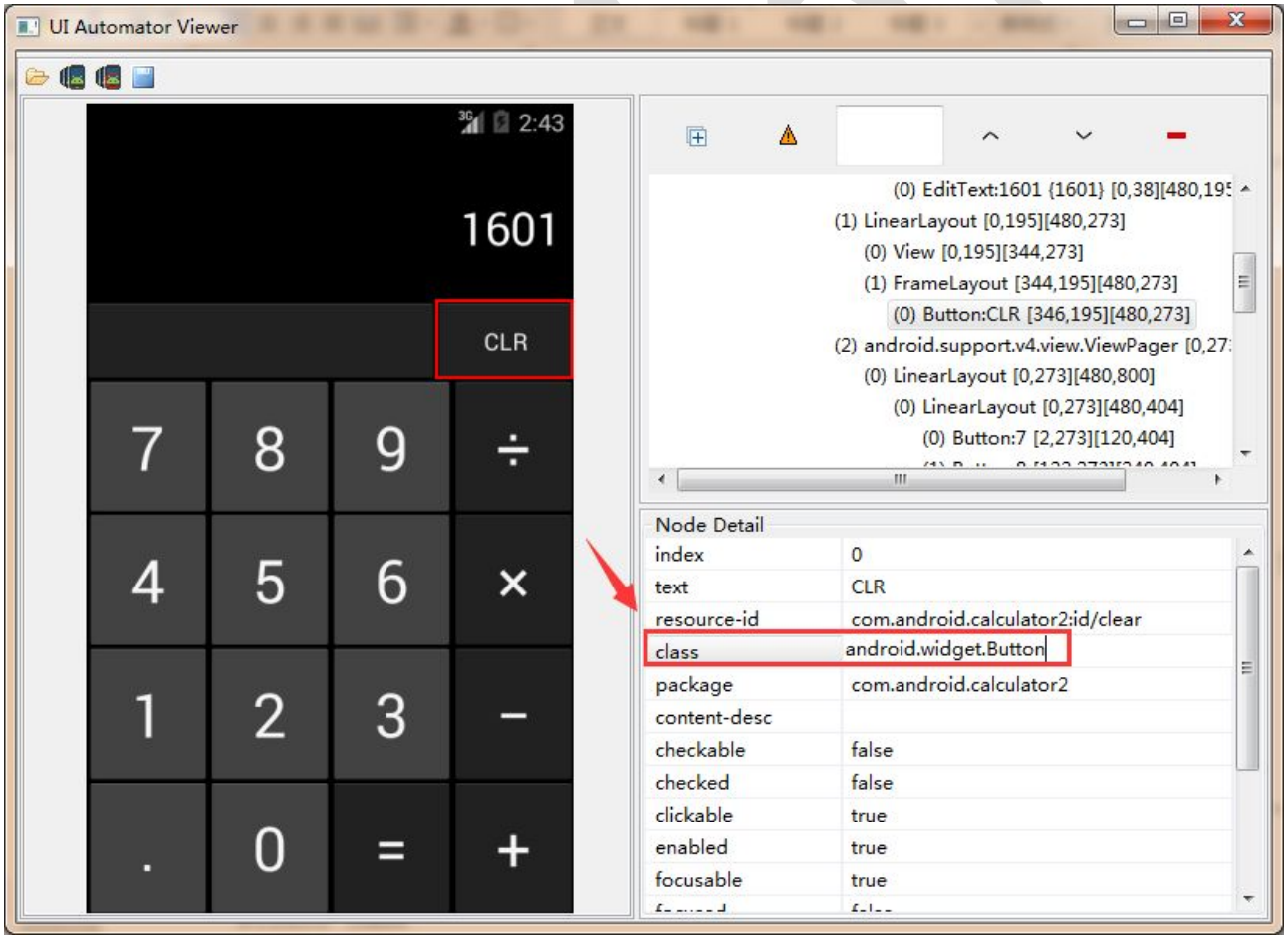
……

三、class name 定位

如何获得控件的 ClassName

可以使用 UIAutomatorViewer.bat 工具直接查看。

使用 ClassName 一般获得的 view 都不止一个，所以应该需要遍历一遍得到的 views，然后缩写搜索条件来获得目标控件。示例中因为只有一个 textview 控件在窗口上面，所以不需要遍历。



cal_test.py

……

```
#class name
```

```
buttons = driver.find_elements_by_class_name("android.widget.Button")

#第一个元素
buttons.pop(0).click()

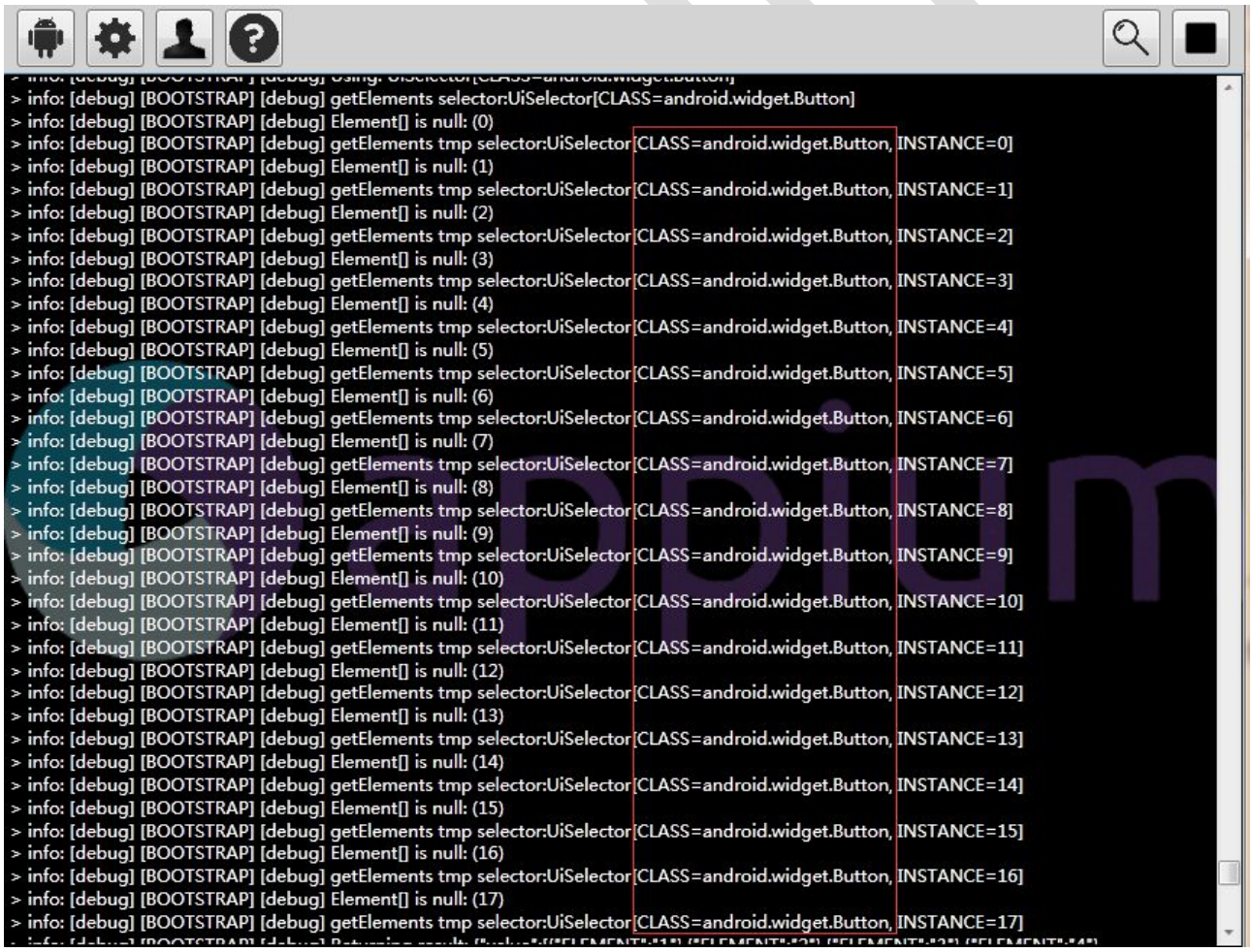
#第二个元素
buttons.pop(1).click()

#最后个元素
buttons.pop().click()

……
```

注意，我们这里用到的是 find_elelments，它获取到的将会是一组元素。因为我们发现计算器上所有按钮的 class name 都叫“android.widget.Button”。

通过 appium 的控制台 log 也可以看到：



可以借助 Python 中的 pop()方法确定想要这一组“按钮”中的第几个，并对它进行 click()或 send_keys()操作。

pop()或 pop(-1) 默认获取一组元素中的最后一个。

pop(0) 默认获取一组元素中的第一个。

pop(1) 默认获取一组元素中的第二个。

……

如果你感兴趣的话可以创建一个叫 all_class()函数，然后告诉这个函数 class 的 name 和第几个元素。让函数去你的要求查询，并给找到的元素返回给你。实现代码如下：

cal_test.py

```
……
def all_class(driver,element,number=0):
        el = driver.find_elements_by_class_name(element).pop(number)
        return el

#点击第 1 个按钮
all_class(driver,"android.widget.Button",1).click()

#点击第 6 个按钮
all_class(driver,"android.widget.Button",5).click()
……
```

为了方便 all_class()会创建于别的文件中调用，所以它需要接收一个 driver，element 为要定位的 class 的 nama，默认 mumber=0 也就是会返回元素中的第一个。

当然，在你使用个定位方法之前要先确定你所定位的元素，是这一组元素中的第几个，这很重要。

四、accessibility id 定位

这个方法属于 appium 扩展的定位方法。

如何获得控件的 ClassName

可以使用 hierarchyviewer.bat 工具直接查看。虽然这个工具已经不推荐使用了。默认放在...\android-sdk-windows\tools 目录下。

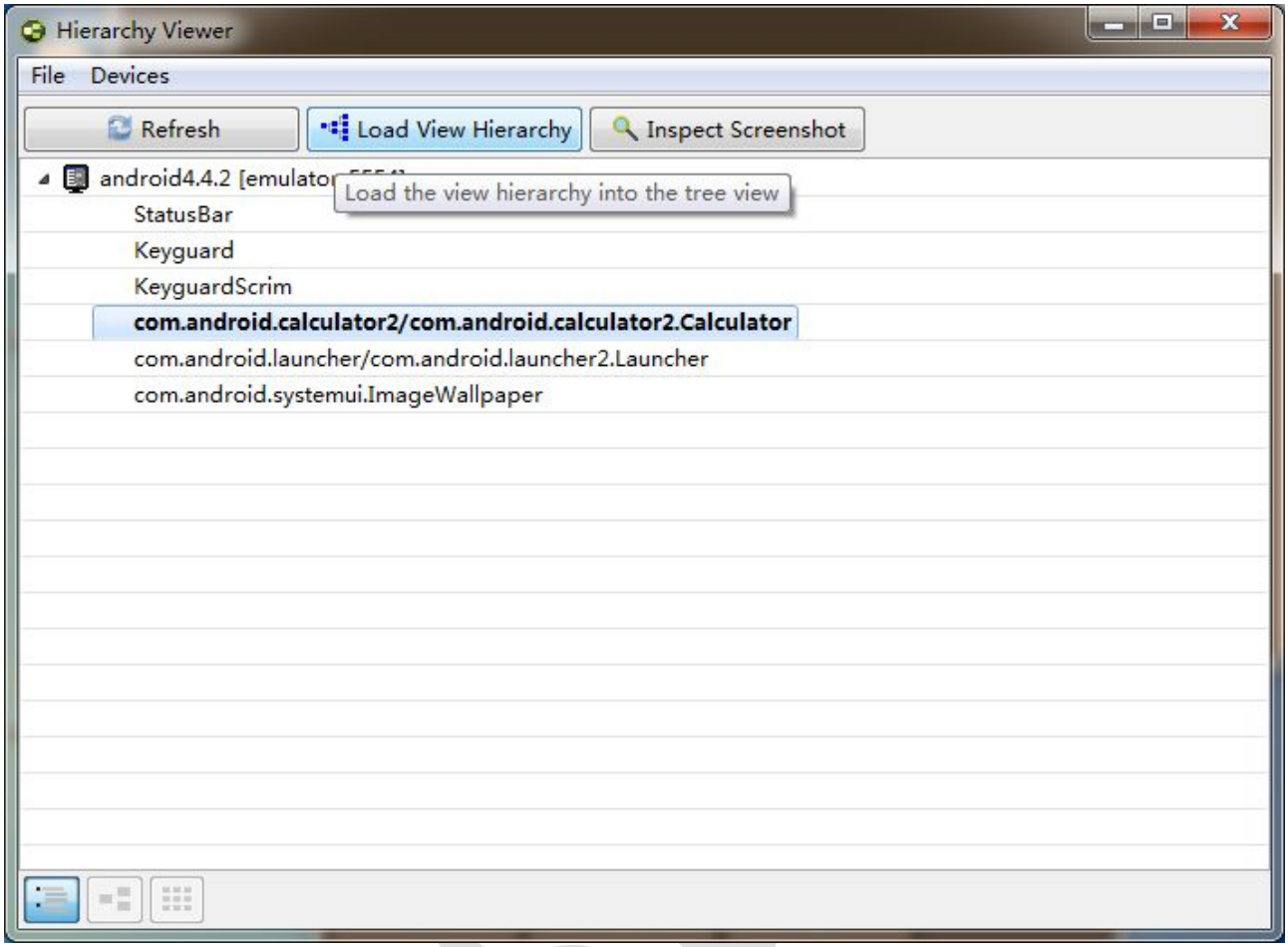
Accessibility ID 在 Android 上面就等同于 contentDescription。这个属性是方便一些生理功能有缺陷的人使用应用程序的。比如我们有一个 ImageView 里面放置一张颜色复杂的图片，可能一些色弱色盲的人，分不清这张图片中画的是什么东西。如果用户安装了辅助浏览工具比如 TalkBack，TalkBack 就会大声朗读出用户目前正在浏览的内容。TextView 控件 TalkBack 可以直接读出里面的内容，但是 ImageView，TalkBack 就只能去读 contentDescription 的值，告诉用户这个图片到底是什么。

鉴于这是一个隐藏属性，而 Android 上用于查找控件的各种属性可能有所缺失或者有重复（比如 id 重复，比如一个 list 下面的所有项可能都是叫做“id/text1”），所以最佳的办法就是跟开发团队沟通好每个

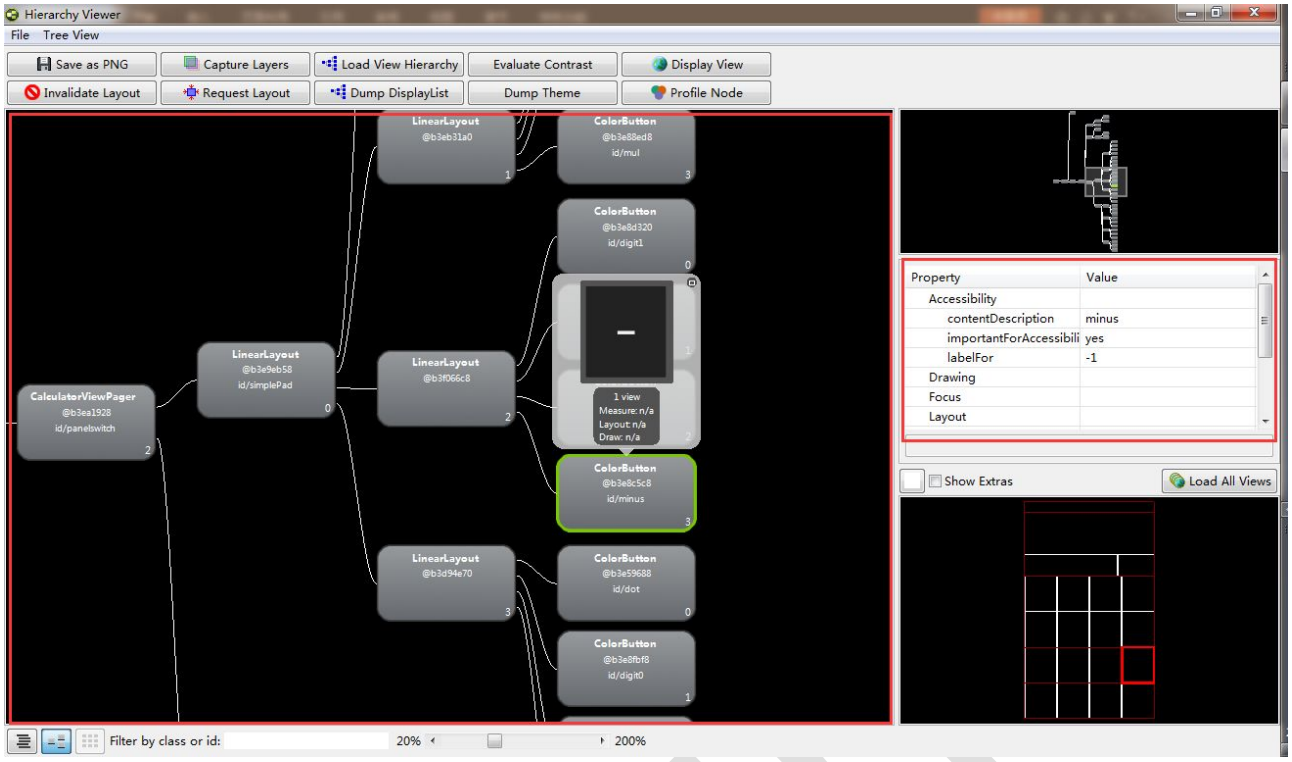
itest.info

view 都赋予一个唯一的 contentDescription。

其实，我们的核心是要找到元素的 contentDescription 属性。打开 hierarchyviewer.bat 工具：

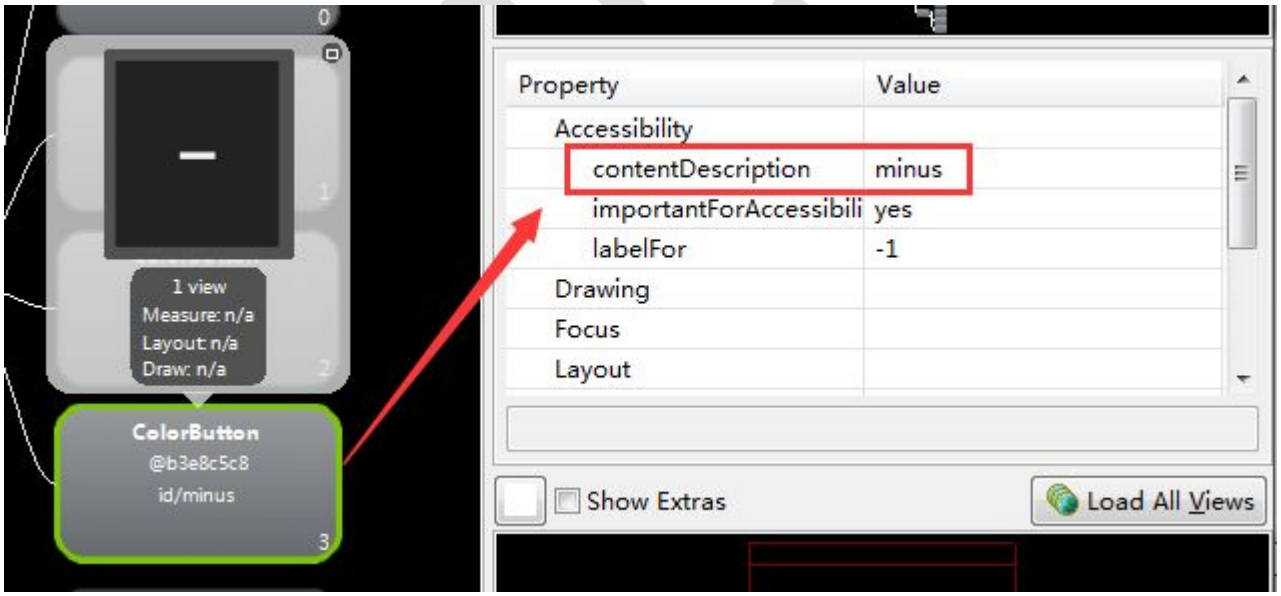


选择“com.android.calculator2/com.android,calculator2.Calculator”，点击“Load View Hierarchy”按钮。

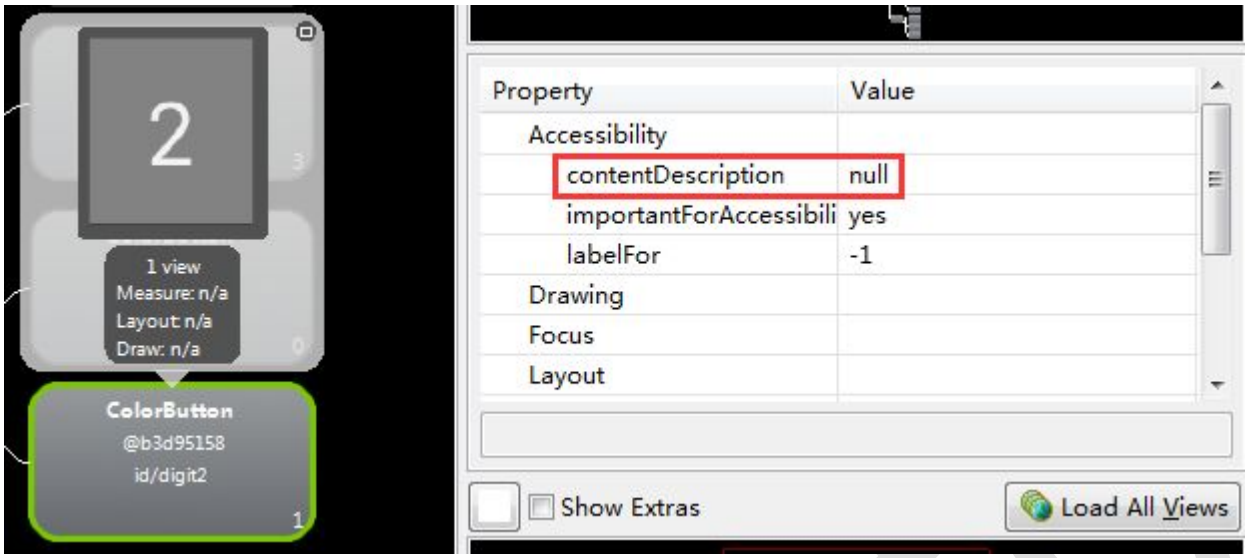


如上图，左则为计算器的结构图，如百度地图一样，我们可以按下鼠标左键进行拖动，底可能调整结构图的大小（20%~200%）。

将结构图拖到右侧，也就是树型结构的子接点，就是计算器上每一个按钮的实现。下再来张特写。



这里选择了一个减法（“-”）按钮，右侧展开就可以看到它的 contentDescription 属性了。当前并不是所有的元素都会有 contentDescription 属性，如计算器中的数字按钮就没有。



所以，为了便于自动化测试的开展，需要和开发同学沟通好，为每一个控件添加 contentDescription 属性。

cal_test.py

```
……
driver.find_element_by_name("8").click()

#定位减法按钮
driver.find_element_by_accessibility_id("minus").click()

driver.find_element_by_name("5").click()

#定位等号按钮
driver.find_element_by_accessibility_id("equals").click()
……
```

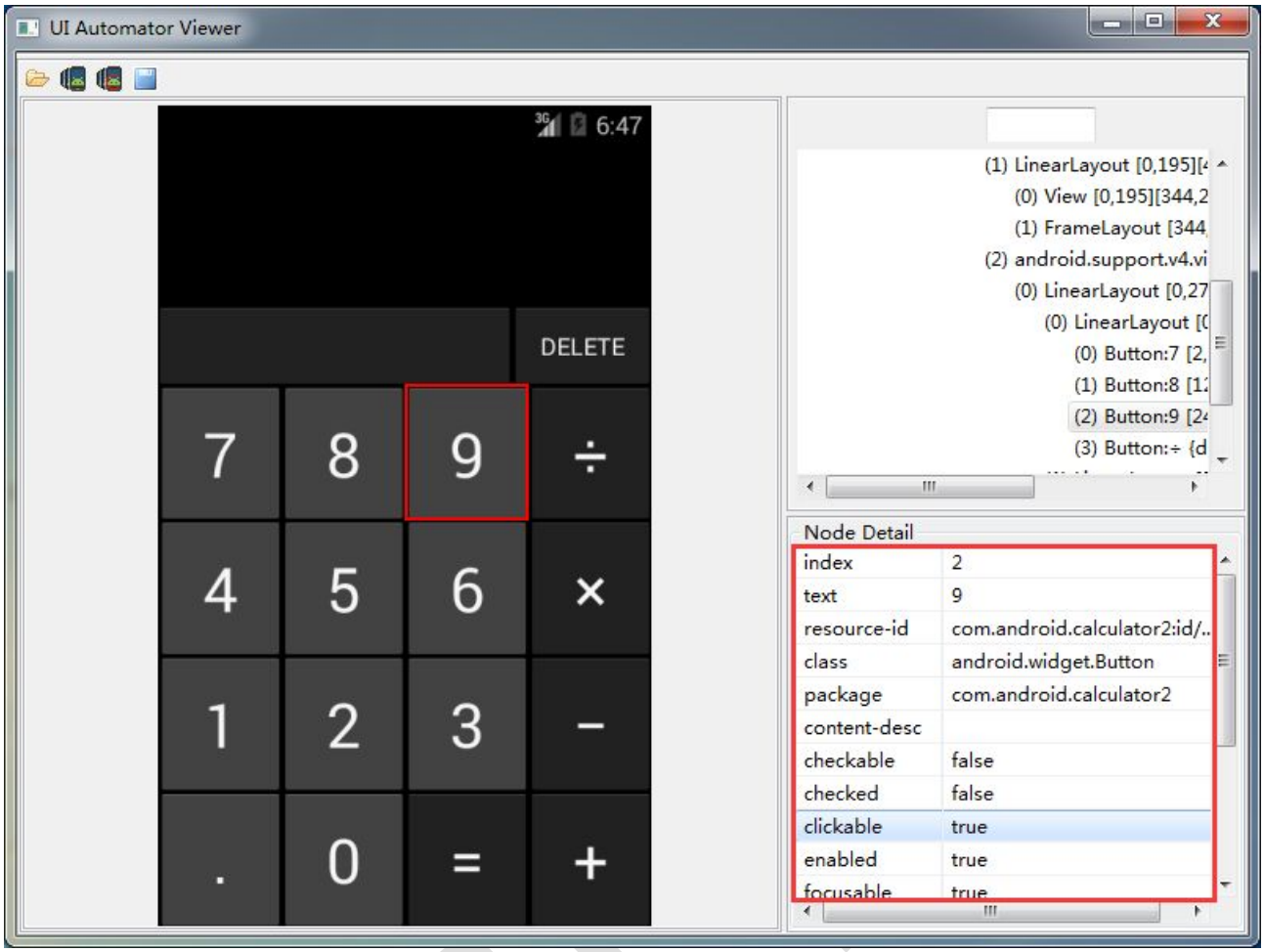
五、android uiautomator 定位

这个方法也属于 appium 扩展的定位方法。

如何获得 UIAutomator 参数

可以使用 UIAutomatorViewer.bat 工具直接查看。

UIAutomator 获取控件的方式多种多样，都是通过 UiSelector 对象来去查找，比如使用 view 的 text 文本去当前窗口查找控件，这里不做累述，往后会另起一篇文章来描述 UIAUtomator 获取控件的方式，到时直接套用进来就可以了。



也就是说一个元素的任意属性都可以通过 android uiautomator 方法来进行定位，但要保证这种定位方式的唯一性。

cal_test.py

```
……
driver.find_element_by_android_uiautomator('new
UiSelector().text("8")').click()

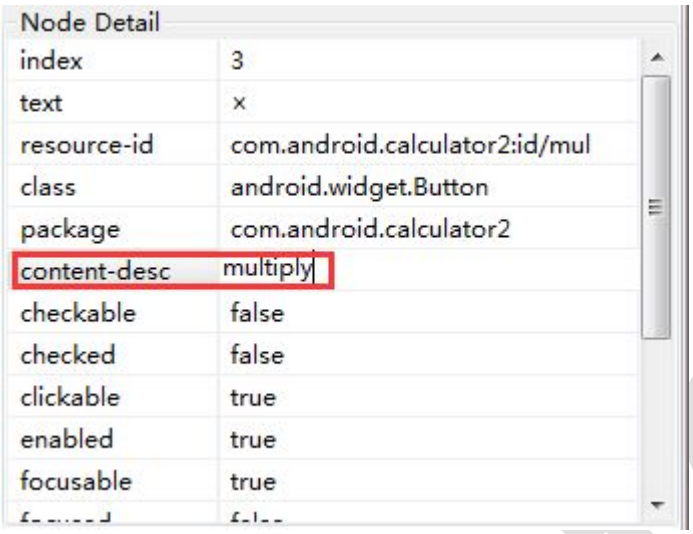
driver.find_element_by_android_uiautomator('new
UiSelector().description("multiply")').click()

driver.find_element_by_android_uiautomator('new
UiSelector().text("5")').click()

driver.find_element_by_android_uiautomator('new
UiSelector().description("equals")').click()

driver.find_element_by_name("8").click()
……
```

需要注意的是 Description()用的 content-desc 属性。



当前，有些属性值并不是唯一的，例如，checkable、checked、clickable 等。那么我们同样可以查找一组元素并筛选。

cal_test.py

```
……
#通过 clickable 属性定位
clicks = driver.find_elements_by_android_uiautomator('new
UiSelector().clickable(true)')
print len(clicks)

clicks.pop(2).click() #7

clicks.pop(8).click() #*

clicks.pop(6).click() #5

clicks.pop(13).click() #=
……
```

当然，你也可以像 class name 一样来实现一个通用的定位函数。

六、xpath 定位

在 webdriver 上 xpath 定位是功能强大的一种定位方式。我个人惯用于此方法来定位 web 页面上的元素。因为在 android 上要定位的为控件，而非页面元素，所以 xpath 定位的写法也会有所不同。

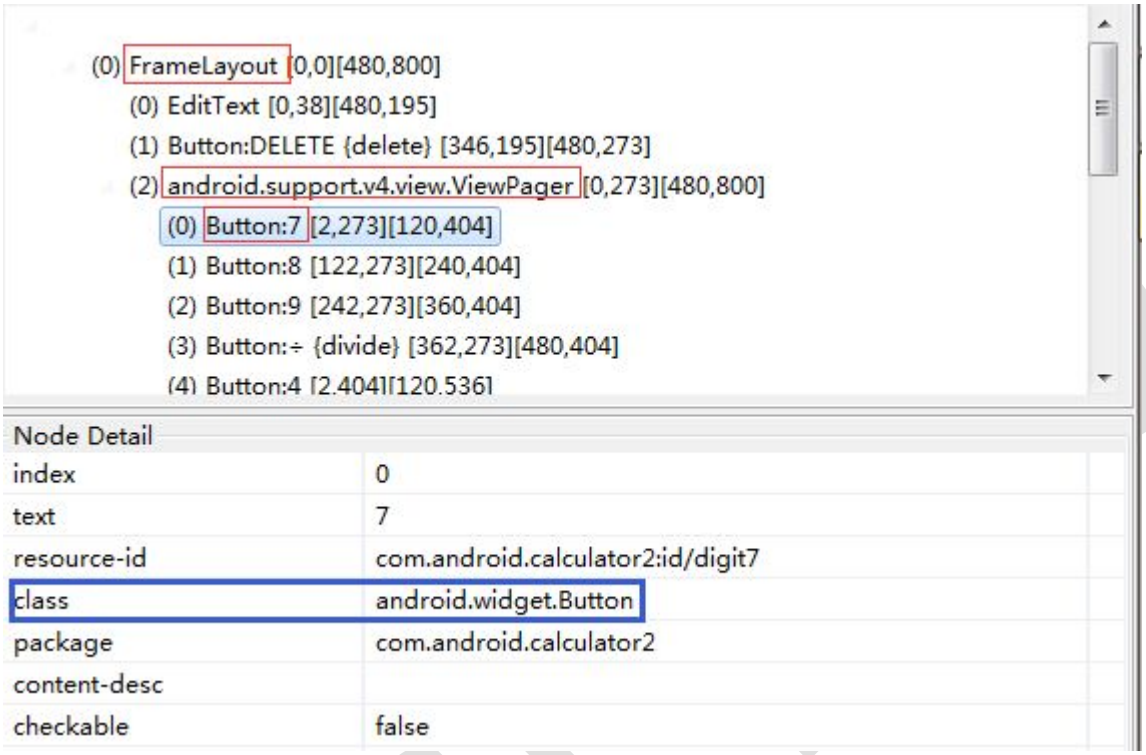
如何获得使用 xpath

testing

同样参考 UIAutomatorViewer.bat 工具。

XPath 格式变化

从老版本的 Appium0.18.x 升级到现在的 Appium1.x 后，xpath 策略的变化。对这种定位方法的理解还是费了我半天功夫的。



如上图，根据 web 上的经验，我理解是找层级标题的名字。所以写出来的路径是这样的：

driver.find_element_by_xpath(/FrameLayout/android.support.v4.view.ViewPager/Button:7)

但其实取的是控件的 class 而非标题的明字。所以正确的写法如下：

driver.find_element_by_xpath("/android.widget.FrameLayout/android.widget.Button").click() #7

只要记好这里的 class name 就是 web 里的 tag name 就对了。

当果如果出现 class 相同的情况下可以用控件的属性值进行区分。

driver.find_element_by_xpath("//android.widget.Button[contains(@text,'7')]").click() #7

cal_test.py

……

```
driver.find_element_by_xpath("//android.widget.Button[contains(@text,'8')]").
click()  #8

driver.find_element_by_xpath("//android.widget.Button[contains(@content-desc,
'multiply')]").click() #*

driver.find_element_by_xpath("//android.widget.Button[contains(@text,'5')]").
click()  #5
```

```
driver.find_element_by_xpath("//android.widget.Button[contains(@content-desc,
'equals')]").click() #=
```

.....

虽然在 webdriver 中 xpath 是非常灵活和强大的定位方法，但在 appium 中 xpath 从 Appium1.x 后需要写全路径，又臭又长，而且稳定性不高，所以不太推荐使用。

七、其它定位

除了前面介绍了定位方法外，还有以下方法：

find_element_by_tag_name()
find_element_by_link_text()
find_element_by_partial_link_text()
find_element_by_css_selector()

这些方法应该是针对 WebView 容器下面的控件定位的，因为现在针对的是 Native App 暂时还没有用到，所以先标记下，后面需要的时候加上去。

定位方法参考：http://testerhome.com/topics/1970

第三章 Appium API

在浏览器 web 页面常会用到输入、点击、鼠标悬停、清除等操作方法。相比较之下，在 android 上的交互会更加丰富，如：锁定/解锁、滑屏、摇晃、重启等，那么这些操作如何实现呢？下面我们就来学习这些操作的方法。

第一节 如何学习 Appium API

因为 appium 相对来说是一个比较新自动化测试框架。能学习的资料相对有限，国内搜索到的大多资料止步于环境搭建。Testerhome 在这方面积累了不少学习资料。别外一个就是看官方文档。

Testerhome：http://testerhome.com/

itest.info

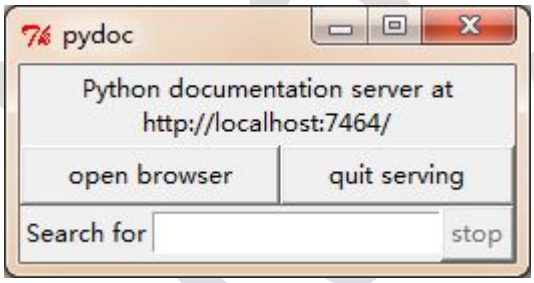
Appium doc：http://appium.io/documentation.html

appium API：http://appium.io/slate/cn/master/

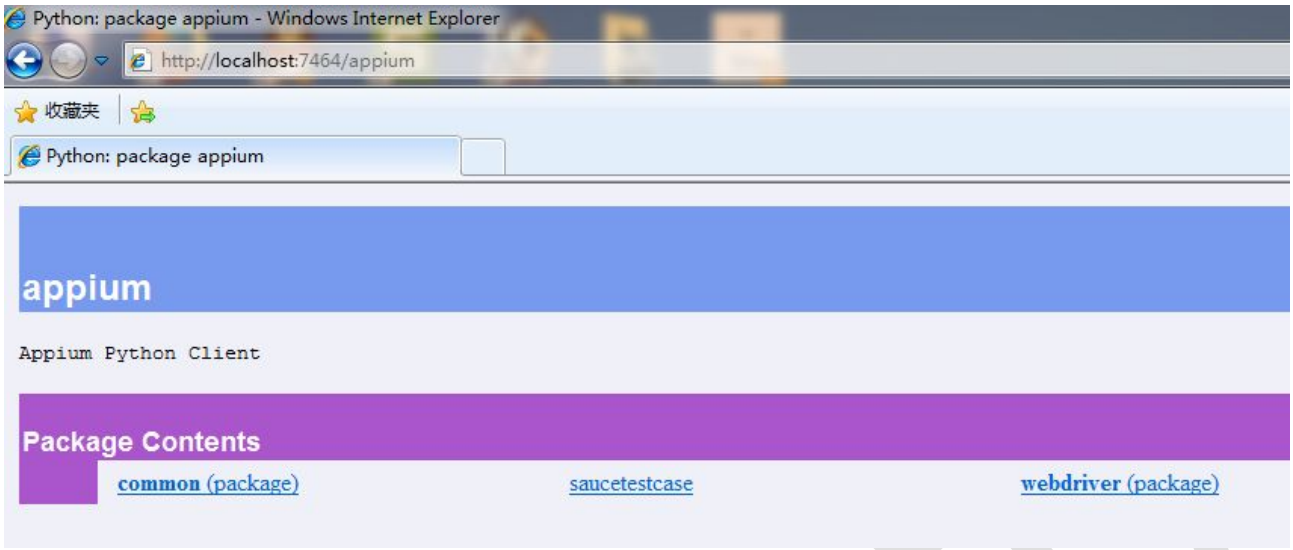
除此之前我个人习惯结合 appium client 框架的代码注释来学习。以 Python 为例。在开始菜单中找到 Python 目录下的 Module Docs 选项。



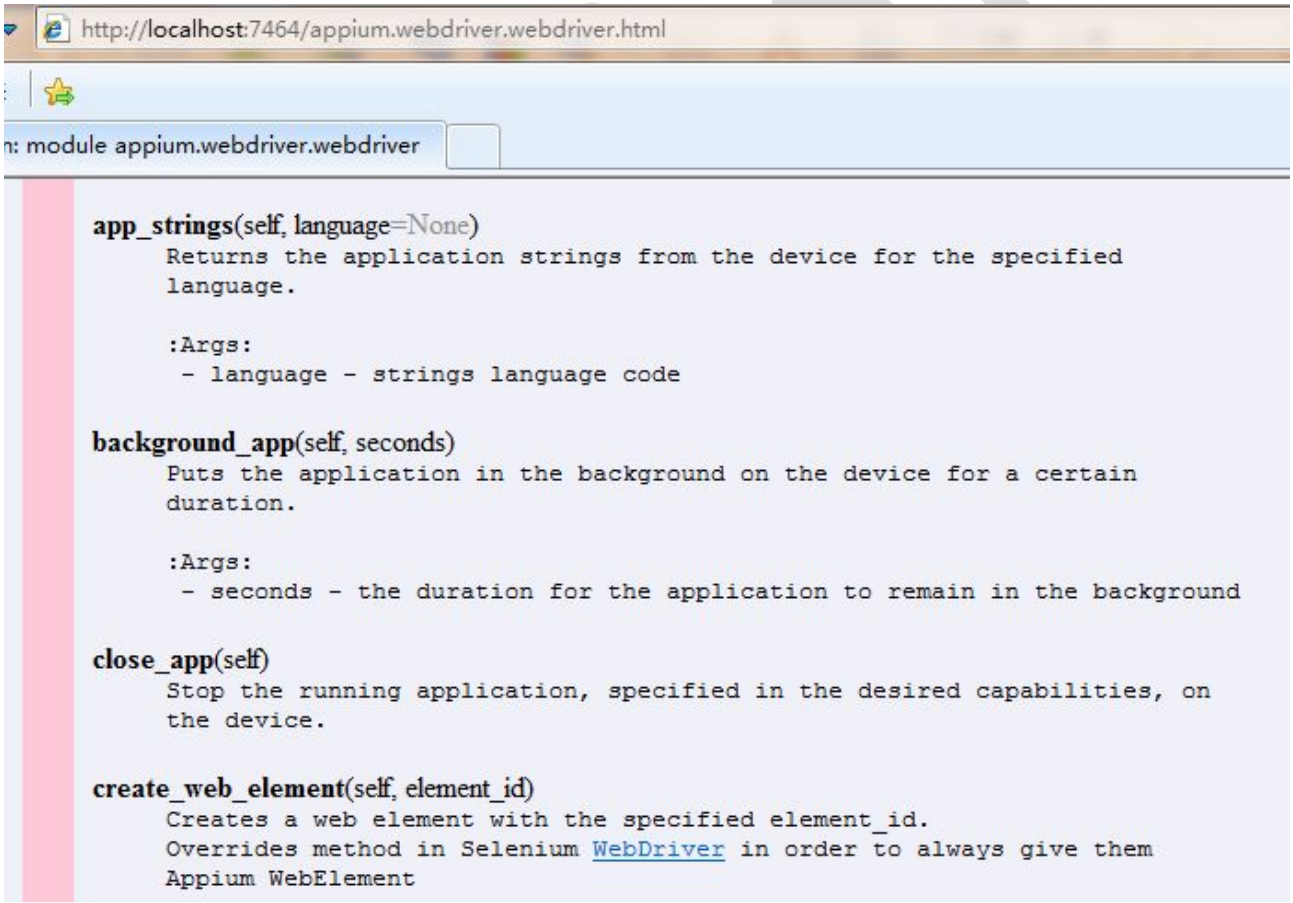
然后，会弹出下面的窗口。



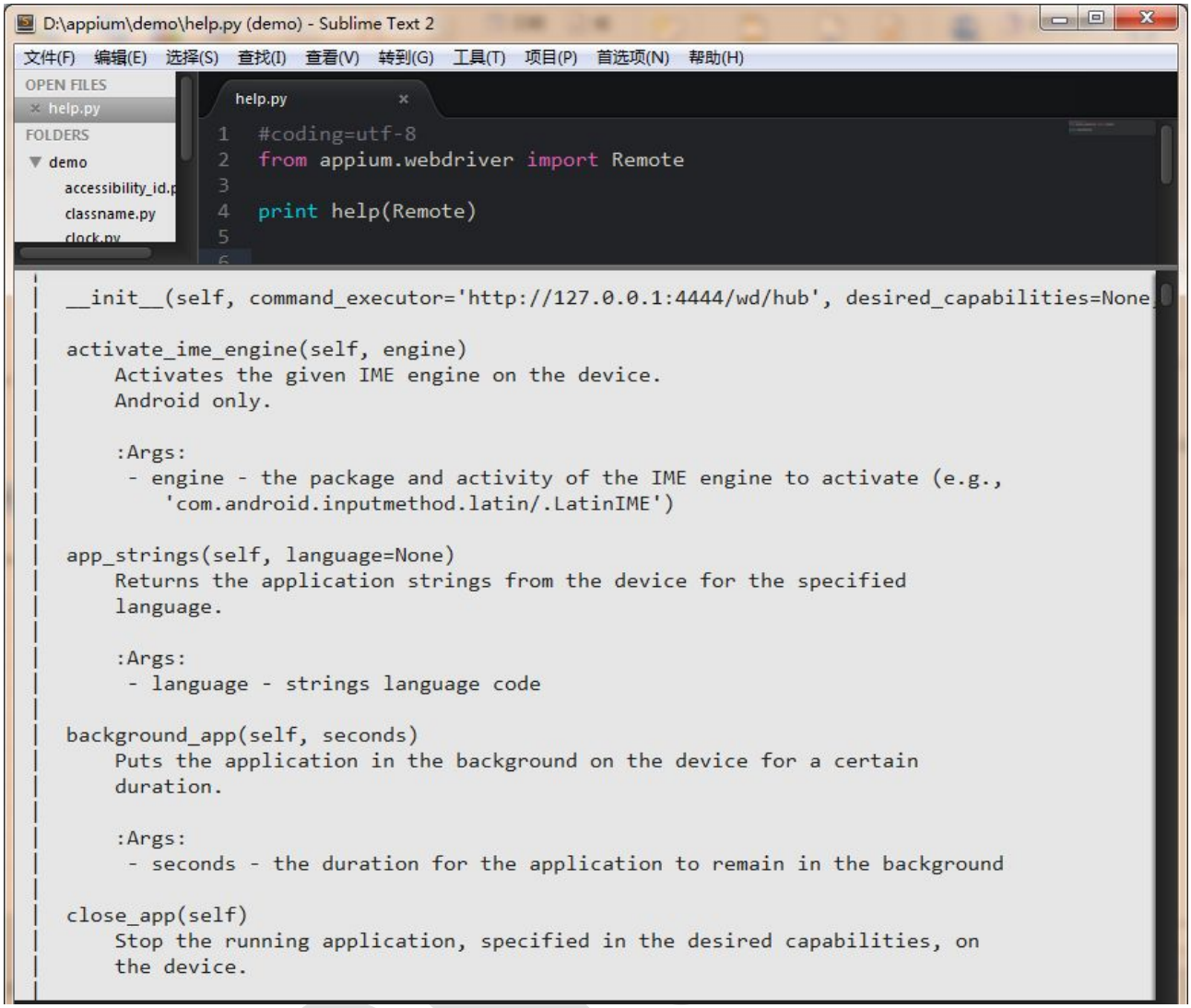
点击“open browser”，访问：http://localhost:7464/appium



一步一步点进去就会看到 appium 框架中所实现的类了方法。



如果你想看某个类所实现了哪些方法，也可以通过 Python 的 help 方法。



当然，这些都是辅助学习的方式，真的要学会 appium API 就要不断的尝试，不断的练习，才能真正理解每一个方法的作用与用法。

第二节 应用操作

本小节的所罗列的方法主要针对应用的操作，如应用的安装、卸载、关闭、启动等。

一、安装应用

方法：

driver.install_app()

安装应用到设备中去。需要 apk 包的路径。

例：

driver.install_app('path/to/my.apk')

driver.install_app("D:\\android\\apk\\ContactManager.apk")

二、卸载应用

方法：

driver.remove_app()

从设备中删除一个应用

driver.remove_app('com.example.android.apis')

三、关闭应用

方法：

driver.close_app()

关闭打开的应用，默认关闭当前打开的应用，所以不需要入参。

四、检查应用是否安装

方法：

driver.is_app_installed()

检查应用是否已经安装。需要传参应用包的名字。返回结果为 Ture 或 False。

例：

driver.is_app_installed('com.example.android.apis')

五、启动应用

方法：

driver.launch_app()

启动应用。你一定很迷惑，不是在初始化的配置信息已经指定了应用，脚本运行的时候就需要启动应用，为什么还要有这个方法法去启动应用。这个方法是配置 close_app()使用的。

例：

driver.close_app()

driver.launch_app()

i.test.info

六、关闭应用

方法：

driver.close_app()

关闭应用。这个方法与 quit()有所不同，quit()是在结果测试时执行的，这个方法并非真正的关闭应用，相当于按 home 键将应用置于后台，可以通过 launch_app()再次启动。

七、将应用置于后台

方法：

driver.background_app()

将当前应用置于后台。这个方法需要入参，需要指定将哪个应用置于后台。

例：

driver.background_app("com.android.calculator2")

八、应用重置

方法：

driver.reset()

重置应用。相当“恢复出厂默认值”的效果。这个方法不需要入参。

第三节 上下文操作

啥是上下文，不太好解释。下面来自百度百科。

> 上下文
>
> 上下文是从英文 context 翻译过来，指的是一种环境。
>
> 在软件工程中，上下文是一种属性的有序序列，它们为驻留在环境内的对象定义环境。在对象的激活过程中创建上下文，对象被配置为要求某些自动服务，如同步、事务、实时激活、安全性等等。又比如计算机技术中，相对于进程而言，上下文就是进程执行时的环境。具体来说就是各个变量和数据，包括所有的寄存器变量、进程打开的文件、内存信息等。

其实上下文的操作主要针对于混合应用。啥是混合应用，简单来说就是 APP 用里面嵌入网页。Android 上的浏览器就属于混合应用。

itest.info

一、可用上下文

方法：

driver.contexts

获取当前所有的可用的上下文。注意这个方不要括号“()”，不要入参。

例：

```
ct = driver.contexts
print ct
```

----------启动自带浏览器的打印结果----------

[u'NATIVE_APP', u'WEBVIEW_1']

二、当前上下文

方法：

driver.current_context

获取当前的可用的上下文。注意这个方不要括号“()”，不要入参。

例：

```
cct = driver.current_context
print    cct
```

----------启动自带浏览器的打印结果----------

WEBVIEW_1

三、切换上下文

driver.switch_to.context()

切换到特定的上下文中。

例：

```
driver.switch_to.context('NATIVE_APP')
driver.switch_to.context('WEBVIEW_1')
```

itest.info

四、实例

因为对于同样一个定位方法，在 app 和在 web 中的查找对象是不一样的。对于执行移动 web 的操作就需要把上下文切换为“WEBVIEW_1”，而对于 app 的操作就切换将上下文切换为“NATIVE_APP”。

test.py

```
#coding=utf-8
from appium import webdriver

desired_caps = {}
desired_caps = {
        'platformName': 'Android',
    'platformVersion': '4.4.2',
    'deviceName': 'Android Emulator',
    'browserName': 'Browser',
}

driver = webdriver.Remote('http://127.0.0.1:4723/wd/hub', desired_caps)

print "start  baidu page!"
driver.get('https://www.baidu.com')

#获取当前上下文
cct = driver.current_context
print  cct

#切换为 web
driver.switch_to.context('WEBVIEW_1')

driver.find_element_by_id("kw").send_keys("appium")
driver.find_element_by_id("su").click()

driver.quit()
```

第四节 键盘操作

模拟键盘输入也是非常重要的操作。这一小节来介绍那些关于键盘的操作。

一、send_keys()方法

经过验证通过 webdriver 中 send_keys()方法可以正常模拟输入。如下面对通讯录的操作。

test.py

```
#coding=utf-8
from appium import webdriver

desired_caps = {}
desired_caps['platformName'] = 'Android'
desired_caps['platformVersion'] = '4.4.2'
desired_caps['deviceName'] = 'Android Emulator'
desired_caps['appPackage'] = 'com.example.android.contactmanager'
desired_caps['appActivity'] = '.ContactManager'

driver = webdriver.Remote('http://127.0.0.1:4723/wd/hub', desired_caps)

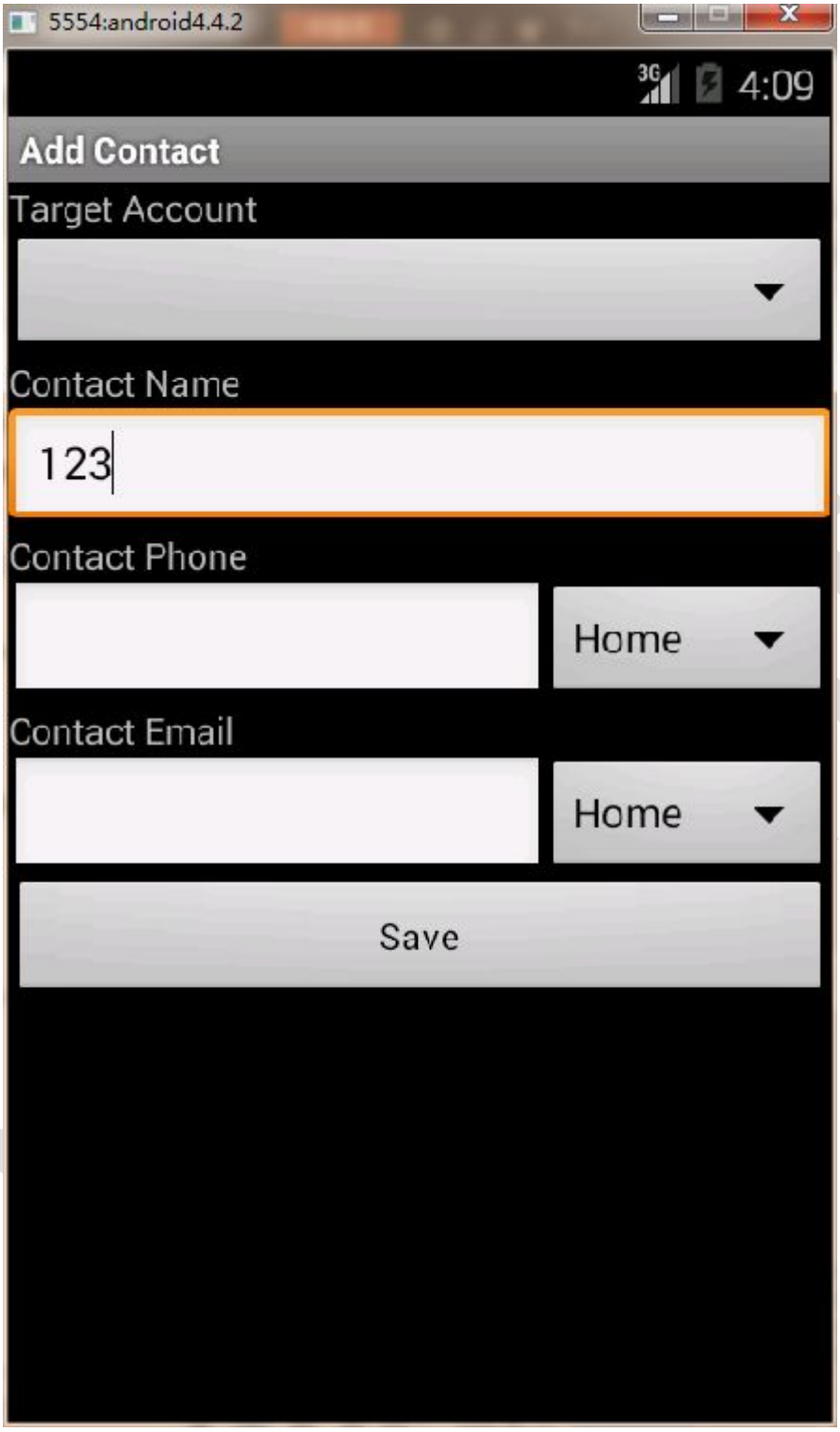
driver.find_element_by_name("Add Contact").click()

#通过 send_key()模拟键盘输入
driver.find_element_by_id("com.example.android.contactmanager:id/contactName
EditText").send_keys('123')

driver.quit()
```

执行效果如下图所示：

itest.info



二、keyevent()方法(Android only)

除此之外，appium 扩展提供了 keyevent()方法。

方法：

driver.keyevent()

发送一个键码的操作。需要一个入参。

例：

test.py

```
#coding=utf-8
from appium import webdriver

desired_caps = {}
desired_caps['platformName'] = 'Android'
desired_caps['platformVersion'] = '4.4.2'
desired_caps['deviceName'] = 'Android Emulator'
desired_caps['appPackage'] = 'com.example.android.contactmanager'
desired_caps['appActivity'] = '.ContactManager'

driver = webdriver.Remote('http://127.0.0.1:4723/wd/hub', desired_caps)

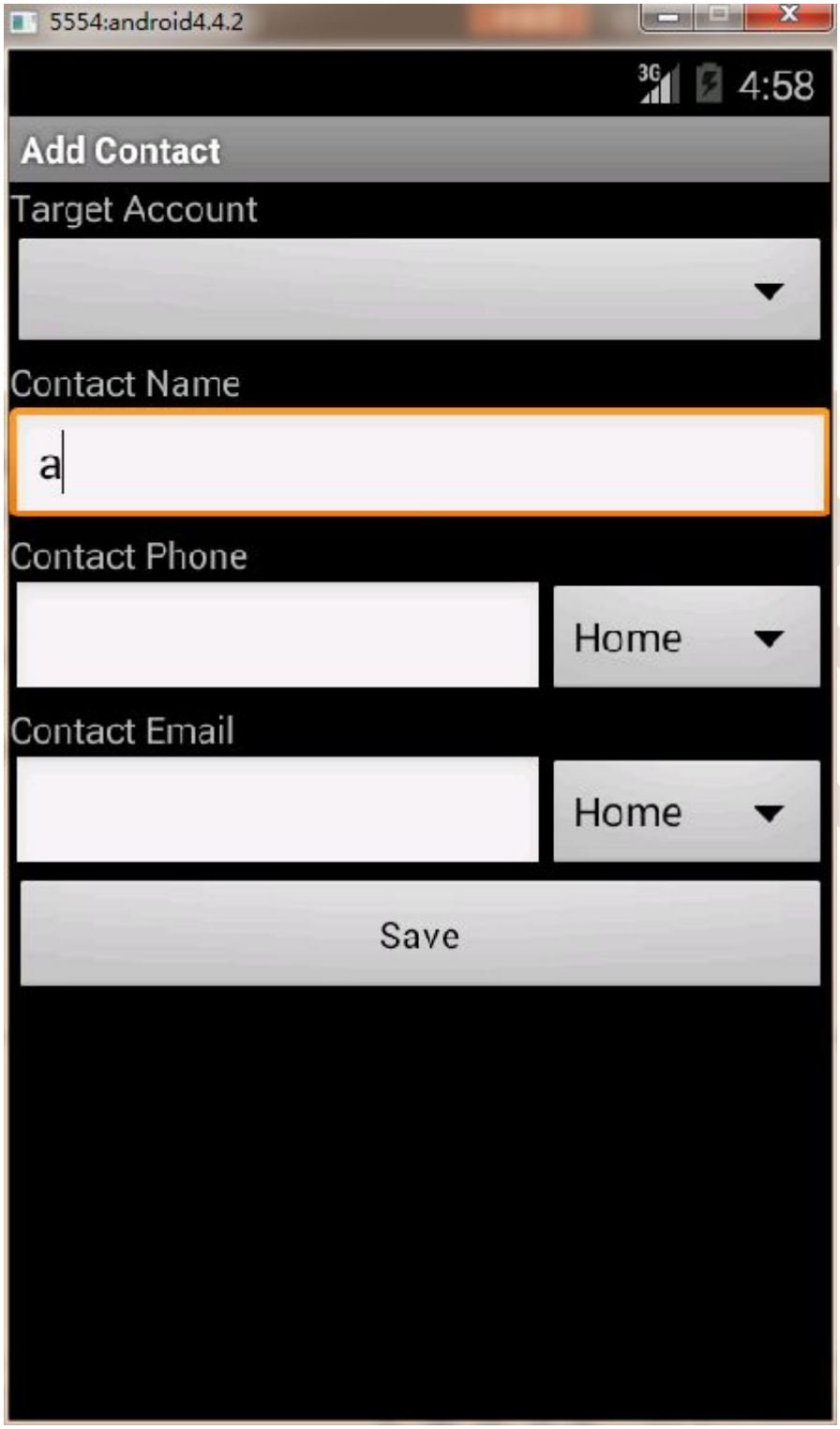
driver.find_element_by_name("Add Contact").click()

#先点击要输入的元素
driver.find_element_by_id("com.example.android.contactmanager:id/contactName
EditText").click()
#通过 keyevent()模拟键盘输入
driver.keyevent(29) #a

driver.quit()
```

执行效果如下图。

itest.info



在输通过 keyevent()输入之前，先通过 click()点击要输入的控件。而且它每次只能输入一个字符。最关键的问题是，我怎么知道 29 代表的字符“a”。

下面提供 keycode 参考表：

电话键

KEYCODE_CALL 拨号键 5
KEYCODE_ENDCALL 挂机键 6
KEYCODE_HOME 按键 Home 3

KEYCODE_MENU 菜单键 82
KEYCODE_BACK 返回键 4
KEYCODE_SEARCH 搜索键 84
KEYCODE_CAMERA 拍照键 27
KEYCODE_FOCUS 拍照对焦键 80
KEYCODE_POWER 电源键 26
KEYCODE_NOTIFICATION 通知键 83
KEYCODE_MUTE 话筒静音键 91
KEYCODE_VOLUME_MUTE 扬声器静音键 164
KEYCODE_VOLUME_UP 音量增加键 24
KEYCODE_VOLUME_DOWN 音量减小键 25

控制键

KEYCODE_ENTER 回车键 66
KEYCODE_ESCAPE ESC 键 111
KEYCODE_DPAD_CENTER 导航键 确定键 23
KEYCODE_DPAD_UP 导航键 向上 19
KEYCODE_DPAD_DOWN 导航键 向下 20
KEYCODE_DPAD_LEFT 导航键 向左 21
KEYCODE_DPAD_RIGHT 导航键 向右 22
KEYCODE_MOVE_HOME 光标移动到开始键 122
KEYCODE_MOVE_END 光标移动到末尾键 123
KEYCODE_PAGE_UP 向上翻页键 92
KEYCODE_PAGE_DOWN 向下翻页键 93
KEYCODE_DEL 退格键 67
KEYCODE_FORWARD_DEL 删除键 112
KEYCODE_INSERT 插入键 124
KEYCODE_TAB Tab 键 61
KEYCODE_NUM_LOCK 小键盘锁 143
KEYCODE_CAPS_LOCK 大写锁定键 115
KEYCODE_BREAK Break/Pause 键 121
KEYCODE_SCROLL_LOCK 滚动锁定键 116
KEYCODE_ZOOM_IN 放大键 168
KEYCODE_ZOOM_OUT 缩小键 169

组合键

KEYCODE_ALT_LEFT Alt+Left
KEYCODE_ALT_RIGHT Alt+Right
KEYCODE_CTRL_LEFT Control+Left
KEYCODE_CTRL_RIGHT Control+Right
KEYCODE_SHIFT_LEFT Shift+Left
KEYCODE_SHIFT_RIGHT Shift+Right

基本

KEYCODE_0 按键'0' 7
KEYCODE_1 按键'1' 8

itest.info

KEYCODE_2 按键'2' 9
KEYCODE_3 按键'3' 10
KEYCODE_4 按键'4' 11
KEYCODE_5 按键'5' 12
KEYCODE_6 按键'6' 13
KEYCODE_7 按键'7' 14
KEYCODE_8 按键'8' 15
KEYCODE_9 按键'9' 16
KEYCODE_A 按键'A' 29
KEYCODE_B 按键'B' 30
KEYCODE_C 按键'C' 31
KEYCODE_D 按键'D' 32
KEYCODE_E 按键'E' 33
KEYCODE_F 按键'F' 34
KEYCODE_G 按键'G' 35
KEYCODE_H 按键'H' 36
KEYCODE_I 按键'I' 37
KEYCODE_J 按键'J' 38
KEYCODE_K 按键'K' 39
KEYCODE_L 按键'L' 40
KEYCODE_M 按键'M' 41
KEYCODE_N 按键'N' 42
KEYCODE_O 按键'O' 43
KEYCODE_P 按键'P' 44
KEYCODE_Q 按键'Q' 45
KEYCODE_R 按键'R' 46
KEYCODE_S 按键'S' 47
KEYCODE_T 按键'T' 48
KEYCODE_U 按键'U' 49
KEYCODE_V 按键'V' 50
KEYCODE_W 按键'W' 51
KEYCODE_X 按键'X' 52
KEYCODE_Y 按键'Y' 53
KEYCODE_Z 按键'Z' 54

rest.info

第五节 TouchAction 操作

这是一个生成触摸动作的接口。这部分文档很快将会补充更多的内容进来。

一、按压控件

方法：

press()

开始按压一个元素或坐标点（x,y）。其实就是通过手指按压手机屏幕的某个位置。

press(self, el=None, x=None, y=None)

此方法默认需要三个参数，点击的元素，触摸屏的 x 坐标与 y 作标。三个参数都非必传。

例：

test.py

```
#coding=utf-8
from appium import webdriver
from appium.webdriver.common.touch_action import TouchAction

desired_caps = {}
desired_caps['platformName'] = 'Android'
desired_caps['platformVersion'] = '4.4.2'
desired_caps['deviceName'] = 'Android Emulator'
desired_caps['appPackage'] = 'com.example.android.contactmanager'
desired_caps['appActivity'] = '.ContactManager'

driver = webdriver.Remote('http://127.0.0.1:4723/wd/hub', desired_caps)

driver.find_element_by_name("Add Contact").click()

#定位要按压的元素
el =
driver.find_element_by_id("com.example.android.contactmanager:id/contactPhon
eEditText")

#执行按压操作
TouchAction(driver).press(el).release().perform()

#通过 keyevent()模拟键盘输入
driver.keyevent(29) #a

driver.quit()
```

其实，只传参元素（el）或坐标（x,y）也是可以的。例：

TouchAction(driver).press(el).release().perform()

TouchAction(driver).press(x=0,y=308).release().perform()

除了 press()方法之外，本例中还用到了别外两个新方法。

relese()

结束的行动取消屏幕上的指针。

itest.info

Perform()

执行的操作发送到服务器的命令操作。

二、长按控件

方法：

long_press()

开始按压一个元素或坐标点（x,y）。

long_press(self, el=None, x=None, y=None, duration=1000)

相比 press()方法，long_press()多了一个入参，既然晃长按，得有按的时间吧。duration 以毫秒为单位。1000 表示按一秒钟。其用法与 press()方法相同。

例：

TouchAction(driver).long_press(el).release().perform()

TouchAction(driver).long_press(x=0,y=308).release().perform()

TouchAction(driver).long_press(el,duration=2000).release().perform()

三、点击控件

方法：

tap()

对一个元素或控件执行点击操作。

tap(self, element=None, x=None, y=None, count=1)

相比上面两个方法，多了 count 的入参。个人猜测是点击的控件的次数，默认为 1 次。用法参考 press()。

例：

TouchAction(driver).tap(el).release().perform()

TouchAction(driver).tap(x=0,y=308).release().perform()

TouchAction(driver).tap(el,count=2).release().perform()

四、移动

方法：

move_to()

itest.info

将指针（光标）从过去指向指定的元素或点。

move_to(self, el=None, x=None, y=None)

其用法参考 press()方法。

例：

TouchAction(driver).move_to(el).release().perform()

TouchAction(driver).move_to(x=0,y=308).release().perform()

五、暂停

方法：

wait()

暂停脚本的执行，单位为毫秒。

wait(self, ms=0)　其用法参考 press()的用法。

例：

TouchAction(driver).wait(1000).release().perform()

TouchAction(driver).wait(5000).release().perform()

第六节 其它操作

其它操作针对移动设备上特有的一些操作。

一、锁定（iOS only）

方法：

driver.lock()

在一段时间内锁定手机，准确的讲就是按手机上的电源键熄屏一段时间。只作用于 iOS 设备。当前然并不是说这个方法在 android 产不起作用，经过我的验证锁定熄屏之后无法点亮屏幕。

例：

driver.lock(5)　　#锁定屏幕 5 秒种。

二、摇晃

itest.info

方法：

driver.shake()

摇晃手机。我能想到的两个应用就是微信摇一摇，另外一个就是 360 清理小球，摇动手机可以清理手机内存。

但是，当我执行这个方法时却报以下错误。

Not yet implemented. Please help us: http://appium.io/get-involved.html

什么情况，未实现，appium client 中提供了这个方法。Appium 中却未实现，那我们期待一下这个方法的实现吧。

三、当前 Activity（Android only）

方法：

driver.current_activity

得到当前应用的 activity。只适用于 android。此方法不需要括号“()”或入参。

例（计算器）：

cct = driver.current_activity

print cct

----------输出结果为----------

.Calculator

四、收起键盘（iOS only）

方法：

driver.hide_keyboard()

在 iOS 上收起键盘。不需要入参。

五、滑动

方法：

driver.swipe(75, 500, 75, 0, 0.8)

模拟用户滑动。将控件或元素从一个位置（x,y）拖动到另一个位置（x,y）。

itest.info

swipe(self, start_x, start_y, end_x, end_y, duration=None)

start_x：开始滑动的 x 坐标。

start_y：开始滑动的 y 坐标。

end_x：结束滑动的 x 坐标。

end_y：结束滑动的 y 坐标。

duration：持续时间。

例：

driver.swipe(75, 500, 75, 0, 0.8)

六、Pinch

方法：

driver.pinch()

在 0% 到 100% 内双指缩放屏幕。

pinch(self, element=None, percent=200, steps=50)

例：

driver.pinch(element=el)

七、Zoom

方法：

driver.zoom()

放大屏幕 在 100% 以上放大屏幕。

zoom(self, element=None, percent=200, steps=50)

例：

driver.zoom(element=el)

pinch 与 zoom 的区别：

我理解的 pinch 是这样的。

比如图片 Test.jpg 它的大小是 800x600， 那么从 0x0 到 800x600 里面的缩放是 pinch。超过 800x600 的缩放是 zoom。

itest.info

八、拉出文件

方法：

driver.pull_file()

从设备中拉出文件。

pull_file(self, path)

例：

driver.pull_file('Library/AddressBook/AddressBook.sqlitedb')

九、推送文件

方法：

driver.pull_file()

推送文件到设备中去

push_file(self, path, base64data)

例：

data = "some data for the file"

path = "/data/local/tmp/file.txt"

driver.push_file(path, data.encode('base64'))

itest.info

第四章　Appium 参考

虽然在前面章节已经学习了 Appium 自动化测试环境的搭建，以及各种操作 API，有些地方交代的并不详细。本章将 Appium 的一些细节进行介绍。

第一节 Appium 服务关键字

前面章节中学了 appium 脚本初始化 Desired Capabilities 的配置。但 Desired Capabilities 中到底可以进行哪些参数的配置，交代的并不详尽，这一节将罗列这些关键字。

关键字	描述	实例
automationName	你想使用的自动化测试引擎	Appium （默认）或 Selendroid
platformName	你要测试的手机操作系统	iOS, Android, 或 FirefoxOS
platformVersion	手机操作系统版本	例如： 7.1, 4.4
deviceName	使用的手机类型或模拟器类型	iPhone Simulator, iPad Simulator, iPhone Retina 4-inch, Android Emulator, Galaxy S4, 等。在 iOS 上，这个关键字的值必须是使用 instruments -s devices 得到的可使用的设备名称之一。在 Android 上，这个关键字目前不起作用。
app	.ipa or .apk 文件所在的本地绝对路径或者远程路径,也可以是一个包括两者之一的.zip。 Appium 会先尝试安装路径对应的应用在适当的真机或模拟器上。针对 Android 系统，如果你指定 app-package 和 app-activity(具体见下面)的话,那么就可以不指定 app。 **会与 browserName 冲突**	比如/abs/path/to/my.apk 或 http://myapp.com/app.ipa
browserName	需要进行自动化测试的手机 web 浏览器名称。如果是对应用进行自动化测试，这个关键字的值应为空。	iOS 系统上可以用 'Safari'，Android 系统上可以用 'Chrome', 'Chromium', 或 'Browser'。
newCommandTimeout	设置命令超时时间，单位：秒。达到超时时间仍未接收到新的命令时 Appium 会假设客户端退出然后自动结束会话。	比如 60
autoLaunch	Appium 是否需要自动安装和启动应用。默认值 true	true, false

关键字	描述	实例
language	(Sim/Emu-only) 设定模拟器 (simulator / emulator) 的语言。	如： fr
locale	(Sim/Emu-only) 设定模拟器 (simulator / emulator) 的区域设置。	如： fr_CA
udid	连接的物理设备的唯一设备标识	如： 1ae203187fc012g
orientation	(Sim/Emu-only) 在一个设定的方向模式中开始测试	LANDSCAPE (横向) 或 PORTRAIT (纵向)
autoWebview	直接转换到 WebView 上下文。 默认值 false、	true, false
noReset	不要在会话前重置应用状态。默认值 false。	true, false
fullReset	(iOS) 删除整个模拟器目录。(Android) 通过卸载——而不是清空数据——来重置应用状态。在 Android 上，这也会在会话结束后自动清除被测应用。默认值 false	true, false

Android 特有

关键字	描述	实例
appActivity	你要从你的应用包中启动的 Android Activity 名称。它通常需要在前面添加 . (如:使用.MainActivity 而不是 MainActivity)	MainActivity, .Settings
appPackage	你想运行的 Android 应用的包名	比如 com.example.android.myApp, com.android.settings
appWaitActivity	你想要等待启动的 Android Activity	SplashActivity

关键字	描述	实例
	名称	
deviceReadyTimeout	设置等待一个模拟器或真机准备就绪的超时时间	5
androidCoverage	用于执行测试的 instrumentation 类。作为命令 adb shell am instrument -e coverage true -w 的-w 参数。	com.my.Pkg/com.my.Pkg.instrumentation.MyInstrumentation
enablePerformanceLogging	(仅适用于 Chrome 和 webview) 开启 Chromedriver 的性能日志。(默认 false)	true, false
androidDeviceReadyTimeout	等待设备在启动应用后准备就绪的超时时间。以秒为单位。	如 30
androidDeviceSocket	开发工具的 socket 名称。只有在被测应用是一个使用 Chromium 内核的浏览器时需要。 socket 会被浏览器打开，然后 Chromedriver 把它作为开发者工具来进行连接。	如 chrome_devtools_remote
avd	需要启动的 AVD (安卓虚拟设备) 名称。	如 api19
avdLaunchTimeout	以毫秒为单位，等待 AVD 启动并连接到 ADB 的超时时间。(默认值 120000)	300000
avdReadyTimeout	以毫秒为单位，等待 AVD 完成启动动画的超时时间。(默认值 120000)	300000
avdArgs	启动 AVD 时需要加入的额外的参数。	如 -netfast
useKeystore	使用一个自定义的 keystore 来对 apk 进行重签名。默认值 false	true or false
keystorePath	自定义 keystore 的路径。默认：~/.android/debug.keystore	如 /path/to.keystore
keystorePassword	自定义 keystore 的密码。	如 foo

关键字	描述	实例
keyAlias	key 的别名	如 androiddebugkey
keyPassword	key 的密码	如 foo
chromedriverExecutable	webdriver 可执行文件的绝对路径（如果 Chromium 核心提供了对应的 webdriver， 应该用它代替 Appium 自带的 webdriver）	/abs/path/to/webdriver
autoWebviewTimeout	以毫秒为单位，等待 Webview 上下文激活的时间。默认值 2000	如 4
intentAction	用于启动 activity 的 intent action。（默认值 android.intent.action.MAIN）	如 android.intent.action.MAIN,android.intent.action.VIEW
intentCategory	用于启动 activity 的 intent category。（默认值 android.intent.category.LAUNCHER）	如 android.intent.category.LAUNCHER,android.intent.category.APP_CONTACTS
intentFlags	用于启动 activity 的标识（ flags ）（默认值 0x10200000）	如 0x10200000
optionalIntentArguments	用于启动 activity 的额外 intent 参数。请查看 Intent 参数	如 --esn <EXTRA_KEY>, --ez <EXTRA_KEY> <EXTRA_BOOLEAN_VALUE>
stopAppOnReset	在使用 adb 启动应用前停止被测应用的进程（ process ）。如果被测应用是被另一个应用创建的，当这个参数被设定为 false 时，允许另一个应用的进程在使用 adb 启动被测应用时继续存活。默认值 true	true 或 false
unicodeKeyboard	使用 Unicode 输入法。默认值 false	true 或 false

关键字	描述	实例
resetKeyboard	在设定了 unicodeKeyboard 关键字的 Unicode 测试结束后，重置输入法到原有状态。如果单独使用，将会被忽略。默认值 false	true 或 false
noSign	跳过检查和对应用进行 debug 签名的步骤。只能在使用 UiAutomator 时使用，使用 selendroid 是不行。默认值 false	true 或 false
ignoreUnimportantViews	调用 uiautomator 的函数 setCompressedLayoutHierarchy()。由于 Accessibility 命令在忽略部分元素的情况下执行速度会加快，这个关键字能加快测试执行的速度。被忽略的元素将不能够被找到，因此这个关键字同时也被实现成可以随时改变的 *设置（settings）* 。默认值 false	true 或 false

iOS 特有

关键字	描述	实例
calendarFormat	(Sim-only) 为 iOS 的模拟器设置日历格式	如 gregorian (公历)
bundleId	被测应用的 bundle ID 。用于在真实设备中启动测试，也用于使用其他需要 bundle ID 的关键字启动测试。在使用 bundle ID 在真实设备上执行测试时，你可以不提供 app 关键字，但你必须提供 udid 。	如 io.appium.TestApp
udid	连接的真实设备的唯一设备编号（Unique device identifier）	如 1ae203187fc012g
launchTimeout	以毫秒为单位，在 Appium 运行失败之前设置一个等待 instruments 的时间	比如： 20000

关键字	描述	实例
locationServicesEnabled	(Sim-only) 强制打开或关闭定位服务。默认值是保持当前模拟器的设定	true 或 false
locationServicesAuthorized	(Sim-only) 通过修改 plist 文件设定是否允许应用使用定位服务，从而避免定位服务的警告出现。默认值是保持当前模拟器的设定。请注意在使用这个关键字时，你同时需要使用 bundleId 关键字来发送你的应用的 bundle ID。	true 或者 false
autoAcceptAlerts	当 iOS 的个人信息访问警告（如 位置、联系人、图片）出现时，自动选择接受(Accept)。默认值 false。	true 或者 false
autoDismissAlerts	当 iOS 的个人信息访问警告（如 位置、联系人、图片）出现时，自动选择不接受(Dismiss)。默认值 false。	true 或者 false
nativeInstrumentsLib	使用原生 intruments 库（即关闭 instruments-without-delay ）	true 或者 false
nativeWebTap	(Sim-only) 在 Safari 中允许"真实的"，非基于 javascript 的 web 点击(tap)。 默认值： false。注意：取决于 viewport 大小/比例， 点击操作不一定能精确地点中对应的元素。	true 或者 false
safariInitialUrl	(Sim-only) (>= 8.1) 初始化 safari 的时使用的地址。默认是一个本地的欢迎页面	如 https://www.github.com
safariAllowPopups	(Sim-only) 允许 javascript 在 Safari 中创建新窗口。默认保持模拟器当前设置。	true 或者 false
safariIgnoreFraudWarning	(Sim-only) 阻止 Safari 显示此网站可能存在风险的警告。默认保持浏览器当前设置。	true 或者 false

关键字	描述	实例
safariOpenLinksInBackground	(Sim-only) Safari 是否允许链接在新窗口打开。默认保持浏览器当前设置。	true 或者 false
keepKeyChains	(Sim-only) 当 Appium 会话开始/结束时是否保留存放密码存放记录(keychains) (库(Library)/钥匙串(Keychains))	true 或者 false
localizableStringsDir	从哪里查找本地化字符串。默认值 en.lproj	en.lproj
processArguments	通过 instruments 传递到 AUT 的参数	如 -myflag
interKeyDelay	以毫秒为单位，按下每一个按键之间的延迟时间。	如 100
showIOSLog	是否在 Appium 的日志中显示设备的日志。默认值 false	true 或者 false
sendKeyStrategy	输入文字到文字框的策略。模拟器默认值：oneByOne(一个接着一个) 。真实设备默认值：grouped (分组输入)	oneByOne, grouped 或 setValue
screenshotWaitTimeout	以秒为单位，生成屏幕截图的最长等待时间。默认值： 10。	如 5
waitForAppScript	用于判断 "应用是否被启动" 的 iOS 自动化脚本代码。默认情况下系统等待直到页面内容非空。结果必须是布尔类型。	例 如 true;,target.elements().length > 0;, $.delay(5000); true;

第二节 Appium for Windows 说明

至于 appium.exe 的配置在前面环境搭建的过程中并没有过多的介绍。这里将对这些按钮及选项进行翻译。

启动界面



Android Button(机器人): 显示 Android 设置。

Settings Button（齿轮）: 显示的一般设置。

Developer Button（头像）: 显示开发人员设置。

About Button（问号）:显示 Appium 版本信息。

Inspector Button（放大镜）: 启动 Appium Inspector。

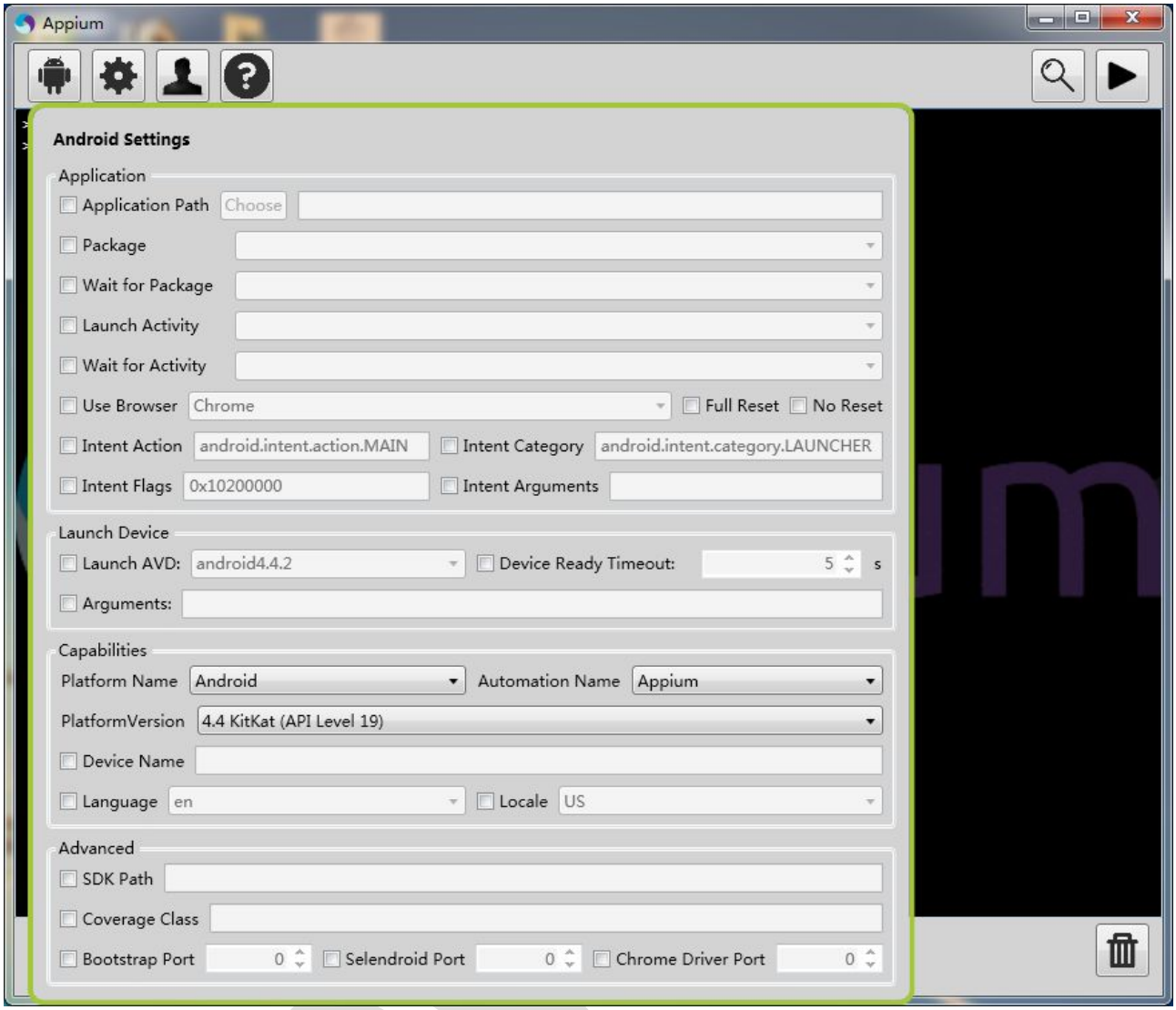
Launch / Stop Button（三角）: 启动或停止 Appium 服务器



Clear Button（垃圾桶）: 清除所有日志的显示输出。

Android Settings

itest.info



Application

App Path	Android 应用程序的路径(. apk)您希望测试。
Choose Button	用于您的应用程序的路径选择。
Package	Android 应用程序运行的 Java 包(例如 com.example.android.myApp)。
Wait for Package	包名的 Android 活动等。
Launch Activity	活动名称 Android 活动启动(e.g.MainActivity)从你的包。
Wait for Activity	活动名称 Android 活动等。
Full Reset	重启应用程序状态通过卸载应用程序而不是清除应用程序数据并删除完成后的应用程序会话。
No Reset	防止设备重置。
Use Browser	启动指定 Android 浏览器(例如 Chrome)。

Intent Action	意图的行动将被用于启动活动。
Intent Category	意图类别用来启动活动。
Intent Flags	旗帜,将用于启动活动。
Intent Arguments	额外的意图参数将用于启动活动。

Launch Device

Launch AVD	AVD 中推出的名称。
Device Ready Timeout	超时秒在等待设备成为准备好了。
Arguments	额外的模拟器参数启动 avd。

Capabilities

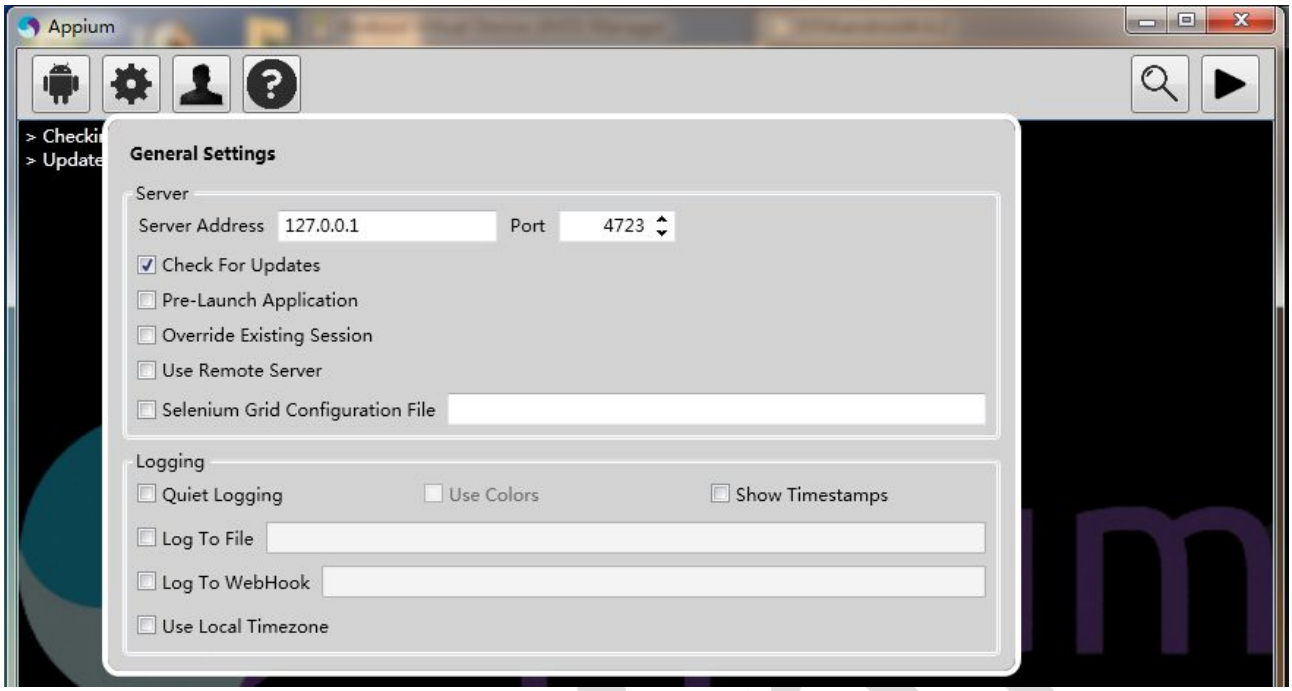
Platform Name	移动平台的名称。
Automation Name	自动化工具的名称(Appium 或 Selendroid)。
Platform Version	版本的移动平台。
Device Name	移动设备使用的名称。
Language	语言为 Android 设备。
Locale	Android 设备现场.

Advanced

SDK Path	Path to Android SDK.
Coverage Class	Fully qualified instrumentation class.
Bootstrap Port	设备上的端口使用 Appium 说话。
Selendroid Port	本地端口用于与 Selendroid 沟通。
Chromedriver Port	ChromeDriver 端口。

General Settings

itest.info



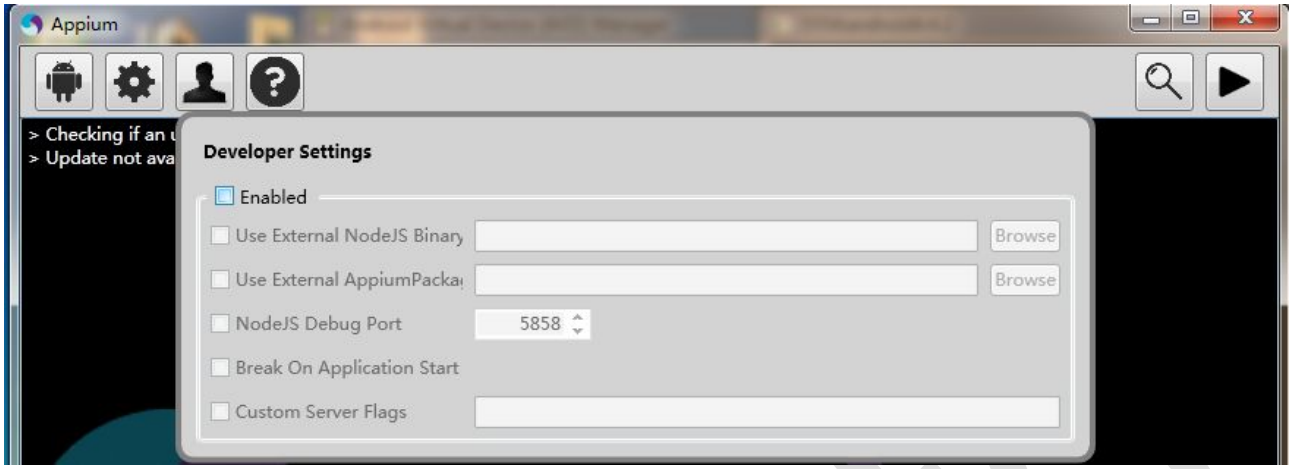
Server

Server Address	你想 Appium 服务器的 IP 地址(本地主机 127.0.0.1)运行。
Port	Appium 服务器将监听的端口 WebDriver 命令(4723 是默认的)。
Check For Updates	Appium 开始时将自动检查更新。
Pre-launch Application	Appium 将发射前的应用程序之前开始监听 WebDriver 命令。
Override Existing Session	将覆盖任何现有 Appium 会话
Use Remote Server	用于连接 Appium Inpector 服务器已经运行。
Selenium Grid Configuration File	selenium grid 路径配置文件。

Logging

Quiet Logging	不要使用详细日志输出。
Use Colors	在控制台输出中使用颜色。
Show Timestamps	在控制台输出显示时间戳。
Log to File	日志输出发送到这个文件中。
Log to WebHook	这个 HTTP 侦听器发送日志输出。

Developer Settings



Enabled:如果勾选此项,开发人员设置将被观察到。

Use External NodeJS Binary	Appium 将使用的版本在这里 NodeJS 提供附带的一个应用程序。
Use External Appium Package	Appium 将使用的版本 Appium 包提供这里不是一个与应用程序绑定。
NodeJS Debug Port	港口的 NodeJS 调试器将运行。
Break on Application Start	NodeJS 调试服务器将在应用程序开始。(相当于提供 debug-brk 开关节点)
Custom Server Flags	自定义旗帜开始 Appium 服务器时使用。这应该只用于如果有使用 Appium UI 选项,无法调整。

Inspector / Recorder
检查员可以访问通过点击启动按钮旁边的放大镜一旦 Appium 服务器已经启动了。Appium 必须运行一个应用程序打开督察工作。否则,它将无法工作。

http://testerhome.com/topics/2594

第五章　测试实例

前面的章节一直在讲基础知识，是时候利用前面所学到的知识做一些实例了。

第二节 Chrome 实例

目前在 appium 只支持 android 上 chrome，Chromium 等浏览器执行移动 web 应用，如何你想在 QQ \UC、360 等浏览器上执行用户，那只能抱歉了。

下面我们以 android 上的 chrome 浏览器为例进行测试。首先下载 chrome 浏览器 android 版本。



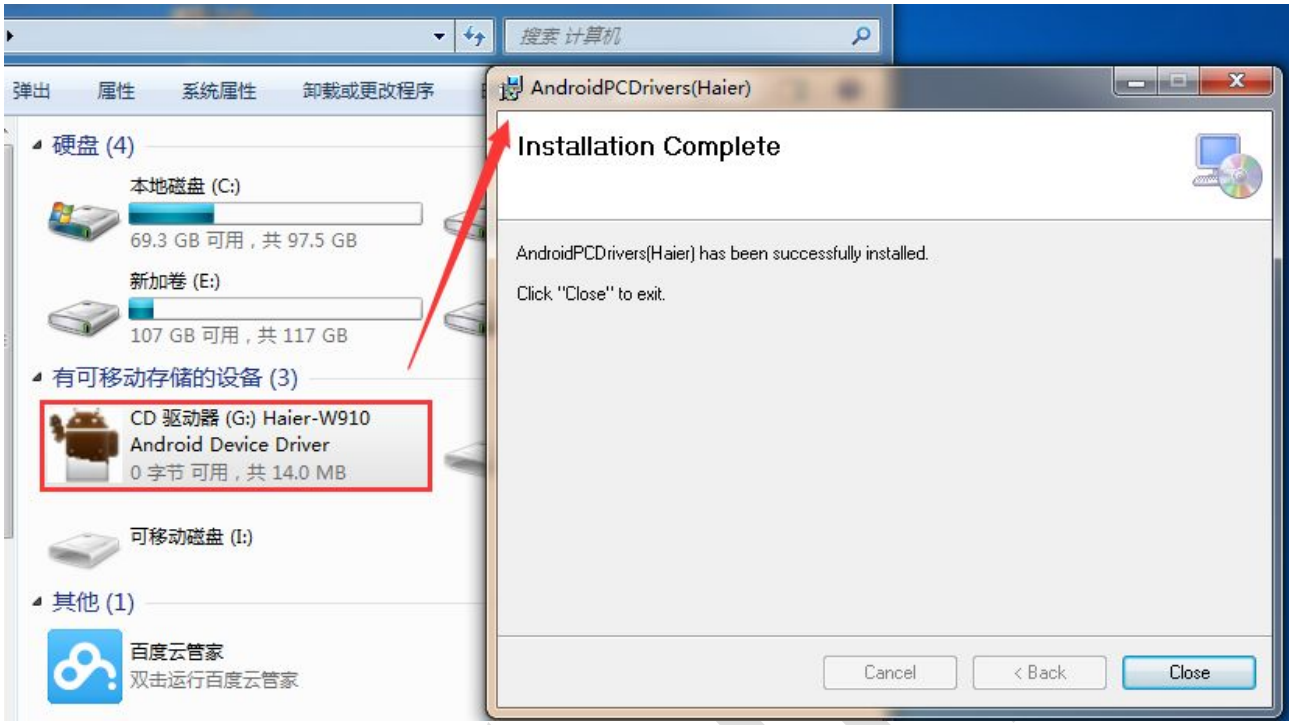
可以通过前面学习的方法安装应用。例：

driver.install_app("D:\\android\\apk\\Chrome_2357093.apk")

第三节 链接 android 真机测试

android 虚拟机自然有它的好处，但它也有缺点，就是用例的运行比较慢。如果你有一台 android 真机的话，不妨链接真机来进行测试。

首先，需要通过 USB 数据线将手机与电脑链接。Windows 7 会自动识别该手机，我们根据提示安装手机驱动。（注意手机要打开 USB 调试）

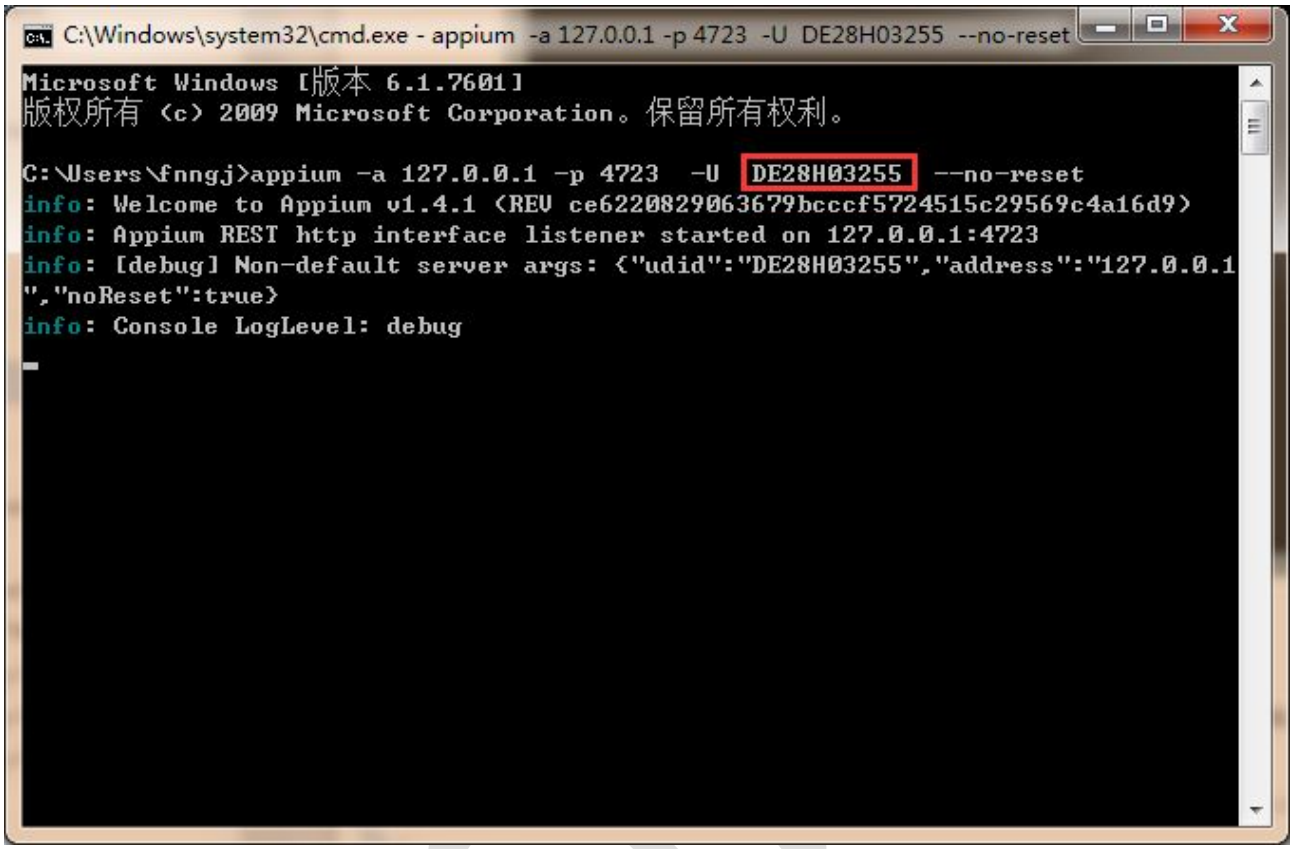


然后通过 adb 查看设备的 udid 号，在 cmd 命令提示符下输入以下命令：adb devices -l 。



启动 appium，前面的学习，我们一般是通过 appium.exe 启动 appium 的，在 cmd 命令提示符下同样可以启动 appium。如下：

appium -a 127.0.0.1 -p 4723 -U DE28H03255 --no-reset



注意，“DE28H03255”为 android 设备的 udid 号。如果出现以上信息，说明 appium 启动成功。



下面是启动 android 通讯录的配置。

setting_test.py

```
#coding=utf-8
from appium import webdriver
from time import sleep

desired_caps = {}
desired_caps['device'] = 'android'
desired_caps['platformName'] = 'Android'
desired_caps['platformVersion'] = '4.0.4'
desired_caps['deviceName'] = 'Haier HW-W910'
```

```
desired_caps['automationName'] = 'selendroid'
desired_caps['app'] = 'D:\\android\\apk\\ContactManager.apk'

driver = webdriver.Remote('http://localhost:4723/wd/hub', desired_caps)

sleep(5)

driver.quit()
……
```

可以通过手机的设置--->关于手机---->型号，来查看手机的型号。我的真机型号为：

```
'deviceName' = 'Haier HW-W910'
```

由于我真机的系统为：android 4.0.4 ，所以需要指定：

```
'automationName' = 'selendroid'
```

只要 android 系统版本小于 4.2.2（API 17），都需要指定 selendroid。

itest.info